

**RANCANG BANGUN SUBSISTEM OTOMATISASI PERANGKAT
IOT BERBASIS *WEBSITE*
(STUDI KASUS : *GREENHOUSE* KEBUN RAYA ITERA)**

TUGAS AKHIR

**MUHAMMAD YUSUF
122140193**



**PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNOLOGI INDUSTRI
INSTITUT TEKNOLOGI SUMATERA
LAMPUNG SELATAN
2025**

LEMBAR PENGESAHAN

Saya menyatakan bahwa Tugas Akhir berjudul “RANCANG BANGUN SUBSISTEM OTOMATISASI PERANGKAT IOT BERBASIS *WEBSITE* (STUDI KASUS : GREENHOUSE KEBUN RAYA ITERA)” merupakan hasil karya saya sendiri dan belum pernah diajukan, baik sebagian maupun seluruhnya, di Institut Teknologi Sumatera atau institusi pendidikan lain oleh saya maupun pihak lain.

Lampung Selatan, 9 Januari 2025

Penulis,

Muhammad Yusuf
NIM.122140193

Diperiksa dan disetujui oleh,
Pembimbing

1. Muhammad Habib Algifari, S.Kom., M.T.I.
NIP. 199105252022031002
2. Winda Yulita, M.Cs.
NIP. 199307272022032022

Penguji

1. Ilham Firman Ashari, S.Kom., M.T
NIP. 199303142019031018
2. Eko Dwi Nugroho, S.Kom., M.Cs.
NIP. 199102092024061001

Disahkan oleh,
Koordinator Program Studi Teknik Informatika
Fakultas Teknologi Industri
Institut Teknologi Sumatera

Andika Setiawan, S.Kom., M.Cs.
NIP. 199111272022031007

HALAMAN PERNYATAAN ORISINALITAS

Tugas Akhir dengan judul “RANCANG BANGUN SUBSISTEM OTOMATISASI PERANGKAT IOT BERBASIS WEBSITE (STUDI KASUS : *GREENHOUSE* KEBUN RAYA ITERA)” adalah karya saya sendiri, dan semua sumber baik yang dikutip maupun dirujuk telah saya nyatakan benar.

Nama : Muhammad Yusuf
NIM : 122140193

Tanda Tangan :
Tanggal :

HALAMAN PERNYATAAN PERSETUJUAN PUBLIKASI TUGAS AKHIR UNTUK KEPENTINGAN AKADEMIS

Sebagai civitas akademik Institut Teknologi Sumatera, saya yang bertanda tangan di bawah ini:

Nama : Muhammad Yusuf
NIM : 122140193
Program Studi : Teknik Informatika
Fakultas : Fakultas Teknologi Industri
Jenis Karya : Tugas Akhir

demi pengembangan ilmu pengetahuan, menyetujui untuk memberikan kepada Institut Teknologi Sumatera Hak Bebas Royalti *Noneksklusif (Non-exclusive Royalty Free Right)* atas karya ilmiah saya yang berjudul:

RANCANG BANGUN SUBSISTEM OTOMATISASI PERANGKAT IOT
BERBASIS *WEBSITE*
(STUDI KASUS : *GREENHOUSE* KEBUN RAYA ITERA)

beserta perangkat yang ada (jika diperlukan). Dengan Hak Bebas Royalti Noneksklusif ini Institut Teknologi Sumatera berhak menyimpan, mengalihmedia/formatkan, mengelola dalam bentuk pangkalan data (database), merawat, dan memublikasikan tugas akhir saya selama tetap mencantumkan nama saya sebagai penulis/pencipta dan sebagai pemilik Hak Cipta.

Demikian pernyataan ini saya buat dengan sebenarnya.

Dibuat di : Lampung Selatan

Pada tanggal : 9-Januari-2025

Yang Menyatakan
Muhammad Yusuf
NIM. 122140193

KATA PENGANTAR

Puji syukur kehadiran Allah SWT/Tuhan Yang Maha Esa atas limpahan rahmat, karunia, serta petunjuk-Nya sehingga penyusunan tugas akhir ini telah terselesaikan dengan baik. Dalam penyusunan tugas akhir ini penulis telah banyak mendapatkan arahan, bantuan, serta dukungan dari berbagai pihak. Oleh karena itu pada kesempatan ini penulis mengucapkan terima kasih kepada:

1. Prof. Dr. I Nyoman Pugeg Aryantha selaku Rektor Institut Teknologi Sumatera.
2. Hadi Teguh Yudistira, S.T., Ph.D. selaku Dekan Fakultas Teknologi Industri.
3. Andika Setiawan, S.Kom., M.Cs. selaku Ketua Program Studi Teknik Informatika.
4. Muhammad Habib Algifari, S.Kom., M.TI. selaku Dosen Pembimbing I atas ide, waktu, tenaga, perhatian, dan masukan yang telah disumbangsihkan kepada penulis pada kelancaran penulisan dan pengerjaan Tugas Akhir.
5. Winda Yulita, M.Cs. selaku Dosen Pembimbing II, atas bimbingan, arahan, motivasi, serta saran-saran yang sangat membantu dalam penyempurnaan penelitian dan penulisan skripsi ini.
6. Terimakasih kepada seluruh Dosen Penelitian *Smart Farming* ITERAHERO karena sudah mengajarkan banyak hal tentang dunia pertanian dan penelitian.
7. Rasa syukur yang mendalam penulis haturkan kepada kedua orang tua dan segenap keluarga, yang telah menjadi fondasi utama serta pemberi pelajaran paling berharga tentang kesabaran dan perjuangan, hingga penulis mampu sampai pada titik ini.
8. Terima kasih tulus untuk teman-teman terbaik yang selalu ada untuk saling mendukung, mengingatkan, dan menguatkan. Perjalanan hingga tahap akhir ini terasa lebih ringan berkat kehadiran, setiap kritik, dan diskusi yang telah dilalui bersama.
9. Terimakasih untuk penulis karena sudah mampu sampai di fase yang tidak mudah untuk dilalui, banyak pelajaran, tantangan, dan penghargaan yang sudah dilalui dengan penuh kekuatan, kegigihan, serta semangat dan aksi yang berjalan beriringan.

Akhir kata penulis berharap semoga tugas akhir ini dapat memberikan manfaat bagi kita semua dan para pembaca.

MOTTO PENULIS

“Keep Doing, Keep Learning, Keep Growing”

“Do the BEST, let GOD the REST”

~ Muhammad Yusuf ~

RINGKASAN

RANCANG BANGUN SUBSISTEM OTOMATISASI PERANGKAT IOT BERBASIS *WEBSITE* (STUDI KASUS : *GREENHOUSE* KEBUN RAYA ITERA)

Muhammad Yusuf

Pemanfaatan teknologi *Internet of Things* (IoT) dalam sektor pertanian modern, khususnya pada pengelolaan *greenhouse*, telah menjadi solusi vital untuk meningkatkan efisiensi operasional dan presisi perawatan tanaman. *Greenhouse* Kebun Raya Institut Teknologi Sumatera (ITERA) sebagai fasilitas konservasi dan penelitian saat ini menghadapi tantangan dalam pengelolaan mikroklimat dan irigasi yang masih bergantung pada mekanisme manual. Keterbatasan pemantauan kondisi lingkungan secara *real-time* dan keterlambatan respons terhadap fluktuasi suhu maupun kelembapan berpotensi menghambat pertumbuhan tanaman. Berangkat dari permasalahan tersebut, penelitian ini bertujuan untuk merancang dan membangun subsistem otomatisasi perangkat IoT yang terintegrasi dalam sebuah *website* manajemen terpusat, guna menjamin ketepatan kontrol perangkat dan kemudahan operasional bagi pengelola.

Metodologi pengembangan sistem yang diterapkan dalam penelitian ini mengacu pada kerangka kerja *Agile Kanban*, yang memungkinkan proses pengerjaan fitur dilakukan secara adaptif dan bertahap. Arsitektur sistem dirancang menggunakan protokol komunikasi MQTT (*Message Queuing Telemetry Transport*) dengan standar *Quality of Service* (QoS) Level 1 untuk menjamin reliabilitas pengiriman pesan instruksi (*command*) tanpa risiko kehilangan data (*packet loss*). Fokus utama pengembangan mencakup integrasi logika otomatisasi pada sisi *backend* serta pembangunan antarmuka pengguna (*frontend*) yang responsif menggunakan teknologi *WebSockets* untuk memfasilitasi interaksi dengan perangkat sensor dan aktuator di lapangan.

Hasil implementasi menunjukkan bahwa subsistem mampu mengakomodasi tiga mekanisme otomatisasi utama yang krusial bagi operasional *greenhouse*. Pertama, otomatisasi berbasis ambang batas (*threshold*) sensor yang mengaktifkan *exhaust fan*, *cooling pad*, dan pompa secara reaktif. Kedua, otomatisasi berbasis penjadwalan (*scheduling*) untuk rutinitas penyiraman dan pengadukan nutrisi. Ketiga, otomatisasi berbasis data rekomendasi robot yang memungkinkan irigasi presisi sesuai volume yang dibutuhkan. Seluruh fitur

tersebut telah divalidasi melalui serangkaian pengujian fungsional dan operasional.

Berdasarkan pengujian komunikasi data dan performa sistem, tercatat rata-rata waktu respons (*latency*) untuk kontrol manual perangkat sensor dan aktuator adalah sebesar 105 ms hingga 106 ms. Sementara itu, eksekusi otomatisasi tercatat memiliki rata-rata waktu respons berkisar antara 1.174 ms hingga 1.663 ms. Latensi pada proses otomatisasi ini merupakan konsekuensi teknis dari penerapan mekanisme antrian (*queue*) server dan validasi QoS Level 1 untuk memastikan integritas instruksi tanpa kegagalan pengiriman. Meskipun demikian, waktu respons tersebut masih berada jauh di bawah ambang batas toleransi perhatian pengguna (*user attention*) sebesar 10 detik, sehingga sistem tetap dinilai responsif dan andal.

Selain aspek teknis, evaluasi aspek *usability* yang melibatkan 43 responden menghasilkan skor *System Usability Scale* (SUS) sebesar 79,19. Nilai ini menempatkan sistem pada kategori *Acceptable* dengan predikat *Grade A-* dan *adjective rating "Good"*, yang mengindikasikan bahwa sistem mudah dipelajari dan dioperasikan. Penelitian ini menyimpulkan bahwa subsistem otomatisasi yang dibangun telah memenuhi spesifikasi kebutuhan fungsional serta mampu meningkatkan efisiensi pengelolaan *Greenhouse* Kebun Raya ITERA secara signifikan.

Kata Kunci: *Greenhouse, Internet of Things (IoT), Otomatisasi, MQTT, Agile Kanban, Website.*

ABSTRAK

RANCANG BANGUN SUBSISTEM OTOMATISASI PERANGKAT IOT BERBASIS *WEBSITE* (STUDI KASUS : *GREENHOUSE* KEBUN RAYA ITERA)

Muhammad Yusuf

Rancang bangun subsistem otomatisasi perangkat IoT berbasis *website* merupakan langkah strategis untuk mengatasi kendala pemantauan manual di *Greenhouse* Kebun Raya ITERA. Penelitian ini bertujuan membangun subsistem otomatisasi terpusat menggunakan metode *Agile Kanban* dan arsitektur komunikasi MQTT yang menerapkan standar *Quality of Service* (QoS) Level 0 untuk pemantauan sensor cepat dan Level 1 untuk keandalan kendali aktuator. Sistem berhasil mengimplementasikan tiga mekanisme kendali otomatis utama—berbasis penjadwalan, ambang batas sensor, dan data rekomendasi robot—yang divalidasi melalui 100% pengujian fungsional dan performa. Hasil pengukuran menunjukkan rata-rata waktu respons (*latency*) untuk kontrol manual sebesar 105-106 ms, sedangkan eksekusi otomatisasi tercatat pada kisaran 1.174 ms hingga 1.663 ms; latensi pada proses otomatisasi ini merupakan konsekuensi teknis dari penerapan mekanisme antrian (*queue*) server demi menjamin integritas instruksi tanpa risiko kehilangan data (*packet loss*). Evaluasi aspek *usability* memperkuat kelayakan sistem dengan skor *System Usability Scale* (SUS) sebesar 79,19, yang menempatkannya pada predikat *Acceptable, Grade A-*, serta masuk dalam kategori *Net Promoter Score* (NPS) *Promoter*, yang mengindikasikan tingkat kepuasan dan loyalitas pengguna yang tinggi terhadap sistem.

Kata Kunci: *Greenhouse*, Otomatisasi IoT, MQTT, *Agile Kanban*, *System Usability Scale* (SUS).

ABSTRACT

WEBSITE-BASED AUTOMATION SUBSYSTEM FOR IOT DEVICES (CASE STUDY: ITERA BOTANICAL GARDEN GREENHOUSE)

Muhammad Yusuf

The design and development of a website-based IoT device automation subsystem serves as a strategic measure to address manual monitoring limitations at the ITERA Botanical Garden Greenhouse. This research aims to construct a centralized automation subsystem utilizing the Agile Kanban method and MQTT communication architecture, implementing Quality of Service (QoS) Level 0 for rapid sensor monitoring and Level 1 for actuator control reliability. The system successfully implemented three primary automatic control mechanisms—schedule-based, sensor threshold-based, and robot recommendation data-based—which were validated through functional and performance testing. Measurement results indicated an average response time (latency) of 105-106 ms for manual control, while automation execution ranged from 1,174 ms to 1,663 ms; this latency in the automation process is a technical consequence of implementing a server queue mechanism to ensure instruction integrity without the risk of packet loss. The usability evaluation reinforced the system's feasibility with a System Usability Scale (SUS) score of 79.19, achieving an Acceptable predicate, Grade A-, and falling into the Net Promoter Score (NPS) Promoter category, indicating high user satisfaction and loyalty toward the system.

Keywords: *Greenhouse, IoT Automation, MQTT, Agile Kanban, System Usability Scale (SUS).*

DAFTAR ISI

LEMBAR PENGESAHAN	ii
HALAMAN PERNYATAAN ORISINALITAS.....	iii
HALAMAN PERNYATAAN PERSETUJUAN PUBLIKASI TUGAS AKHIR	iv
KATA PENGANTAR	v
MOTTO PENULIS	vi
RINGKASAN	vii
ABSTRAK	ix
ABSTRACT	x
DAFTAR ISI.....	xi
DAFTAR GAMBAR.....	xiv
DAFTAR TABEL.....	xv
DAFTAR RUMUS	xvi
DAFTAR KODE.....	xvii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	3
1.3 Tujuan Penelitian	3
1.4 Batasan Masalah.....	4
1.5 Sistematika Penulisan	4
1.6 Manfaat Penelitian	4
1.6.1 BAB I	5
1.6.2 BAB II.....	5
1.6.3 BAB III.....	5
1.6.4 BAB IV	5
1.6.5 BAB V.....	5
BAB II TINJAUAN PUSTAKA	6
2.1 Tinjauan Pustaka	6
2.2 Dasar Teori.....	7
2.2.1 <i>Subsistem</i>	8
2.2.2 <i>Website</i>	8
2.2.3 <i>Greenhouse Kebun Raya ITERA</i>	9
2.2.4 IoT	9
2.2.5 Protokol MQTT	9

2.2.6	MQTTX.....	10
2.2.7	Perangkat IoT.....	11
2.2.8	<i>Otomatisasi Perangkat IoT</i>	12
2.2.9	<i>Agile Kanban</i>	12
2.2.10	<i>Blackbox Testing</i>	13
2.2.11	<i>System Usability Scale (SUS)</i>	14
2.2.12	<i>Diagram Use Case</i>	16
2.2.13	<i>Diagram Aktivitas</i>	17
2.2.14	<i>Entity Relationship Diagram</i>	18
2.2.15	<i>Purposive Sampling</i>	18
2.2.16	<i>Websocket</i>	19
2.2.17	<i>Laravel</i>	19
2.2.18	<i>Redis</i>	20
2.2.19	<i>XHR Polling</i>	20
2.2.20	<i>Event Driven</i>	20
2.2.21	<i>Cron Scheduler</i>	21
2.2.22	<i>Queue Worker</i>	21
2.2.23	<i>ISO/IEC 25010</i>	22
BAB III METODE PENELITIAN.....		24
3.1	Alur Penelitian.....	24
3.2	Lingkup Penelitian	25
3.3	Alat dan Bahan	26
3.2.1	Alat Penelitian.....	27
3.2.2	Bahan Penelitian	27
3.4	Tahapan Seluruh Alur Penelitian.....	28
3.3.1	Inisialisasi.....	28
3.3.2	Proses Metode Agile Kanban	32
3.3.3	Pengujian <i>Usability</i>	48
3.3.4	Analisis dan Kesimpulan	49
BAB IV HASIL DAN PEMBAHASAN		51
4.1	Implementasi Hasil.....	51
4.1.1	Implementasi <i>Arsitektur</i>	51
4.1.2	Implementasi <i>Backend</i>	62
4.1.3	Implementasi <i>Frontend</i>	65
4.2	<i>Deploy</i>	73

4.3 Pengujian	75
4.2.1 Pengujian Fungsional <i>Blackbox Testing</i>	75
4.2.2 Pengujian Komunikasi Data dan Otomatisasi	76
4.2.3 Pengujian <i>Non-Fungsional</i>	77
4.4 <i>Review</i>	79
4.5 Pengujian <i>Usability</i>	84
4.6 Analisis	86
BAB V KESIMPULAN	87
5.1 Kesimpulan	87
5.2 Saran	87
DAFTAR PUSTAKA	89
LAMPIRAN	93
A. Dokumen-Dokumen <i>Capstone</i>	93
B. Hasil Wawancara	98
C. Daftar Bukti Dokumentasi Penelitian <i>Capstone</i>	100
D. Rincian Proses Agile Kanban	103
E. <i>Stakeholder</i> yang memiliki akses	106
F. Rincian Pengujian <i>Blackbox Testing</i>	108
G. Rincian Pengujian Komunikasi Data dan Otomatisasi	117

DAFTAR GAMBAR

Gambar 2.1 Visualisasi Subsistem	8
Gambar 2.2 Tampak Dalam dan Depan Greenhouse Kebun Raya ITERA	9
Gambar 2.3 Tampilan Aplikasi MQTTX.....	11
Gambar 2.4 Papan dan Metode Agile Kanban.....	13
Gambar 2.5 Visualisasi Blackbox Testing	13
Gambar 2.6 Instrumen Penilaian SUS.....	15
Gambar 2.7 Detail Angka Penilaian SUS	15
Gambar 2.8 Entity Relationship Diagram (ERD)	18
Gambar 2.9 Ilustrasi Arsitektur Event-Driven.....	21
Gambar 2.10 Ilustrasi Queue Worker.....	22
Gambar 3.1 Alur Penelitian	24
Gambar 3.2 Fokus Penelitian Otomatisasi Perangkat IoT.....	26
Gambar 3.3 (a) dan (b) Masalah XHR Polling Code Sensor.....	28
Gambar 3.4 Alur Logika Mekanisme XHR Polling	29
Gambar 3.5 Arsitektur Subsistem Otomatisasi Perangkat IoT.....	35
Gambar 3.6 Diagram Sekuens Otomatisasi Penjadwalan.....	36
Gambar 3.7 Diagram Sekuens Otomatisasi Ambang Batas Sensor	36
Gambar 3.8 Diagram Sekuens Otomatisasi Data Definisi Robot.....	37
Gambar 3.9 Use Case Diagram Otomatisasi Perangkat IOT.....	38
Gambar 3.10 Diagram Aktivitas Subsistem Otomatisasi Perangkat IoT	39
Gambar 3.11 Desain ERD Subsistem Otomatisasi Perangkat IOT	39
Gambar 4.1 Hasil Deployment	75
Gambar 4.2 Kumpulan Hasil Pengujian Non-Fungsional k6	78

DAFTAR TABEL

Tabel 2.1 Kajian Literatur Penelitian Terdahulu	6
Tabel 2.2 Daftar Pertanyaan SUS.....	14
Tabel 2.3 Use Case Diagram Symbol	16
Tabel 2.4 Activity Diagram Symbol	17
Tabel 2.5 Karakteristik dan Deskripsi Kualitas ISO/IEC 25010.....	22
Tabel 3.1 Penjelasan Alur Penelitian	24
Tabel 3.2 Alat Penelitian.....	27
Tabel 3.3 Kebutuhan Bahan Penelitian	27
Tabel 3.4 Hasil Wawancara.....	29
Tabel 3.5 Daftar Perangkat dan Data Definisi Robot	30
Tabel 3.6 Data Kebutuhan Otomatisasi.....	32
Tabel 3.7 Kebutuhan Fungsional.....	33
Tabel 3.8 Daftar Kebutuhan Non-Fungsional.....	34
Tabel 3.9 Pola Payload Data Perangkat IoT dan Data Definisi Robot	35
Tabel 3.10 Daftar Desain Wireframe	40
Tabel 3.11 Rencana Pengujian Blackbox Testing	44
Tabel 3.12 Rencana Pengujian Non-Fungsional.....	44
Tabel 3.13 Skenario Pengujian Komunikasi Data	45
Tabel 3.14 Pengujian Skenario Otomatisasi Perangkat IoT	46
Tabel 3.15 Gambaran Responden.....	48
Tabel 3.16 Skala Likert 1-5 [53]	49
Tabel 3.17 Simulasi Perhitungan 1 Responden SUS	49
Tabel 4.1 Komponen Arsitektur Subsistem Otomatisasi Perangkat IoT	51
Tabel 4.2 Implementasi Backend Subsistem Otomatisasi Perangkat IoT.....	62
Tabel 4.3 Hasil Implementasi Frontend	66
Tabel 4.4 Spesifikasi Lingkungan Produksi dan Status Akses.....	74
Tabel 4.5 Rekapitulasi Hasil Pengujian Blackbox Testing	75
Tabel 4.6 Ringkasan Hasil Pengujian Komunikasi Data dan Otomatisasi	76
Tabel 4.7 Hasil Pengujian Non-Fungsional	78
Tabel 4.8 Tahapan Review Hasil Implementasi	79
Tabel 4.9 Implementasi Perbaikan Backend.....	80
Tabel 4.10 Implementasi Perbaikan Frontend	80
Tabel 4.11 Hasil Pengujian Black Box Implementasi Perbaikan.....	83
Tabel 4.12 Tahapan Review Hasil Setelah Perbaikan	84
Tabel 4.13 Hasil Pengujian SUS	84

DAFTAR RUMUS

Rumus 2.1 Persentase Keberhasilan Blackbox Testing	14
Rumus 2.2 Skor Pertanyaan Ganjil	14
Rumus 2.3 Skor Pertanyaan Genap	15
Rumus 2.4 Perhitungan akhir SUS	15

DAFTAR KODE

Kode 4.1 Pendaftaran handler topic MQTT	52
Kode 4.2 Alur pemrosesan data sensor	52
Kode 4.3 Evaluasi kondisi otomatisasi berbasis sensor.....	53
Kode 4.4 Alur pemrosesan data robot	53
Kode 4.5 Evaluasi kondisi otomatisasi berbasis data robot.....	54
Kode 4.6 Konfigurasi task scheduling	55
Kode 4.7 Pemeriksaan dan eksekusi jadwal otomatisasi.....	55
Kode 4.8 Job eksekusi iterasi penjadwalan.....	55
Kode 4.9 Job eksekusi siklus aktuator.....	56
Kode 4.10 Job untuk menyalakan aktuator	57
Kode 4.11 Manajemen lock aktuator	57
Kode 4.12 Event broadcast status sensor	58
Kode 4.13 Penerimaan pembaruan status aktuator di frontend.....	59
Kode 4.14 Konfigurasi otorisasi channel WebSocket	60
Kode 4.15 Service pengiriman notifikasi otomatisasi	60
Kode 4.16 Kelas WebPushNotification.....	61
Kode 4.17 Pencatatan log eksekusi otomatisasi.....	61
Kode 4.18 Service pencatatan log aktivitas	61
Kode 4.19 Konfigurasi Deploy Menggunakan Docker	73

BAB I

PENDAHULUAN

1.1 Latar Belakang

Peraturan Presiden No. 83 Tahun 2023 mengarahkan Kebun Raya di Indonesia untuk fokus pada tiga tugas utama: konservasi, penelitian, dan pendidikan [1]. Kebijakan ini sejalan dengan Undang Undang No. 11 Tahun 2019 yang mendorong pemanfaatan teknologi dalam kegiatan riset penelitian dan pengembangan inovasi [2]. Institut Teknologi Sumatera (ITERA) melalui Rencana Strategis 2025-2029 menargetkan peningkatan Kebun Raya ITERA untuk meraih akreditasi Kelas A dengan pemanfaatan teknologi digital sebagai pendukung proses bisnis dalam aspek operasional [3]. Kebun Raya ITERA sebagai pusat konservasi tumbuhan dan edukasi telah mengadopsi teknologi *Internet of Things* (IoT) pada *greenhouse* untuk mendukung operasionalnya [4]. Dengan perkembangan tersebut, kebutuhan akan sistem teknologi digital menjadi semakin penting agar pengelolaan *greenhouse* dapat berjalan lebih responsif dan terintegrasi terhadap kebutuhan konservasi maupun penelitian.

Riset Setyawan *et al.* menunjukkan tren peningkatan signifikan publikasi ilmiah tentang *smart greenhouse*, dari 4.8% pada 2017 menjadi 28.6% pada Agustus 2022, yang mengindikasikan pertumbuhan minat akademis terhadap teknologi di bidang pertanian [5]. Studi *systematic literature review* oleh Abou-mehdi-Hassani *et al.* memperkuat temuan ini dengan mengidentifikasi empat kluster riset *smart greenhouse* yang mencakup *Internet of Things* (IoT), *smart greenhouses*, *precision agriculture*, dan kombinasi *sustainability* dengan *Industry 4.0*, di mana IoT menjadi teknologi tulang punggung yang meningkatkan operasi *greenhouse* dan praktik pertanian [6]. Teknologi *smart greenhouse* menggunakan sensor dan aktuator untuk memantau dan mengontrol kondisi lingkungannya [5]. Platform *web* dipilih sebagai media monitoring karena sifatnya yang *cross-platform* dan tidak memerlukan instalasi tambahan, sehingga memberikan akses yang lebih luas dibandingkan aplikasi *native* [7].

Penelitian Nugraha (2023) telah berhasil meletakkan dasar digitalisasi yang solid pada *Greenhouse* Kebun Raya ITERA melalui pengembangan *website monitoring* dengan tingkat penerimaan pengguna yang baik, ditunjukkan oleh skor *System Usability Scale* (SUS) sebesar 75,83 [8]. Sebagai langkah pengembangan keberlanjutan, dilakukan penelusuran lebih lanjut terhadap struktur kode dan arsitektur sistem yang ada. Hasil investigasi menunjukkan bahwa mekanisme komunikasi data saat ini menerapkan metode *XHR Polling*. Mengacu pada studi komparasi performa oleh Appelqvist dan

Örnmyr (2017), pendekatan ini dinilai kurang efisien karena setiap permintaan HTTP membawa beban *header* sekitar 200 hingga 800 *bytes*, sangat kontras dengan protokol *WebSockets* yang hanya membutuhkan *overhead* 2 hingga 10 *bytes* per pesan setelah koneksi terbentuk [9]. Inefisiensi ini terkonfirmasi melalui pengujian empiris pada interval pesan 2000 ms, di mana *XHR Polling* mengonsumsi lalu lintas data pengiriman sebesar 49,5 KB/s, jauh lebih boros dibandingkan *WebSockets* yang hanya menggunakan 4,3 KB/s [9]. Selain itu, Kleppmann (2017) menyatakan bahwa pendekatan *polling* pada basis data dapat menciptakan beban komputasi yang tidak perlu (*overhead*) karena sistem terus memproses permintaan kosong saat tidak ada jadwal aktif [10]. Berdasarkan tinjauan tersebut, perancangan subsistem ini akan menerapkan arsitektur hibrida yang memadukan protokol *WebSockets* untuk efisiensi komunikasi data *real-time* serta mekanisme pemrosesan latar belakang asinkron (*asynchronous background processing*) guna menjamin eksekusi otomatisasi yang andal tanpa membebani kinerja utama sistem.

Pembaruan ini diperlukan sejalan dengan hasil wawancara bersama Ibu Zunanik Mufidah, S.TP., M.Si. selaku Penanggung Jawab *Greenhouse* Kebun Raya ITERA, yang mengharapkan adanya sistem kendali untuk menangani penjadwalan, pengaturan batas sensor, serta respons otomatis terhadap data robot. Pengembangan ini difokuskan untuk mengurangi keterlibatan manual pada tugas-tugas berulang dalam operasional *greenhouse*. Hal ini selaras dengan Suhandi *et al.*, yang menyatakan bahwa fungsi utama otomasi adalah mendukung efisiensi kegiatan operasional dengan mengurangi beban kerja manusia [11]. Secara teknis, pendekatan ini diperkuat oleh pandangan Abolhassani Khajeh *et al.*, yang menjelaskan bahwa penerapan IoT sebaiknya tidak hanya memantau, tetapi juga mampu mengaktifkan perangkat secara otomatis untuk merespons kondisi lingkungan [12]. Oleh karena itu, pengelolaan data IoT dan robot pada subsistem ini dirancang agar logika otomatisasi dapat dijalankan secara terstruktur dan konsisten sesuai parameter yang ditentukan.

Pengembangan subsistem ini menggunakan metode Agile Kanban setelah membandingkan berbagai pendekatan *Software Development Life Cycle* (SDLC). Model lama seperti *Waterfall* dianggap kurang cocok karena terlalu kaku terhadap perubahan kebutuhan [13]. Sementara itu, model *Spiral* yang fokus pada risiko dinilai terlalu rumit dan butuh banyak sumber daya untuk tim kecil [13]. Dalam kelompok metode *Agile*, *Scrum* memang terstruktur namun mewajibkan peran khusus dan waktu kerja yang kaku [14], [15], yang dimana kontras dengan *Kanban* yang lebih luwes dan mengalir terus-menerus tanpa perlu perencanaan yang berlebihan [14], [15], sehingga lebih pas untuk tim kecil *capstone* penelitian ini yang hanya berisi dua orang

yang masing masing memiliki fokus pengembangan yang berbeda dan membutuhkan hasil yang terukur jelas [16]. Karena alasan itu, *Kanban* dipilih sebab mampu menggambarkan alur kerja secara visual dan membatasi pekerjaan yang sedang berjalan atau *Work-In-Progress* (WIP) [15]. Cara ini mencegah tugas menumpuk dan memastikan hasil kerja siap pakai atau *production-ready* untuk sebuah produk [15].

Pengujian pada penelitian subsistem otomatisasi perangkat IoT yang mencakup pengendalian berdasarkan jadwal, ambang batas sensor, serta rekomendasi data robot dilakukan menggunakan aplikasi MQTT (*Message Queuing Telemetry Transport*) Explorer untuk pengujian komunikasi dan metode *Black-Box Testing* untuk pengujian fungsional, karena pendekatan ini dinilai efisien dari sisi waktu dan biaya serta memadai untuk memastikan perangkat *Internet of Things* (IoT) dapat berkomunikasi dan berfungsi sesuai kebutuhan sistem tanpa perlu menelaah kode program secara mendalam [8]. Aspek kepuasan pengguna terhadap subsistem diukur menggunakan kuesioner *System Usability Scale* (SUS), yang digunakan untuk memperoleh gambaran tingkat penerimaan dan kemudahan penggunaan subsistem secara menyeluruh dari sudut pandang pengguna [17]. Selanjutnya, sesuai dengan prinsip standar internasional mengenai *pengguna tertentu* serta kebutuhan penyelesaian masalah nyata dari mitra, penelitian ini menerapkan teknik *purposive sampling* dalam pemilihan responden, yaitu dengan melibatkan pihak-pihak yang memahami proses bisnis dan penggunaan sistem agar hasil pengujian yang diperoleh lebih relevan dan sesuai dengan konteks penerapan subsistem [18], [19], [20].

1.2 Rumusan Masalah

Latar belakang yang ada menunjukkan bahwa rumusan masalah dalam penelitian ini dapat dijabarkan sebagai berikut:

1. Bagaimana cara rancang bangun subsistem otomatisasi perangkat IoT berbasis *website* menggunakan metode Agile Kanban?
2. Bagaimana hasil pengujian subsistem otomatisasi perangkat IoT berdasarkan *Black-Box Testing*, MQTTX, serta *usability* berdasarkan SUS?

1.3 Tujuan Penelitian

Latar belakang dan rumusan masalah memberikan dasar bahwa tujuan dari penelitian ini adalah sebagai berikut:

1. Menghasilkan rancangan dan implementasi subsistem otomatisasi perangkat IoT berbasis *website* dengan menerapkan Agile Kanban.

2. Mengukur tingkat keberhasilan fungsi, uji komunikasi, dan *usability* subsistem otomatisasi menggunakan metode *Black-Box Testing*, MQTTX, dan *usability* menggunakan SUS.

1.4 Batasan Masalah

Penelitian ini dibatasi dengan batasan-batasan pengembangan sebagai berikut:

1. Penelitian difokuskan pada pengembangan subsistem otomatisasi perangkat *Internet of Things* (IoT) berbasis website yang mencakup penjadwalan, ambang batas sensor, dan rekomendasi data robot.
2. Penelitian tidak membahas perancangan dan detail teknis perangkat keras IoT serta mekanisme algoritma rekomendasi berbasis *Artificial Intelligence* (AI) pada robot.
3. Penelitian tidak mencakup analisis maupun perbandingan hasil tanaman yang dibudidayakan di dalam *greenhouse*.
4. Penerapan dan pengujian subsistem dibatasi pada *greenhouse* Kebun Raya ITERA dengan responden yang memiliki akses terhadap sistem.
5. Metode pengembangan dibatasi pada pendekatan *Agile Kanban*, sedangkan pengujian dilakukan menggunakan *Black-Box Testing*, MQTTX, dan *System Usability Scale* (SUS).
6. Pengujian komunikasi dibatasi pada verifikasi jalur komunikasi MQTT menggunakan MQTTX tanpa pengujian performa jaringan lanjutan.
7. Penelitian ini tidak melakukan perbandingan performa terhadap sistem yang dikembangkan pada penelitian sebelumnya.

1.5 Sistematika Penulisan

Sistematika penulisan penelitian ini disusun untuk memberikan gambaran umum mengenai struktur laporan tugas akhir sebagai berikut:

1.6 Manfaat Penelitian

Penelitian ini diharapkan dapat memberikan manfaat bilamana rumusan masalah terjawab dan tujuan tercapai, antara lain:

1. Tersedianya subsistem otomatisasi yang dapat membantu proses pengelolaan *greenhouse* menjadi lebih efisien melalui penerapan otomatisasi perangkat IoT berbasis *website*.
2. Memberikan data pengujian serta evaluasi mengenai kinerja fungsi otomatisasi dan tingkat kepuasan pengguna sebagai tolak ukur keberhasilan subsistem.

1.6.1 BAB I

Bab ini menjelaskan latar belakang, rumusan masalah, tujuan, batasan, manfaat penelitian, serta sistematika penulisan penelitian.

1.6.2 BAB II

Bab ini memaparkan tinjauan pustaka yang meliputi kajian teori dan penelitian terdahulu yang relevan dengan penelitian ini.

1.6.3 BAB III

Bab ini menjelaskan alur sistematis dan metodologi penelitian yang digunakan untuk menyelesaikan penelitian ini.

1.6.4 BAB IV

Bab ini memaparkan hasil pengembangan dan hasil pengujian sesuai dengan metodologi penelitian. Selanjutnya hasil akan dievaluasi dan dianalisis.

1.6.5 BAB V

Bab ini menjelaskan kesimpulan dari penelitian yang dilakukan, serta saran untuk pengembangan lebih lanjut.

BAB II

TINJAUAN PUSTAKA

2.1 Tinjauan Pustaka

Tabel 2.1 menyajikan rangkuman penelitian-penelitian sebelumnya yang relevan. Rangkuman ini mencakup metodologi, temuan, dan kontribusi dari setiap penelitian rujukan. Kajian ini digunakan sebagai landasan logis untuk penelitian ini. Penelitian ini memposisikan sebagai rancang bangun subsistem otomatisasi menggunakan metode agile kanban, pengujian menggunakan *black-box testing*, MQTTX untuk uji komunikasi, dan evaluasi *usability* dengan SUS untuk pengujian secara menyeluruh terkait subsistem otomatisasi pada *website greenhouse* Kebun Raya ITERA.

Tabel 2.1 Kajian Literatur Penelitian Terdahulu

No.	Penulis [tahun][judul]	Permasalahan dan Tujuan	Metode	Hasil
1.	Nugraha [2023] Sistem <i>Monitoring Smart Greenhouse</i> berbasis IoT Dengan Menggunakan Metode Agile Kanban (Studi Kasus Greenhouse Melon ITERA) [8]	<i>Monitoring hardware IoT pada greenhouse spesifik tanaman melon melalui website;</i> bertujuan membangun <i>website</i> pemantauan berbasis IoT.	Agile Kanban, <i>Blackbox Testing</i> , dan SUS	Sistem <i>monitoring</i> berhasil dibangun dan diuji dengan hasil skor SUS 75.875 (kategori "Good").
2.	Morchid <i>et al.</i> [2024] <i>Agri-tech innovations for sustainability: A fire detection system based on MQTT broker and IoT to improve environmental risk management</i> [21]	Sistem deteksi kebakaran pertanian belum memiliki visualisasi data IoT real-time yang terpusat. Penelitian ini bertujuan memanfaatkan MQTTX untuk memantau data sensor dan status sistem secara langsung.	MQTTX	MQTTX efektif untuk monitoring IoT <i>real-time</i> dengan akurasi 98,77% dan respons ± 2 detik.

No.	Penulis [tahun][judul]	Permasalahan dan Tujuan	Metode	Hasil
3.	Bagus Hardika <i>et al.</i> [2024] Merdekawan, A. P., & Sari, P. [2022] [Studi Awal Rancang Bangun <i>Indoor Farming Monitoring System</i> Berbasis IoT dengan Protokol Websocket]	Penelitian ini bertujuan mengembangkan sistem monitoring <i>indoor farming</i> berbasis IoT untuk mengatasi kendala lahan dan mobilitas tinggi masyarakat kota	Protokol <i>WebSocket</i> yang diuji fungsionalitasnya (<i>blackbox</i>) dan latensi jaringannya (<i>delay</i>)	Prototipe sukses menampilkan data real-time dengan delay pengiriman kategori bagus sebesar 0,028 detik.
4.	Sekar Sari dan Oktavia Citra Resmi Rachmawati [2025] <i>The Implementation of Agile Kanban in the Development of an IoT-Based Sugarcane Growth Monitoring System</i> [23]	Permasalahan utama adalah belum adanya sistem pemantauan lingkungan real-time untuk mendukung keputusan budidaya tebu; Tujuannya adalah mengembangkan sistem IoT pemantauan pertumbuhan tebu menggunakan Agile Kanban.	Pendekatan rekayasa eksperimental menggunakan Agile Kanban, melibatkan studi literatur, perencanaan, implementasi, dan analisis iteratif dengan <i>Kanban Board</i> .	Sistem pemantauan tebu IoT berhasil diselesaikan tepat waktu. Efisiensi kerja Kanban terbukti andal, di mana rata-rata satu tugas selesai dalam 2,5 hari, dan total 10 tugas tuntas setiap dua minggu.
5.	H. Y. Riskiawan <i>et al.</i> [2023] <i>Artificial Intelligence Enabled Smart Monitoring and Controlling of IoT-Green House</i> [24]	Masalah utama adalah kontrol <i>manual</i> rumah kaca; tujuannya mengembangkan sistem IoT-AI untuk otomasi.	Arsitektur <i>Laravel</i> sebagai pusat perintah terintegrasi perangkat <i>IoT</i> via protokol <i>MQTT</i> .	Memfasilitasi arsitektur <i>Smart Agent Based on Cloud</i> untuk mekanisme <i>Publish</i> data dan <i>Subscribe</i> otomasi.

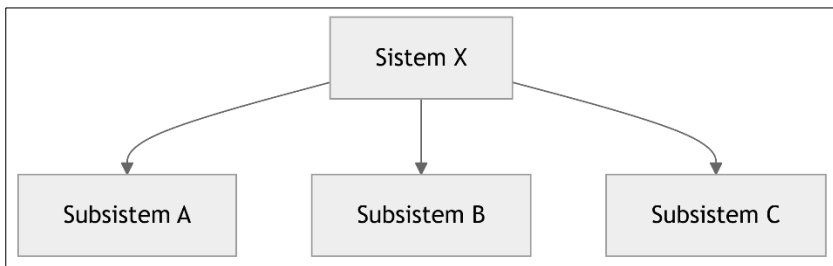
2.2 Dasar Teori

Sub-bab ini menguraikan landasan teori dan konsep-konsep fundamental yang menjadi acuan utama dalam pelaksanaan penelitian.

Pemaparan teori ini bertujuan untuk memberikan kerangka pemikiran yang sistematis serta memperkuat validitas metodologi yang diterapkan. Pembahasan mencakup tinjauan mendalam terhadap teknologi pengembangan sistem, arsitektur perangkat lunak, serta standar pengujian yang relevan guna mendukung analisis dan perancangan solusi yang diusulkan.

2.2.1 *Subsistem*

Subsistem didefinisikan sebagai bagian atau komponen yang terpadu dan saling keterkaitan di dalam sebuah sistem yang lebih besar (Lihat Gambar 2.1). Setiap subsistem harus mendapatkan perhatian penuh karena memiliki fungsi spesifik dan kinerjanya berkontribusi langsung pada pencapaian tujuan organisasi secara keseluruhan; misalnya, Sistem Akuntansi merupakan subsistem dari sistem organisasi. Konsep ini penting karena jika satu subsistem tidak berfungsi dengan baik, hal itu akan berdampak pada proses hingga hasil yang dikeluarkan oleh sistem induk, menekankan bahwa keberhasilan sistem bergantung pada kerjasama tim dan integrasi subsistemnya [25].



Gambar 2.1 Visualisasi Subsistem

2.2.2 *Website*

Website merupakan sekumpulan konten digital yang tersimpan dalam domain tertentu di internet, dirancang dengan tujuan spesifik dan saling berkaitan satu sama lain. Konten tersebut mencakup berbagai elemen multimedia seperti teks, gambar, video, dan berbagai bentuk media digital lainnya yang dapat diakses melalui peramban *web* menggunakan nama domain atau alamat IP yang unik [26]. Sebagai salah satu bentuk media informasi dan publikasi, situs *web* menawarkan kemudahan akses yang fleksibel tanpa batasan waktu dan lokasi geografis, sehingga dapat dimanfaatkan oleh berbagai pihak termasuk institusi pendidikan tinggi [27]. Dalam perspektif yang lebih luas, situs *web* dapat dipahami sebagai kumpulan halaman yang berada dalam jaringan *World Wide Web* (WWW) yang dapat diakses melalui koneksi internet [26].

2.2.3 *Greenhouse Kebun Raya ITERA*

Greenhouse atau rumah tanaman adalah bangunan tertutup yang berfungsi sebagai tempat budidaya tanaman dengan pengaturan kondisi lingkungan yang mendukung pertumbuhan optimal. Saat ini, *greenhouse* banyak dimanfaatkan untuk kegiatan penelitian karena memungkinkan penyesuaian suhu, kelembapan, dan pencahayaan sesuai kebutuhan eksperimen. Di Balai Penelitian dan Pengembangan Tanaman Pangan Kabupaten Pemalang, *greenhouse* digunakan sebagai laboratorium lapangan atau kebun percobaan yang menjadi fasilitas penelitian bagi berbagai jenis tanaman hortikultura, terutama sayur dan buah [28]. Gambar 2.2 menunjukkan *greenhouse* yang ada di Kebun Raya ITERA.



Gambar 2.2 Tampak Dalam dan Depan *Greenhouse* Kebun Raya ITERA

2.2.4 **IoT**

Internet of Things (IoT) adalah teknologi yang memungkinkan koneksi dan komunikasi antarperangkat melalui jaringan internet untuk mengotomatisasi aktivitas seperti penelusuran, pemrosesan, serta pengiriman data dalam sistem terintegrasi [29]. Arsitektur *IoT* menghubungkan perangkat keras dan lunak dalam satu ekosistem yang terdiri atas tiga komponen utama: sensor, *gateway*, dan *cloud* [30]. Sensor berfungsi mengumpulkan data lingkungan seperti suhu, kelembapan, nutrisi dari objek yang terhubung, sedangkan *gateway* bertugas meneruskan data tersebut ke basis data *cloud* melalui protokol komunikasi internet serta mengatur konektivitas perangkat secara otomatis. Data yang tersimpan di server *cloud* kemudian dapat diakses untuk pengendalian sistem *IoT* secara terpusat, sehingga mendukung interoperabilitas dan *remote monitoring* terhadap berbagai perangkat dalam jaringan [31].

2.2.5 **Protokol MQTT**

MQTT, singkatan dari *Message Queuing Telemetry Transport*, merupakan sebuah protokol komunikasi *machine-to-machine (M2M)* yang dirancang secara khusus agar sangat ringan untuk perangkat dalam ekosistem *Internet of Things (IoT)* [32]. Protokol ini sangat ideal untuk

diimplementasikan pada sistem *embedded*, seperti Raspberry Pi yang terpasang pada robot bergerak, karena dibangun untuk perangkat dengan keterbatasan daya dan *bandwidth*. MQTT mengadopsi model arsitektur *publish-subscribe* (Pub/Sub), yang memisahkan tanggung jawab antara pihak yang mengirim informasi (*publisher*) dengan pihak yang menerima pesan (*subscriber*) melalui perantara terpusat yang disebut broker [33].

Elemen-elemen inti yang wajib hadir untuk menjalankan protokol MQTT dalam suatu implementasi sistem [33]:

1. *Broker*

Broker bertindak sebagai pusat kendali dan perantara pesan, yang merupakan *server* utama yang bertanggung jawab menerima semua data yang dipublikasikan (*published*) dan kemudian mendistribusikan data tersebut kepada semua klien (*client*) yang telah mendaftar (*subscribe*) pada label (*topic*) yang relevan.

2. *Publisher*

Publisher merupakan klien atau perangkat yang memiliki peran mengirimkan data atau informasi spesifik ke label (*topic*) tertentu yang telah didefinisikan pada *broker*.

3. *Subscriber*

Merupakan klien atau perangkat yang berfungsi menerima atau menarik informasi dengan cara mendaftarkan diri (*subscribing*) ke label (*topic*) tertentu yang berada di *broker*.

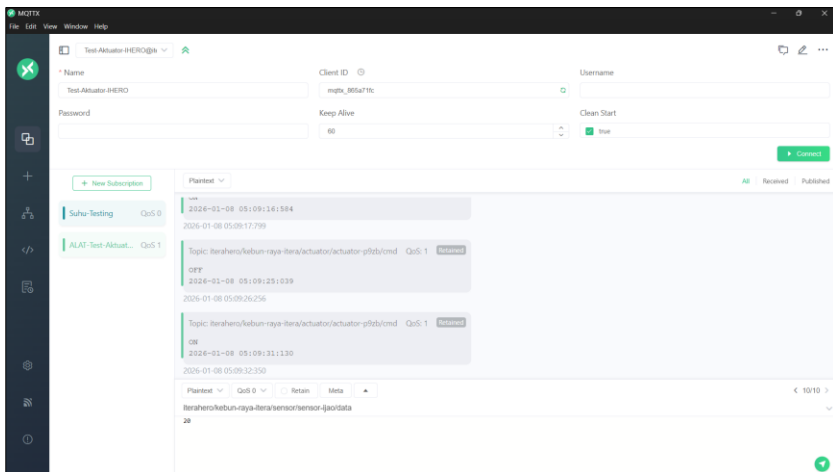
4. *Topic*

Dapat diibaratkan sebagai alamat digital atau label berformat *string* yang terorganisasi di dalam *broker*. *Topic* berfungsi sebagai wadah penyimpanan sementara untuk data yang dikirimkan oleh *publisher*.

2.2.6 MQTXX

MQTXX didefinisikan sebagai perangkat lunak klien MQTT (*Message Queuing Telemetry Transport*) yang berfungsi untuk memvisualisasikan, memantau, dan mengelola struktur topik serta muatan pesan (*payload*) dalam ekosistem *Internet of Things* (IoT) (Lihat Gambar 2.3) [34]. Aplikasi ini menyediakan antarmuka grafis yang menampilkan hierarki topik secara terstruktur sehingga memungkinkan pengembang untuk mengamati pertukaran data secara *real-time*, melakukan *debugging* sistem, serta memverifikasi integritas pengiriman pesan dari perangkat *publisher* ke *broker*. Selain itu, MQTXX mendukung pengamatan mekanisme *Quality of Service* (QoS) yang digunakan dalam komunikasi MQTT, mencakup QoS level 0 (*at most once*), QoS level 1 (*at least once*), dan QoS level 2 (*exactly once*), sehingga memungkinkan evaluasi perilaku pengiriman pesan terkait

keandalan transmisi, potensi duplikasi data, dan latensi komunikasi [35]. Dalam penelitian implementasi sistem kendali IoT, MQTTX dimanfaatkan sebagai instrumen validasi untuk memastikan bahwa parameter yang dikirimkan oleh mikrokontroler diterima pada sisi *broker* dan *subscriber* dengan format pesan serta tingkat QoS yang sesuai dengan perancangan sistem, sehingga membantu mengidentifikasi kesalahan konfigurasi protokol komunikasi selama tahap pengembangan [35].



Gambar 2.3 Tampilan Aplikasi MQTTX

2.2.7 Perangkat IoT

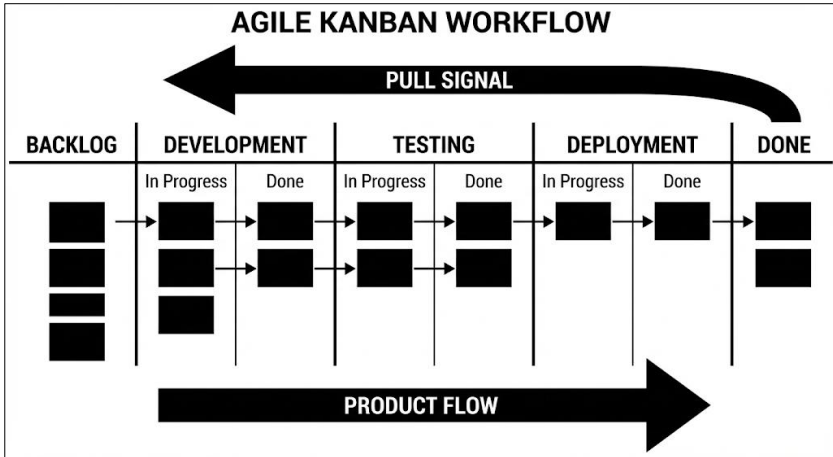
Perangkat *Internet of Things* (IoT) merupakan komponen perangkat keras fisik yang berfungsi sebagai jembatan interaksi antara dunia nyata dan sistem komputasi digital. Dalam arsitektur IoT, perangkat ini beroperasi pada lapisan persepsi (*perception layer*) yang terdiri dari dua elemen fundamental, yaitu sensor dan aktuator. Sensor berperan sebagai unit akuisisi data yang bertugas mendeteksi perubahan parameter fisik di lingkungan, seperti suhu, kelembaban, gerak, atau intensitas cahaya, untuk kemudian mengonversinya menjadi sinyal listrik atau data digital untuk diproses lebih lanjut. Sebaliknya, aktuator berfungsi sebagai unit eksekusi yang menerima sinyal kendali dari sistem untuk melakukan aksi mekanis atau fisik, seperti mengaktifkan motor, membuka katup, atau menyalakan alarm. Sinergi antara kemampuan penginderaan sensor dan kemampuan aksi aktuator inilah yang memungkinkan sistem IoT untuk melakukan pemantauan dan pengendalian lingkungan secara otonom dan *real-time* [36], [37].

2.2.8 Otomatisasi Perangkat IoT

Otomatisasi didefinisikan sebagai penerapan sistem dan proses teknologi untuk melaksanakan tugas dengan mengurangi intervensi manusia guna mencapai presisi, konsistensi, dan kecepatan operasional [11]. Dalam ekosistem *Internet of Things* (IoT), perangkat fisik saling terhubung untuk memungkinkan pertukaran data secara *real-time*, di mana sensor-sensor cerdas mampu melakukan tugas pemantauan dan eksekusi secara otonom [11]. Integrasi mendalam antara otomatisasi dan IoT ini berpotensi menciptakan sistem yang dapat mengatur dirinya sendiri (*self-regulating systems*) untuk mengoptimalkan operasi secara dinamis berdasarkan kondisi lingkungan terkini [11]. Mekanisme penjadwalan waktu nyata (*real-time scheduling*) menjadi komponen vital dalam arsitektur ini untuk memastikan perangkat merespons kebutuhan komputasi segera tanpa penundaan yang berarti [12]. Sinergi teknologi ini tidak hanya meningkatkan efisiensi, tetapi juga memberikan manfaat eksponensial yang melampaui kontribusi masing-masing teknologi jika diterapkan secara terpisah [11].

2.2.9 Agile Kanban

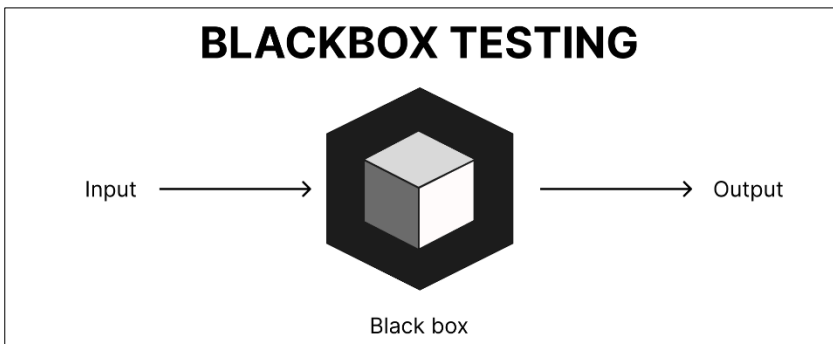
Agile Kanban mengadaptasi prinsip fleksibilitas Agile dengan fokus utama pada visualisasi alur kerja dan pengiriman berkelanjutan (*continuous flow*) tanpa batasan *sprint* yang kaku [15]. Seperti yang ditunjukkan pada Gambar 2.4, metode ini membagi proses pengembangan ke dalam tahapan spesifik, mulai dari *Backlog*, *Development*, *Testing*, hingga *Deployment*, yang di mana setiap kartu tugas bergerak secara linear mengikuti arah *Product Flow*. Mekanisme *Pull Signal* digunakan untuk menarik tugas dari kolom sebelumnya hanya ketika kapasitas tersedia, yang dikombinasikan dengan pembatasan *Work-In-Progress* (WIP) untuk mencegah *bottlenecks* dan menjamin transparansi penyelesaian tugas di setiap sub-kolom (*In progress* dan *Done*) [38]. Selain itu, penerapan standar *definition of done* memastikan bahwa setiap luaran fitur di kolom terakhir memiliki kualitas *production-ready* secara konsisten [23]. Dengan demikian, pendekatan ini mampu menawarkan efisiensi sumber daya sekaligus responsivitas yang tinggi terhadap perubahan prioritas [14].



Gambar 2.4 Papan dan Metode Agile Kanban [39]

2.2.10 Blackbox Testing

Blackbox Testing adalah teknik evaluasi perangkat lunak yang berfokus pada verifikasi respons sistem terhadap data masukan tanpa menganalisis arsitektur internal atau alur logika program (Lihat Gambar 2.5). Pendekatan ini dapat diimplementasikan untuk menguji aspek fungsional maupun *non-fungsional* sistem [8]. Penguji tidak diharuskan memiliki pemahaman mendalam mengenai struktur kode sumber yang digunakan dalam aplikasi. Melalui metode ini, dapat dilakukan deteksi anomali pada sistem serta validasi bahwa setiap fungsi operasional sesuai dengan kriteria dan spesifikasi yang telah ditetapkan sebelumnya [40]. Setelah dilakukan pengujian seluruh skenario, hasil akan di kalkulasi dengan Rumus 2.1.

Gambar 2.5 Visualisasi *Blackbox Testing*

$$\text{Persentase Keberhasilan} = \frac{\text{Jumlah Skenario yang Berhasil}}{\text{Jumlah Total Skenario}} \times 100\%$$

Rumus 2.1 Persentase Keberhasilan *Blackbox Testing* [41]

2.2.11 *System Usability Scale* (SUS)

System Usability Scale (SUS) adalah instrumen kuesioner ciptaan John Brooke yang dirancang untuk mengevaluasi kebergunaan sistem komputer berdasarkan persepsi subjektif pengguna. Instrumen ini menawarkan kemudahan interpretasi melalui sistem *skoring* numerik 0 hingga 100 dan bersifat praktis tanpa memerlukan komputasi statistik yang kompleks [42]. Selain dapat diakses secara gratis, SUS terbukti memiliki validitas dan reliabilitas yang kuat meskipun diterapkan pada sampel berukuran kecil [8]. Implementasinya dilakukan menggunakan 10 butir pernyataan standar untuk mengukur pengalaman pengguna terhadap perangkat lunak yang dikembangkan (Lihat Tabel 2.2). Berbagai keunggulan tersebut menjadikan metode ini sangat diandalkan dalam bidang evaluasi *usability* hingga saat ini.

Tabel 2.2 Daftar Pertanyaan SUS [42]

No.	Pertanyaan	Rentang Nilai				
		1	2	3	4	5
1.	Saya merasa akan sering menggunakan sistem ini.	☐	☐	☐	☐	☐
2.	Saya merasa sistem ini memiliki kerumitan yang tidak perlu.	☐	☐	☐	☐	☐
3.	Saya merasa sistem ini mudah untuk digunakan.	☐	☐	☐	☐	☐
4.	Saya merasa perlu bantuan teknisi atau ahli untuk bisa mengoperasikan sistem ini.	☐	☐	☐	☐	☐
5.	Saya melihat fungsi-fungsi dalam sistem ini saling terhubung dengan baik.	☐	☐	☐	☐	☐
6.	Saya merasa ada terlalu banyak inkonsistensi dalam sistem ini.	☐	☐	☐	☐	☐
7.	Saya membayangkan kebanyakan orang dapat mempelajari penggunaan sistem ini dengan cepat.	☐	☐	☐	☐	☐
8.	Saya merasa sistem ini terasa canggung atau tidak nyaman digunakan.	☐	☐	☐	☐	☐
9.	Saya merasa percaya diri ketika menggunakan sistem ini.	☐	☐	☐	☐	☐
10.	Saya merasa perlu mempelajari banyak hal terlebih dahulu sebelum benar-benar dapat menggunakan sistem ini.	☐	☐	☐	☐	☐

Catatan: 1 (Sangat tidak setuju) – 5 (Sangat setuju)

Pertanyaan yang nantinya akan di isi responden akan di kalkulasi dengan Rumus 2.2, 2.3, 2.4, serta berikut langkahnya [8]:

- 1) Pertanyaan nomor ganjil

$$\text{Skor Pertanyaan} = \text{Penilaian pengguna nomor ganjil} - 1$$

Rumus 2.2 Skor Pertanyaan Ganjil

- 2) Pertanyaan nomor genap

$$\text{Skor Pertanyaan} = 5 - \text{Penilaian pengguna nomor genap}$$

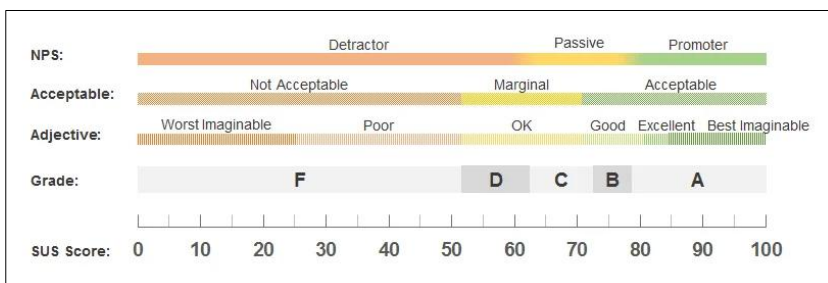
Rumus 2.3 Skor Pertanyaan Genap

3) Perhitungan Akhir

$$\text{Nilai SUS} = \frac{(\sum \text{Skor pertanyaan } 1 - 10) \times 2.5}{\text{Total Responden}}$$

Rumus 2.4 Perhitungan akhir SUS

Hasil perhitungan akhir akan di kalkulasi dan di nilai dengan standar instrumen penilaian subjektif pada Gambar 2.6 dan Gambar 2.7.



Gambar 2.6 Instrumen Penilaian SUS [42]

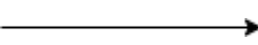
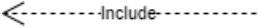
Grade	SUS	Percentile range	Adjective	Acceptable	NPS
A+	84.1-100	96-100	Best Imaginable	Acceptable	Promoter
A	80.8-84.0	90-95	Excellent	Acceptable	Promoter
A-	78.9-80.7	85-89		Acceptable	Promoter
B+	77.2-78.8	80-84		Acceptable	Passive
B	74.1 – 77.1	70 – 79		Acceptable	Passive
B-	72.6 – 74.0	65 – 69		Acceptable	Passive
C+	71.1 – 72.5	60 – 64	Good	Acceptable	Passive
C	65.0 – 71.0	41 – 59		Marginal	Passive
C-	62.7 – 64.9	35 – 40		Marginal	Passive
D	51.7 – 62.6	15 – 34	OK	Marginal	Detractor
F	25.1 – 51.6	2– 14	Poor	Not Acceptable	Detractor
F	0-25	0-1.9	Worst Imaginable	Not Acceptable	Detractor

Gambar 2.7 Detail Angka Penilaian SUS [43]

2.2.12 Diagram Use Case

Use case diagram merupakan diagram perilaku dalam *Unified Modeling Language* (UML) yang digunakan untuk memodelkan interaksi antara aktor dan sistem berdasarkan kebutuhan pengguna akhir [44]. Diagram ini menampilkan fungsi-fungsi utama yang dijalankan oleh aktor terhadap sistem, seperti pengguna atau administrator. Dengan menampilkan relasi antar unsur dan batas sistem, *use case diagram* memudahkan pemahaman mengenai peran serta interaksi yang terjadi dalam sistem secara menyeluruh. *Use case diagram* juga berfungsi sebagai alat komunikasi antara *developer* dan *stakeholder* untuk memastikan pemahaman yang sama terhadap fungsionalitas sistem [44]. Dalam konteks perancangan *interface*, diagram ini menjadi dasar untuk mengidentifikasi kebutuhan halaman dan alur navigasi yang harus dirancang. Penjelasan rinci mengenai elemen-elemen tersebut disajikan pada Tabel 2.3.

Tabel 2.3 *Use Case Diagram Symbol*

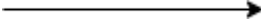
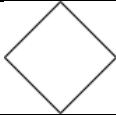
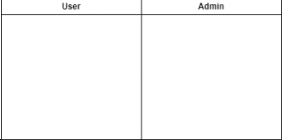
No	Simbol	Nama simbol	Fungsi
1		<i>Use Case</i>	Mewakili bagian dari fungsionalitas sistem dan ditempatkan dalam batasan sistem.
2		Aktor	Menggambarkan tokoh atau sistem yang berhubungan dengan sistem.
3		Asosiasi	Menghubungkan aktor dengan <i>use case</i> atau dengan <i>use case</i> lain nya.
4		<i>Generalisasi</i>	Menunjukkan generalisasi dari <i>use case</i> khusus ke umum.
5		<i>Include</i>	Hubungan antar <i>use case</i> tambahan dan <i>use case</i> lain nya.

No	Simbol	Nama simbol	Fungsi
6		<i>Extend</i>	Hubungan <i>use case</i> dengan <i>use case</i> tambahan yang berdiri sendiri.

2.2.13 Diagram Aktivitas

Activity diagram merupakan diagram perilaku dalam *Unified Modeling Language* (UML) yang digunakan untuk memodelkan alur aktivitas atau proses kerja dalam sistem. Diagram ini mengurutkan kegiatan pada proses bisnis atau sistem informasi, baik yang linier maupun bercabang, sehingga memudahkan perancangan prosedur dan identifikasi hambatan dalam alur kerja [44]. Melalui representasi visual yang jelas, *activity diagram* membantu memahami logika proses secara sistematis. Dalam perancangan *user interface*, diagram ini berperan penting untuk memvisualisasikan alur interaksi pengguna mulai dari titik awal hingga pencapaian tujuan akhir [44]. Penjabaran lebih lanjut mengenai elemen diagram ini tercantum pada Tabel 2.4.

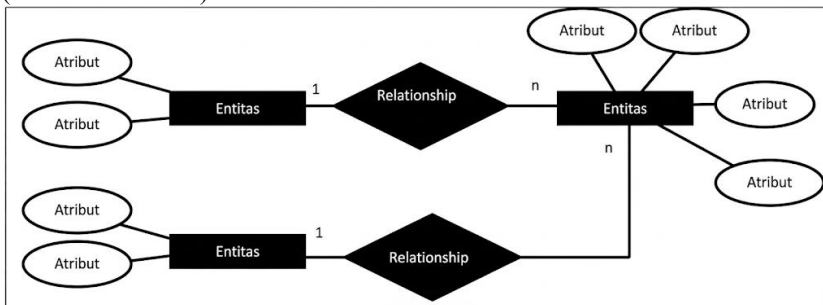
Tabel 2.4 *Activity Diagram Symbol*

No	Simbol	Nama simbol	Fungsi
1		Mulai	Menunjukkan simbol awal dari aktivitas sistem.
2		Aktivitas	Menggambarkan kegiatan yang dijalankan oleh sistem.
3		Transisi	Memperlihatkan alur proses yang berlangsung.
4		Kondisi	Menjelaskan kondisi bercabang apabila terdapat lebih dari satu kemungkinan keadaan.
		Penggabungan	Kasus lebih dari satu aktivitas yang digabung.
5		<i>Swimlane</i>	Mengklasifikasikan jenis tugas atau aktivitas yang dilakukan.
6		Akhir	Menandakan pula simbol akhir yang menunjukkan berakhirnya suatu aktivitas dalam sistem

2.2.14 Entity Relationship Diagram

Entity Relationship Diagram (ERD) merupakan diagram visual yang dibuat untuk mendeskripsikan data atau objek-objek berdasarkan kondisi dunia nyata yang disebut sebagai entitas. Diagram ini secara spesifik berfungsi menggambarkan hubungan (*relationship*) yang terjalin antar entitas tersebut dengan menggunakan serangkaian untuk memvisualisasikan struktur data secara jelas [45]. Manfaat utama dari penerapan diagram ini adalah untuk mempermudah proses perancangan desain basis data dengan memetakan objek-objek nyata ke dalam skema logis sistem. Dalam proses pengembangan perangkat lunak, ERD berperan sebagai cetak biru yang memastikan bahwa seluruh kebutuhan penyimpanan informasi dapat terakomodasi dengan baik sebelum tahap implementasi dilakukan [45].

ERD memvisualisasikan data melalui tiga komponen utama: Entitas (objek penting, dilambangkan kotak), Atribut (detail dari entitas, dilambangkan oval), dan Relasi (hubungan antar entitas, dilambangkan belah ketupat). Komponen-komponen ini dihubungkan oleh Garis Penghubung, yang juga dilengkapi dengan indikator Kardinalitas (seperti 1 atau n) untuk menjelaskan seberapa banyak *instance* dari satu entitas dapat berinteraksi dengan entitas lainnya, sehingga menghasilkan skema basis data yang logis (Lihat Gambar 2.8).



Gambar 2.8 Entity Relationship Diagram (ERD) [45]

2.2.15 Purposive Sampling

Sampling bertujuan (*purposive sampling*) adalah sebuah metode pengambilan sampel *non-probabilitas* di mana peneliti secara sengaja memilih partisipan berdasarkan penilaian (*judgment*) dan tujuan spesifik dari penelitian [19]. Berbeda dari sampling acak (*probability sampling*) yang mengejar generalisasi statistik, *purposive sampling* digunakan untuk mengidentifikasi individu atau kasus yang "kaya informasi" (*information-rich*) dan paling sesuai untuk berkontribusi pada pemahaman mendalam tentang fenomena yang diteliti. Teknik ini sering digunakan, baik dalam penelitian kualitatif maupun kuantitatif, ketika peneliti menargetkan individu yang memiliki pengetahuan khusus, pengalaman mendalam, atau memegang peran/posisi tertentu yang

relevan dengan fokus studi [20]. Salah satu variasi dari teknik ini adalah *Total Population Sampling* (Sampling Populasi Total), yang diterapkan ketika populasi yang dituju berukuran kecil, memiliki karakteristik unik, dan terdefinisi dengan jelas [20]. Dalam pendekatan ini, seluruh anggota populasi tersebut dilibatkan sebagai responden (Tertera pada Lampiran F), yang bertujuan untuk mengeliminasi bias sampling dan memberikan pemahaman yang komprehensif terhadap kelompok spesifik tersebut [19].

2.2.16 *Websocket*

Websocket dapat dipahami sebagai sebuah mekanisme komunikasi yang memungkinkan browser dan server saling bertukar data secara langsung dan terus-menerus melalui satu jalur koneksi yang tetap terbuka. Berbeda dengan komunikasi berbasis HTTP biasa, yang selalu mengharuskan klien mengirim permintaan terlebih dahulu sebelum server memberikan jawaban, *Websocket* menghilangkan kebutuhan akan proses tersebut. Setelah koneksi awal dibuka dan disetujui oleh server, keduanya dapat mengirim data kapan saja tanpa harus memulai ulang siklus komunikasi. Cara kerja yang langsung ini membuat pertukaran informasi berlangsung lebih cepat dan stabil. Selain itu, data yang dikirim melalui *Websocket* dikemas dengan cara yang sederhana dan ringan sehingga tidak membebani jaringan. Karena itulah, secara teoritis *Websocket* mampu memberikan kinerja yang lebih efisien dibanding metode komunikasi lain seperti *polling* atau *long polling*, terutama pada aplikasi yang memerlukan pembaruan data secara *real-time*, misalnya sistem monitoring, aplikasi chat, atau notifikasi langsung [46].

2.2.17 *Laravel*

Laravel adalah kerangka kerja pengembangan aplikasi web modern yang menyediakan berbagai alat canggih untuk membangun sistem, termasuk fitur-fitur khusus untuk menangani komunikasi data yang kompleks [47]. Yang digunakan pada penelitian ini adalah Laravel Reverb, Laravel Horizon, Laravel Notifications. Laravel Reverb berfungsi sebagai jalur komunikasi super cepat yang memungkinkan aplikasi memperbarui data di layar pengguna secara langsung (*real-time*) [48]. Sementara itu, Laravel Horizon bertindak sebagai pusat kendali cerdas yang memantau dan mengatur pekerjaan berat di belakang layar [49]. Untuk menjaga pengguna tetap terinformasi, Laravel Notifications bertugas mengirimkan pesan penting atau peringatan melalui berbagai saluran seperti email atau aplikasi pesan secara otomatis [50]. Ketika digabungkan, Reverb dan Horizon memastikan notifikasi dapat muncul seketika di layar pengguna sementara proses pengirimannya dikelola dengan baik agar tidak memberatkan kinerja aplikasi [48], [49].

2.2.18 Redis

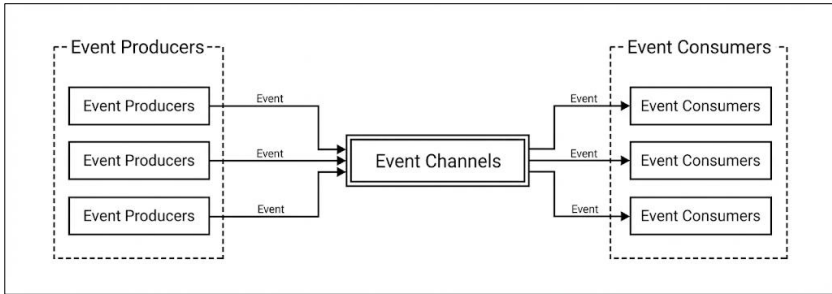
Redis (*Remote Dictionary Server*) didefinisikan sebagai penyimpanan struktur data *in-memory* jarak jauh yang bersifat *open-source* dan berkinerja tinggi, yang beroperasi dengan mekanisme pemetaan kunci ke berbagai jenis struktur data seperti *strings*, *lists*, *sets*, *hashes*, dan *sorted sets*. Berbeda dengan basis data tradisional yang menyimpan data pada *hard disk* dengan latensi akses tinggi, Redis menyimpan data di dalam memori utama (RAM) yang secara teoritis 150.000 kali lebih cepat, menjadikannya solusi efektif untuk mengatasi masalah *bottleneck* pada kinerja pembacaan (*read performance*) sistem. Selain kecepatan, Redis menawarkan fitur persistensi data ke disk, replikasi untuk redundansi, serta mendukung *horizontal scaling* yang lebih ekonomis dibandingkan *vertical scaling* untuk menangani beban *throughput* yang besar [51].

2.2.19 XHR Polling

XHR Polling merupakan mekanisme komunikasi konvensional di mana *client* secara berkala mengirimkan permintaan HTTP ke *server* pada interval waktu yang telah ditentukan untuk memvalidasi ketersediaan data baru. Metode ini memiliki kelemahan mendasar berupa inefisiensi penggunaan *bandwidth* dan beban kerja *server* yang berlebihan, karena koneksi harus terus dibangun berulang kali meskipun tidak terdapat pembaruan informasi yang signifikan untuk dikirimkan kembali. Latensi penerimaan data menjadi konsekuensi yang tidak terhindarkan karena klien harus menunggu siklus permintaan berikutnya untuk mendapatkan status terkini. Kondisi ini menjadikan metode *polling* kurang reliabel untuk diterapkan pada arsitektur sistem yang menuntut standar responsivitas waktu nyata [9].

2.2.20 Event Driven

Arsitektur *Event Driven* adalah paradigma sistem yang mengandalkan produksi dan deteksi peristiwa (*events*) sebagai pemicu utama alur eksekusi program secara asinkron [52]. Dalam pendekatan ini, ketika suatu peristiwa terjadi di dalam atau di luar sistem, peristiwa tersebut segera didistribusikan kepada semua pihak yang tertarik dan memicu tindakan yang sesuai. Model komunikasi berbasis *event* memfasilitasi pemisahan ketergantungan (*decoupling*) antar komponen, di mana produsen pesan tidak perlu mengetahui konsumen spesifik dari pesan tersebut, sehingga memungkinkan komponen beroperasi secara independen [52]. Karakteristik *loose coupling* ini memberikan fleksibilitas dalam menangani perubahan sistem tanpa memerlukan modifikasi pada komponen produsen pesan.



Gambar 2.9 Ilustrasi Arsitektur *Event-Driven*

Gambar 2.9 mengilustrasikan mekanisme kerja mendasar dalam arsitektur *event-driven*. Pada paradigma ini, produksi dan pendeteksian peristiwa (*events*) berfungsi sebagai pemicu utama alur eksekusi program yang berjalan secara asinkron. Model komunikasi tersebut memfasilitasi pemisahan ketergantungan (*decoupling*) antar komponen, yang memungkinkan bagian produsen dan konsumen pesan untuk beroperasi secara independen.

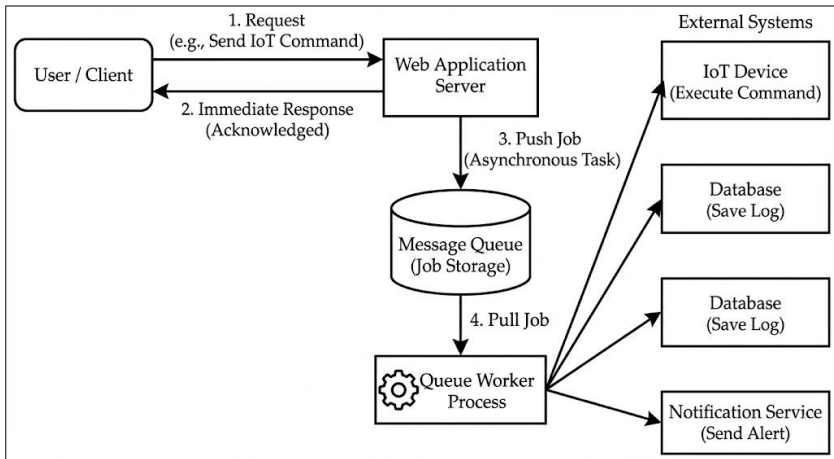
2.2.21 Cron Scheduler

Cron scheduler adalah mekanisme penjadwalan otomatis yang berasal dari sistem Unix/Linux. Fungsinya adalah menjalankan perintah atau *script* pada waktu-waktu tertentu yang telah ditentukan sebelumnya, misalnya setiap jam, setiap hari, atau pada waktu spesifik seperti jam 6 pagi. Dalam sistem otomatisasi berbasis web, *cron scheduler* berfungsi sebagai pemicu (*trigger*) untuk mengecek aturan otomatis yang disimpan di database. Alurnya sederhana: (1) *scheduler* menjalankan perintah pada waktu yang dijadwalkan, (2) perintah tersebut membaca aturan otomatis dari database untuk melihat mana yang harus dijalankan, (3) perintah menciptakan tugas (*job*) yang akan dijalankan kemudian. Keuntungan pemisahan ini adalah *scheduler* hanya bertugas memicu, sementara eksekusi sesungguhnya ditangani oleh sistem lain (*queue worker*). Dengan cara ini, konfigurasi jadwal dapat diubah tanpa perlu menghentikan *scheduler* [53].

2.2.22 Queue Worker

Queue worker adalah sistem yang menangani pekerjaan-pekerjaan berat secara terpisah dan tidak sinkron (*asynchronous*). Sistem ini menyimpan pekerjaan dalam antrian (*queue*), kemudian *worker* mengambil satu pekerjaan, mengerjakannya sampai selesai, dan menandainya sebagai selesai atau gagal. Dalam sistem otomatisasi IoT, *queue worker* digunakan untuk menangani operasi yang memakan waktu atau tergantung pada sumber eksternal, seperti mengirim perintah ke perangkat IoT, menyimpan *log* ke *database*, atau mengirim notifikasi. Keuntungan utama adalah aplikasi *web* tidak perlu menunggu operasi-operasi ini selesai, namun aplikasi dapat langsung

merespons pengguna, sementara pekerjaan berat dikerjakan di belakang layar oleh *worker*. Jika pekerjaan gagal, sistem dapat secara otomatis mencoba lagi (Lihat Gambar 2.10) [54].



Gambar 2.10 Ilustrasi *Queue Worker*

2.2.23 ISO/IEC 25010

Dalam arsitektur *System and Software Quality Requirements and Evaluation* (SQuaRE), ISO/IEC 25010 ditetapkan sebagai instrumen fundamental untuk memetakan parameter kelayakan perangkat lunak (Rojas et al., 2025). Validitas model ini diperkuat oleh studi Rojas et al. (2025) yang menempatkannya sebagai acuan universal dalam pengukuran performa sistem komputasi lintas *platform* [55]. Meskipun kerangka kerja ini membagi evaluasi ke dalam aspek *quality in use* dan *product quality*, penelitian ini membatasi ruang lingkup pengujian secara eksklusif pada dimensi *product quality*. Pembatasan ini dilakukan untuk mengisolasi analisis pada karakteristik internal dan struktur logika kode program, memastikan bahwa spesifikasi teknis terpenuhi secara fundamental sebelum ditinjau dari perspektif interaksi pengguna. Rincian karakteristik kualitas produk berdasarkan standar ISO/IEC 25010 disajikan dalam Tabel 2.5 [55].

Tabel 2.5 Karakteristik dan Deskripsi Kualitas ISO/IEC 25010

No	Karakteristik Kualitas	Deskripsi Teknis
1	<i>Functional Suitability</i>	Tingkat sejauh mana fitur sistem memenuhi spesifikasi kebutuhan fungsional untuk mencapai tujuan pengguna secara akurat.
2	<i>Performance Efficiency</i>	Pengukuran kinerja sistem relatif terhadap penggunaan sumber daya komputasi dan waktu respons di bawah beban kerja tertentu.

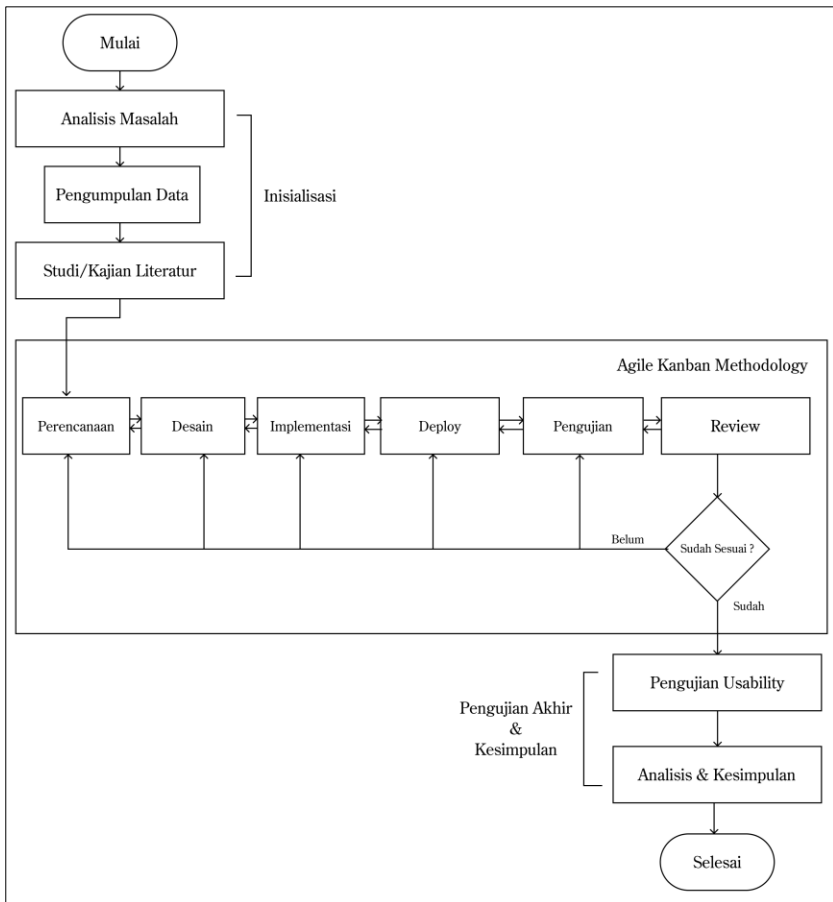
No	Karakteristik Kualitas	Deskripsi Teknis
3	<i>Compatibility</i>	Kapabilitas sistem untuk bertukar data atau beroperasi berdampingan dengan perangkat lunak lain tanpa konflik.
4	<i>Usability</i>	Derajat kemudahan antarmuka sistem untuk dipelajari, dipahami, dan dioperasikan oleh pengguna guna mencapai interaksi efektif.
5	<i>Reliability</i>	Probabilitas sistem beroperasi tanpa kegagalan dan mempertahankan kinerja stabil dalam kurun waktu serta kondisi tertentu.
6	<i>Security</i>	Kemampuan sistem melindungi integritas data dari akses ilegal serta menjamin hak akses hanya bagi pengguna terotorisasi.
7	<i>Portability</i>	Fleksibilitas sistem untuk dipindahkan atau diadaptasi dari satu lingkungan perangkat keras/sistem operasi ke lingkungan lain.

BAB III

METODE PENELITIAN

3.1 Alur Penelitian

Alur penelitian merupakan rangkaian tahapan sistematis yang dilakukan mulai dari perencanaan hingga penyelesaian penelitian. Gambar 3.1 menunjukkan visualisasi alur dalam bentuk diagram alir, sedangkan Tabel 3.1 berisi penjelasan singkat dari setiap tahapan (Tahapan detail seluruh alur penelitian terdapat pada sub-bab 3.3).



Gambar 3.1 Alur Penelitian

Tabel 3.1 Penjelasan Alur Penelitian

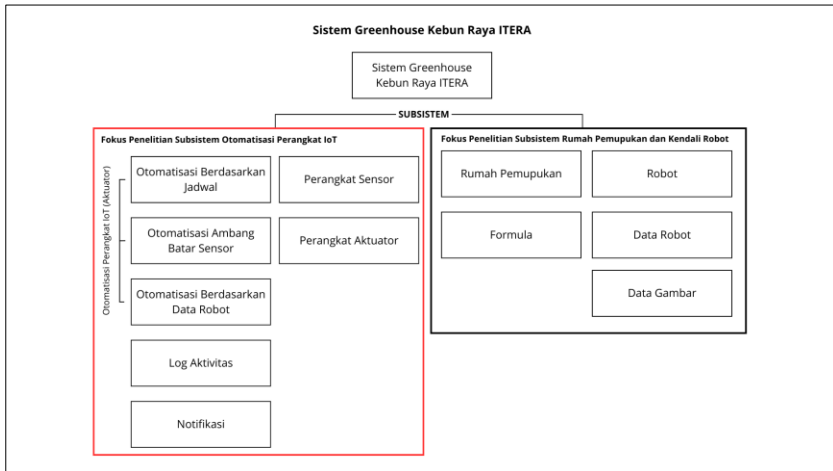
Tahapan	Proses	Deskripsi
Inisialisasi	Analisis Masalah	Mengidentifikasi inefisiensi komunikasi data metode <i>XHR Polling</i> pada sistem lama serta memetakan kebutuhan

Tahapan	Proses	Deskripsi
		subsistem otomatisasi (penjadwalan, ambang batas sensor, dan data definisi rekomendasi robot) berdasarkan masukan Penanggung Jawab Greenhouse.
	Pengumpulan Data	Mengumpulkan data teknis dan operasional melalui wawancara mendalam dengan mitra dan observasi langsung di Greenhouse Kebun Raya ITERA.
	Studi/Kajian Literatur	Mengkaji landasan teori terkait IoT, protokol komunikasi <i>real-time</i> (MQTT & <i>Websocket</i>) serta manajemen proses asinkron (<i>Queue & Scheduler</i>), serta metode pengembangan Agile Kanban dan standar pengujian yang relevan.
Agile Kanban Metodologi (Detail implementasi terdapat pada Lampiran D)	Perencanaan	Mengidentifikasi kebutuhan fungsional dan non-fungsional dari subsistem <i>website</i> otomatisasi perangkat IOT berdasarkan analisis masalah.
	Desain	Merancang arsitektur komunikasi, <i>use case diagram</i> , <i>activity diagram</i> , ERD, dan <i>wireframe</i> sebagai <i>blueprint</i> arsitektur subsistem.
	Implementasi	Tahap mengembangkan <i>frontend</i> dan <i>backend</i> sesuai desain yang dibuat.
	<i>Deploy</i>	Menjalankan sistem ke lingkungan operasional agar dapat diakses dan digunakan pengguna.
	Pengujian	Melakukan pengujian Fungsional(<i>blackbox testing</i>), Non-Fungsional(k6) dan MQTXX dari setiap fitur yang dikembangkan pada subsistem.
	<i>Review</i> oleh penanggung jawab penelitian di greenhouse Kebun Raya ITERA	Melakukan evaluasi hasil pengembangan dengan penanggung jawab greenhouse Kebun Raya ITERA untuk memastikan kesesuaian dengan kebutuhan.
Pengujian Akhir & Kesimpulan	Pengujian <i>Usability</i>	Mengukur tingkat kebergunaan sistem menggunakan metode <i>System Usability Scale</i> (SUS) kepada <i>stakeholder</i> serta pengguna terkait.
	Analisis & Kesimpulan	Menganalisis hasil penelitian ini berdasarkan hasil pengujian Fungsional (<i>blackbox testing</i>), Non-Fungsional(k6), Keberhasilan Uji Komunikasi dan Otomatisasi (MQTXX), serta <i>usability</i> dengan SUS.

3.2 Lingkup Penelitian

Kompleksitas operasional pada *Greenhouse* Kebun Raya ITERA menuntut adanya pemisahan fungsional yang tegas guna membatasi ruang lingkup pengembangan dan mengeliminasi risiko redundansi fitur antar pengembang. Sebagaimana divisualisasikan pada Gambar 3.2, arsitektur

sistem dikelompokkan menjadi dua entitas utama, yakni *subsistem* otomatisasi perangkat *IoT* dan *subsistem* manajemen rumah pemupukan serta kendali robot berbasis *website*. Strukturisasi ini bertujuan memastikan setiap domain memiliki fokus penyelesaian masalah yang spesifik namun tetap terintegrasi dalam satu ekosistem data.



Gambar 3.2 Fokus Penelitian Otomatisasi Perangkat IoT

Berdasarkan diagram blok Gambar 3.2, ruang lingkup penelitian ini dibatasi secara spesifik pada area Subsistem Otomatisasi Perangkat IoT yang berfungsi sebagai unit kendali pusat untuk manajemen perangkat keras di dalam rumah kaca. Logika pengendalian subsistem ini dirancang menggunakan pendekatan banyak variabel sehingga pengaktifan perangkat aktuator dapat dipicu oleh penjadwalan waktu, ambang batas pembacaan sensor, atau instruksi berbasis data dari robot. Integrasi tersebut menciptakan siklus umpan balik tertutup di mana data yang diperoleh sensor divalidasi oleh modul otomatisasi untuk menentukan keputusan teknis yang kemudian dijalankan oleh aktuator. Demi menjamin akuntabilitas operasional, fitur catatan aktivitas dan notifikasi diterapkan sebagai mekanisme pelaporan agar setiap perubahan status perangkat, baik karena pemicu otomatis maupun intervensi manual, terekam secara digital guna mempermudah penelusuran apabila terjadi anomali pada kondisi lingkungan.

3.3 Alat dan Bahan

Penelitian ini menggunakan sejumlah alat dan bahan pendukung yang spesifikasinya diuraikan secara rinci pada sub-bab 3.2.1 Alat Penelitian dan sub-bab 3.2.2 Bahan Penelitian.

3.2.1 Alat Penelitian

Penelitian ini menggunakan perangkat keras dan perangkat lunak dengan spesifikasi yang disesuaikan dengan kebutuhan analisis, perancangan, dan implementasi. Rincian alat penelitian disajikan pada Tabel 3.2.

Tabel 3.2 Alat Penelitian

Nama Alat	Spesifikasi	Keterangan
<i>Laptop</i>	Laptop AMD Ryzen 4600H, RAM 24 GB, SSD 512 GB, OS Windows 11	Perangkat utama peneliti untuk pengembangan dan pengujian subsistem <i>website</i>
<i>Virtual Private Server (VPS)</i>	Sistem operasi Ubuntu 24.04 LTS, CPU 4 Core 3.25 GHz, kapasitas penyimpanan 200 GB, serta RAM sebesar 16 GB.	Lingkungan <i>server</i> untuk <i>men-deploy</i> seluruh sistem termasuk aplikasi <i>backend</i> dan basis data agar sistem dapat diakses secara <i>online</i> .
<i>Visual Studio Code</i>	Versi 1.105.1	Editor kode untuk implementasi pengembangan subsistem
GitHub	Github : github.com	Platform berbasis <i>Git</i> untuk manajemen kode sumber (<i>source code</i>) dan kendali versi (<i>version control</i>) guna memfasilitasi kolaborasi pengembangan.
Figma & Draw.io	Figma : Versi 125.10.5 Draw.io : <i>Website</i>	Perancangan desain <i>Use Case</i> , ERD, <i>activity diagram</i> , dan <i>wireframe</i>
<i>Browser</i>	Chrome, Firefox, Brave, dll	Kebutuhan pengembangan dan tahap pengujian subsistem <i>website</i> otomatisasi perangkat IOT
MQTTX	Versi 1.12.1	Sebagai alat pengujian komunikasi protokol MQTT subsistem ke perangkat IoT atau sebaliknya.

3.2.2 Bahan Penelitian

Bahan penelitian yang digunakan mencakup data dari wawancara penanggung jawab *greenhouse* Kebun Raya ITERA, dokumentasi *website*, serta data yang diperoleh selama proses desain, implementasi, pengujian dan analisis kesimpulan. Rincian bahan penelitian disajikan pada Tabel 3.3.

Tabel 3.3 Kebutuhan Bahan Penelitian

Nama Bahan	Sumber	Kegunaan
Dokumentasi Sistem Terdahulu	Laporan dan kode program penelitian sebelumnya [8].	Dasar analisis untuk mengidentifikasi inefisiensi komunikasi data (XHR <i>Polling</i>) yang menjadi urgensi perancangan ulang subsistem.
Data Kebutuhan Pengguna	Data wawancara dari penanggung jawab <i>greenhouse</i> Kebun Raya ITERA. (Lihat Tabel 3.4)	Memetakan kebutuhan fungsional pengguna terkait permasalahan, alur kerja operasional, dan aturan logika otomatisasi di <i>greenhouse</i> .
Referensi Literatur & Teknis	Jurnal ilmiah, buku, dan dokumentasi resmi (<i>official documentation</i>).	Landasan teoritis penerapan teknologi komunikasi <i>real-time</i> (Websocket/MQTT), manajemen proses asinkron (<i>Queue</i>), metode <i>Agile Kanban</i> , dan standar pengujian

Nama Bahan	Sumber	Kegunaan
		yang relevan dan mendukung penelitian.
Data Perangkat IoT dan Aturan Otomatisasi	Kebutuhan otomatisasi dari wawancara <i>penanggung jawab</i> , dan salah satu pengelola pada <i>greenhouse Kebun Raya ITERA</i> .	Menentukan angka ambang batas (<i>threshold</i>) sensor, aturan waktu penjadwalan, dan logika respon terhadap data definisi robot sebagai inti algoritma kendali otomatis.
Data pengujian	Proses pengujian dengan <i>Black Box Testing</i> , k6, MQTTX, dan kuesioner SUS.	Bukti empiris untuk memvalidasi keberhasilan fungsi fitur, stabilitas komunikasi data dan otomatisasi, serta tingkat penerimaan pengguna terhadap subsistem.

3.4 Tahapan Seluruh Alur Penelitian

Pembahasan lengkap alur penelitian dalam penelitian ini terdiri dari tiga tahap utama, yaitu Inisialisasi, Tahapan Agile Kanban, dan Analisis Kesimpulan. Setiap tahap memiliki peran penting dalam memastikan proses penelitian berjalan secara sistematis dan terarah. Uraian dari masing-masing tahap dijelaskan dengan rinci pada 3.3.1 – 3.3.4.

3.3.1 Inisialisasi

1. Analisis Masalah

Analisis kode sistem terdahulu (Lihat Gambar 3.2 dan 3.3) mengidentifikasi inefisiensi metode *XHR Polling* yang menghasilkan trafik data 49,5 KB/s dan beban CPU 4,11%, jauh melampaui *Websockets* yang hanya membutuhkan 4,3 KB/s dan CPU 0,26% [46]. Kesenjangan performa ini disebabkan oleh *overhead* HTTP sebesar 800 bytes per permintaan dibandingkan *Websockets* yang hanya berkisar 2 hingga 10 bytes. Oleh sebab itu, transisi menuju integrasi protokol komunikasi *real-time* dan manajemen proses asinkron diterapkan guna menghilangkan beban server dan menjamin responsivitas perangkat IoT [46].

```

const getValueRefreshFirst = async () => {
  setTimeout(() => {
    axios.get(`${brokerSensor}${idSensor}`) // Request XHR
    .then(response => {
      setValueSensor(response.data.data[0],value)
      setStatus(response.data.data[0].status)
      setTime(response.data.data[0].createdAt)
      setOnRefresh(true) // Memicu useEffect untuk loop selanjutnya
    })
  }, 1000) // Interval 1 detik
}

const getValueRefreshSecond = async () => {
  setTimeout(() => {
    axios.get(`${brokerSensor}${idSensor}`) // Request XHR
    .then(response => {
      setValueSensor(response.data.data[0],value)
      setStatus(response.data.data[0].status)
      setOnRefresh(false) // Memicu useEffect untuk loop selanjutnya
    })
  }, 1000) // Interval 1 detik
}

```

(a) Fungsi Eksekusi Polling (*Request & Delay*)

```

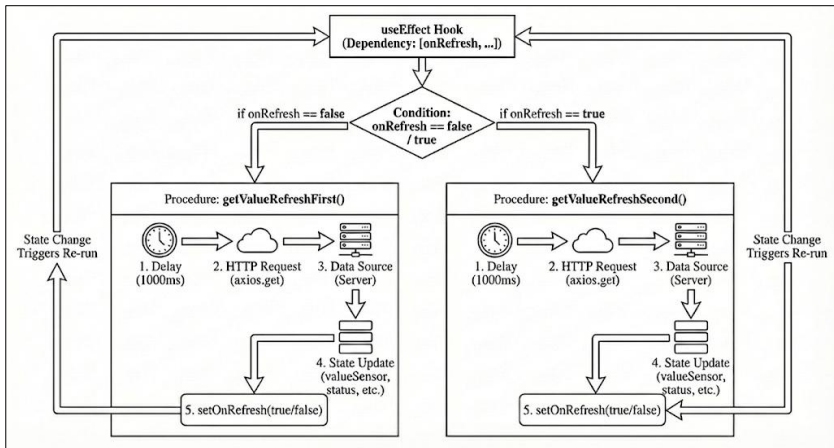
const onRefreshUpdate = () => {
  // Logic ping-pang: jika false panggil First, jika true panggil Second
  if(onRefresh === false){
    getValueRefreshFirst()
  }
  if (onRefresh === true){
    getValueRefreshSecond()
  }
}

useEffect(() => {
  if(!firstCheck === true){
    getValue()
  }
  else{
    onRefreshUpdate() // Dipanggil setiap kali state 'onRefresh' berubah
  }
  return () => setFirstCheck(false);
  // Dependency array ini menyebabkan loop tak terbatas (Polling)
  // karena onRefresh selalu berubah di dalam fungsi getValueRefresh...
},[onRefresh, firstCheck, valueSensor,status]);

```

(b) Logika Pengulangan (*Looping Logic*)

Gambar 3.3 (a) dan (b) Masalah *XHR Polling* Code Sensor [8]



Gambar 3.4 Alur Logika Mekanisme XHR Polling

Gambar 3.3 memvisualisasikan alur kerja sistem terdahulu dengan mekanisme *XHR Polling*, di mana *hooks useEffect* memicu permintaan data berulang setiap 1000 milidetik. Logika ini memaksa aplikasi terus mengirimkan *HTTP Request* ke *server* untuk memeriksa pembaruan, tanpa memedulikan kondisi data aktual di lapangan. Siklus tersebut mengakibatkan inefisiensi kinerja karena membebani proses komputasi serta meningkatkan konsumsi *bandwidth* akibat pengiriman *header* berulang yang seharusnya dapat dihindari.

2. Pengumpulan Data

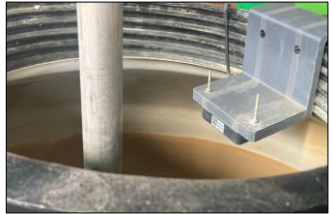
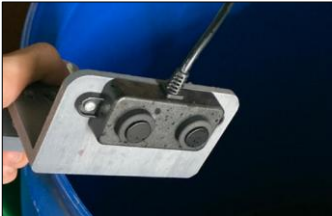
Pengumpulan data dilakukan untuk mengidentifikasi spesifikasi teknis dan logika operasional yang diperlukan dalam pengembangan subsistem otomatisasi perangkat IoT. Proses ini dilaksanakan melalui observasi infrastruktur sensor, aktuator, data robot di lapangan serta wawancara dengan pengelola (Lihat Tabel 3.4, detail instrumentasi pada Lampiran B) guna merumuskan skenario kendali otomatis sesuai kebutuhan mitra. Selain itu, dokumentasi sistem terdahulu dikaji sebagai acuan standar integrasi arsitektur komunikasi. Seluruh rincian parameter data yang menjadi landasan analisis dan perancangan sistem dijabarkan pada Tabel 3.5, Tabel 3.6, dan Tabel 3.7.

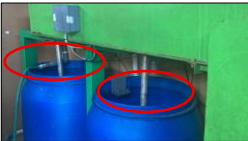
Tabel 3.4 Hasil Wawancara

No.	Ringkasan Poin Jawaban
1.	Sistem lama membantu, tetapi respons perangkat sering lambat.
2.	Pengaturan suhu, penyiraman, dan pemupukan berjalan terpisah.
3.	Notifikasi sering terlambat dan kurang informatif.
4.	Tampilan antarmuka kurang modern dan tidak konsisten di HP.
5.	Diperlukan penjadwalan otomatis untuk penyiraman dan pengadukan tandon.
6.	Sistem lama tidak stabil saat server berat sehingga jadwal kadang telat.
7.	<i>Greenhouse</i> berencana mengintegrasikan otomatisasi berdasarkan data definisi rekomendasi robot

No.	Ringkasan Poin Jawaban
8.	Pengguna membutuhkan notifikasi keberhasilan/gagal otomatisasi dan informasi stok nutrisi.
9.	Pengguna ingin mengetahui perangkat mana yang menyala, kapan menyala, dan siapa yang mengubah aturan.

Tabel 3.5 Daftar Perangkat dan Data Definisi Robot

No.	[Kode] Nama + Gambar	Deskripsi Singkat dan Fungsi
Perangkat Sensor		
1.	[S-01]A Suhu dan [S-01]B Kelembapan 	Tipe: Temperature and Humidity Transmitter Modbus SHT20 Sensor XY-MD02 - XY-MD02 Mengukur suhu udara dan kelembapan relatif di dalam <i>greenhouse</i> secara <i>real-time</i> untuk pemantauan iklim mikro.
2.	[S-02] Total Dissolved Solids (TDS) 	Tipe: MODBUS RTU RS485 0 - 2000 PPM Mengukur konduktivitas listrik (<i>electrical conductivity</i>) larutan untuk dikonversi menjadi nilai <i>Total Dissolved Solids</i> (TDS), berfungsi memvalidasi konsentrasi nutrisi sebelum didistribusikan.
3.	[S-03] Level Ketinggian Tandon Air 	Tipe: Waterproof Ultrasonic Sensor - A02YYUW - UART Memanfaatkan prinsip pantulan gelombang ultrasonik sebagai data untuk menghitung jarak permukaan air baku, data dikonversi menjadi persentase volume ketersediaan air pada <i>microcontroller</i> .
4.	[S-04] Level Ketinggian Nutrisi Pupuk 	Tipe: Waterproof Ultrasonic Sensor - A02YYUW - UART Mendeteksi jarak permukaan stok larutan nutrisi dengan menggunakan gelombang ultrasonik, yang berfungsi memberikan peringatan dini apabila stok bahan baku kritis.
.Data Definisi Rekomendasi Robot		
5.	[DR-01] Ml/tanaman	Data kiriman dari robot untuk otomatisasi perangkat pompa sesuai ml/tanaman, dengan menyalakan sesuai durasi yang ditentukan.

No.	[Kode] Nama + Gambar	Deskripsi Singkat dan Fungsi
Perangkat Aktuator		
6.	[A-01] Cooling Pad 	Perangkat ini bekerja dengan memanfaatkan proses penguapan air saat udara melewati media basah. Proses tersebut menyerap panas dari udara lingkungan, sehingga menurunkan suhu dan meningkatkan kelembapan udara sebelum masuk ke area tanam.
7.	[A-02] Exhaust Fan 	Berfungsi menarik udara panas dari dalam <i>greenhouse</i> ke luar ruangan. Mekanisme ini meningkatkan pergantian udara dan membantu menurunkan suhu di dalam ruangan.
8.	[A-03] Agitator Pencampuran 	Bertugas mengaduk larutan di tandon pencampuran akhir menggunakan motor penggerak. Tujuannya adalah memastikan nutrisi tercampur rata sebelum dialirkan ke sistem penyiraman, serta mencegah terjadinya endapan.
9.	[A-04] Agitator Bahan Nutrisi 	Berfungsi menjaga larutan bahan nutrisi di tangki penyimpanan agar tetap tercampur rata dan mencegah pengendapan yang dapat mempengaruhi ketepatan takaran nutrisi.
10.	[A-05] Pompa Listrik Nutrisi 	Berfungsi memindahkan cairan nutrisi dari tangki penyimpanan menuju tandon pencampuran utama untuk proses pengenceran sesuai takaran yang dibutuhkan.
11.	[A-06] Katup Air Tandon Pencampuran 	Merupakan katup elektrik yang berfungsi mengatur mekanisme buka-tutup aliran air untuk proses pengisian tandon utama pencampuran, dengan kendali manual/otomatis yang beroperasi berdasarkan instruksi dari sistem pusat.
12.	[A-07] Pompa Penyiraman 	Bertugas memberikan tekanan yang cukup untuk mengalirkan larutan nutrisi yang sudah tercampur dari tandon utama menuju seluruh jaringan pipa penyiraman di area tanam.

Tabel 3.6 Data Kebutuhan Otomatisasi

No.	Nama Instrumen	Kode Perangkat Terlibat	Deskripsi Mekanisme Otomatisasi
Tipa Otomatisasi: Ambang Batas Sensor			
1	Pengendalian Suhu	[S-01] A Suhu [A-01] <i>Exhaust Fan</i>	ON saat suhu $\geq 32^{\circ}\text{C}$ OFF saat suhu $\leq 27^{\circ}\text{C}$
2	Pengendalian Kelembapan	[S-01] B Kelembapan [A-02] <i>Cooling Pad</i>	ON saat kelembapan $\leq 65\%$ OFF saat kelembapan $\geq 75\%$
3	Pengisian Air Tandon	[S-03] Level Ketinggian Air [A-06] Katup Air Tandon	ON saat level air $\leq 10\%$ OFF saat level air $\geq 80\%$
4	Penyesuaian Konsentrasi	[S-02] TDS [A-05] Pompa Listrik Nutrisi	ON saat TDS $\leq$ minimum OFF saat TDS $\geq$ target
Tipa Otomatisasi: Penjadwalan			
5	Jadwal Suhu	[A-01] <i>Exhaust Fan</i>	Aktif setiap hari pukul 07:00 [Durasi: 10 jam]
6	Jadwal Kelembapan	[A-02] <i>Cooling Pad</i>	Aktif setiap hari pukul 07:00 [Durasi: 10 jam]
7	Pengadukan Tandon Air Utamar	[A-03] Agitator Pencampuran	Aktif setiap hari pukul 06:55 [Durasi: 300 detik] [5 pengulangan, interval 2 jam]
8	Pengadukan Tandon Nutrisi	[A-04] Agitator Bahan Nutrisi	Aktif setiap hari pukul 06:55 [Durasi: 300 detik] [5 pengulangan, interval 2 jam]
9	Penyiraman Nutrisi	[A-07] Pompa Penyiraman	Aktif setiap hari pukul 07:00 [Durasi: 300 detik] [5 pengulangan, interval 2 jam]
Tipa Otomatisasi: Data definisi robot			
10	Penyiraman Nutrisi	[DR-01] MI/Tanaman [A-07] Pompa Penyiraman	ON saat ml/tanaman 120-380 OFF setelah beberapa menit

Catatan: Angka otomatisasi berdasarkan wawancara, dan kebutuhan lapangan

3. Studi dan Kajian Literatur

Tahap studi literatur telah dilakukan dan diuraikan secara lengkap pada BAB II, meliputi sub-bab 2.1 Tinjauan Pustaka dan 2.2 Dasar Teori sebagai landasan *teoritis, konseptual*, serta *metodologis* penelitian ini.

3.3.2 Proses Metode Agile Kanban

Metodologi pengembangan sistem dalam penelitian ini menerapkan kerangka kerja *Agile Kanban* untuk mengakomodasi kebutuhan alur kerja yang adaptif dan transparan. Pendekatan ini dipilih karena keunggulannya dalam memvisualisasikan progres tugas secara jelas serta meminimalkan hambatan (*bottleneck*) melalui mekanisme pembatasan pekerjaan yang sedang berlangsung (*Work In Progress*) [23]. Sesuai dengan alur penelitian, siklus pengembangan dilaksanakan secara iteratif dan terstruktur melalui lima tahapan utama, yakni Perencanaan, Desain, Implementasi, Pengujian, Deploy, dan Tinjauan penanggung jawab *greenhouse* Kebun Raya ITERA.

1. Perencanaan

Tahap perencanaan difokuskan pada pendefinisian ruang lingkup teknis untuk subsistem otomatisasi berdasarkan hasil analisis masalah dan kebutuhan

mitra. Pada fase ini, seluruh persyaratan operasional dipetakan menjadi spesifikasi kebutuhan sistem yang terukur guna memastikan fungsi otomatisasi yang dikembangkan tepat sasaran dan dapat diimplementasikan. Spesifikasi yang disusun sebagai landasan pengembangan ini dikelompokkan menjadi dua kategori utama, yaitu:

a. Kebutuhan Fungsional

Kebutuhan fungsional pada Tabel 3.8 menjelaskan fungsi-fungsi yang harus disediakan oleh subsistem yang dirancang untuk mendukung aktivitas pengguna. Setiap kebutuhan diberi kode [F-XX] yang merujuk pada analisis masalah dan pengumpulan data. Daftar lengkap kebutuhan fungsional tersebut dirincikan pada Tabel 3.7.

Tabel 3.7 Kebutuhan Fungsional

Kode	Nama Fitur	Hak Akses	Keterangan Singkat
F-01	Login dan Logout Subsistem	Staff, Pengelola, Admin	Mengelola autentikasi pengguna untuk akses fitur berbasis hak akses serta terminasi sesi sistem.
F-02	Manajemen Perangkat Sensor	Staff (lihat), Pengelola & Admin (kelola)	Memantau nilai dan status konektivitas sensor secara <i>real-time</i> serta mengelola konfigurasi perangkat.
F-03	Manajemen Perangkat Aktuator	Staff (lihat), Pengelola & Admin (kelola)	Memantau status operasional aktuator dan mengelola kontrol manual serta konfigurasi perangkat.
F-04	Otomatisasi Berdasarkan Penjadwalan	Staff (lihat), Pengelola & Admin (kelola)	Memicu eksekusi perangkat IoT (ON/OFF) secara otomatis berdasarkan parameter waktu yang ditentukan.
F-05	Otomatisasi Berdasarkan Ambang Batas Sensor	Staff (lihat), Pengelola & Admin (kelola)	Memicu aksi perangkat IoT secara otomatis saat nilai sensor melewati batas parameter (<i>threshold</i>) yang ditetapkan.
F-06	Otomatisasi Berdasarkan Data Definisi Rekomendasi Robot	Staff (lihat), Pengelola & Admin (kelola)	Mengeksekusi instruksi kontrol aktuator secara otomatis berdasarkan data rekomendasi yang dikirim dari robot.
F-07	Notifikasi Otomatisasi dan Rangkuman Otomatisasi Harian	Staff, Pengelola, Admin	Mengirimkan peringatan (<i>alert</i>) sistem terkait kondisi kritis, stok habis, atau anomali perangkat. Menyajikan rekapitulasi data aktivitas otomatisasi yang telah berjalan dalam periode 24 jam.
F-08	Log Seluruh Aktivitas dan Otomatisasi	Admin	Merekam seluruh riwayat aktivitas sistem, pemicu otomatisasi, dan interaksi pengguna untuk keperluan tinjauan aktivitas.

b. Kebutuhan Non-Fungsional

Analisis kebutuhan non-fungsional didefinisikan untuk menetapkan batasan teknis dan standar kualitas yang mengatur perilaku operasional sistem di luar fitur utama. Rincian spesifikasi tersebut dipetakan secara terstruktur dalam Tabel 3.8 sebagai acuan parameter performa perangkat lunak maupun

keras. Setiap atribut kebutuhan diberikan identitas unik dengan kode [NF-XX] guna memudahkan penelusuran (*traceability*) dan validasi pada tahap pengujian. Pemenuhan kendala ini bertujuan menjamin keandalan (*reliability*) sistem agar mampu beroperasi secara stabil tanpa mengalami kegagalan teknis saat beban kerja meningkat.

Tabel 3.8 Daftar Kebutuhan *Non-Fungsional*

Kode	Kategori	Deskripsi Kebutuhan
[NF-01]	<i>Performance</i>	Sistem harus mampu memberikan respons data rata-rata di bawah 200 ms pada kondisi beban normal untuk menjamin fluiditas interaksi pengguna.
[NF-02]	<i>Availability</i>	Subsistem otomatisasi harus beroperasi 24 jam sehari (24/7) untuk tetap bisa diakses dan bisa menjalankan logika otomatisasi perangkat IoT.
[NF-03]	<i>Security</i>	Sistem wajib menerapkan mekanisme autentikasi untuk membatasi hak akses dan mencatat identitas pengguna (nama dan email) pada setiap riwayat aktivitas demi akuntabilitas.
[NF-04]	<i>Reliability</i>	Sistem harus memiliki ketahanan (<i>resilience</i>) yang tinggi, dipastikan dengan tidak adanya kegagalan permintaan (<i>zero error rate</i>) meskipun beroperasi pada kondisi saturasi beban pengguna.
[NF-05]	<i>Usability</i>	Antarmuka pengguna harus dirancang intuitif dengan standar kelayakan <i>Acceptable</i> berdasarkan pengujian SUS, meminimalisir hambatan kognitif dalam navigasi fitur.
[NF-06]	<i>Interoperability</i>	Sistem harus mendukung protokol komunikasi standar (MQTT) untuk pertukaran data secara <i>real-time</i> antara server, perangkat sensor, dan aktuator.
[NF-07]	<i>Portability</i>	Aplikasi harus bersifat responsif dan dapat diakses melalui berbagai peramban web (<i>web browser</i>) pada perangkat seluler maupun desktop tanpa instalasi tambahan.

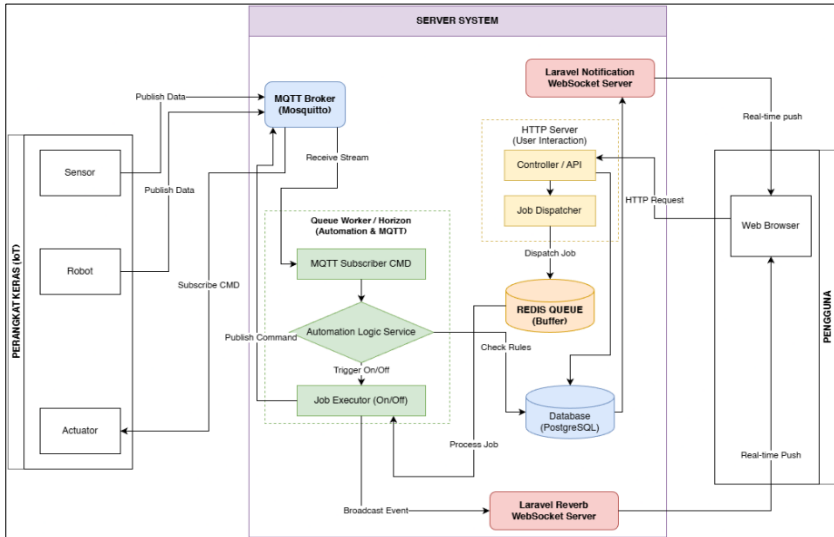
2. Desain

Tahap desain dilakukan setelah kebutuhan sistem terdefinisi dengan jelas. Desain disusun dalam bentuk *use case diagram*, *activity diagram*, ERD dan *wireframe* untuk memvisualisasikan sistem sebelum tahap implementasi.

a. Desain Rancangan Alur Komunikasi Subsistem

Rancangan komunikasi sistem ini difokuskan pada integrasi protokol komunikasi *real-time* dan manajemen proses asinkron guna mengatasi inefisiensi metode terdahulu. Alur dimulai saat data dari sensor dan robot *publish* data, diterima oleh MQTT Broker, dan diteruskan ke layanan *subscriber* untuk dievaluasi terhadap aturan otomatisasi di basis data. Apabila kondisi aturan terpenuhi, sistem tidak langsung mengeksekusi perintah, melainkan mengirimkan tugas (*dispatch job*) ke dalam antrian (*queue*) berbasis Redis. Selanjutnya, *Queue Worker* memproses tugas tersebut secara latar belakang (*background processing*) untuk mengirimkan perintah kendali ke perangkat aktuator melalui MQTT Broker. Secara paralel, setiap pembaruan

data sensor maupun perubahan status aktuator langsung disiarkan (*broadcast*) menggunakan teknologi *WebSocket* (Laravel Reverb) ke antarmuka pengguna, sehingga *monitoring* berjalan secara *real-time* tanpa memerlukan mekanisme *polling* dari sisi klien (Lihat Gambar 3.5).



Gambar 3.5 Arsitektur Subsistem Otomatisasi Perangkat IoT

b. Bentuk *Payload* Data

Implementasi mekanisme otomatisasi bergantung pada pertukaran data yang terstruktur antara perangkat keras dan *server*. Struktur *payload* dan hierarki topik pada protokol MQTT dirancang secara spesifik untuk memisahkan aliran data sensor, perintah aktuator, dan telemetri robot, sehingga mesin otomatisasi dapat memproses sinyal masuk secara akurat. Spesifikasi desain topik dan konfigurasi pengiriman data tersebut dijabarkan pada Tabel 3.9 berikut.

Tabel 3.9 Pola *Payload* Data Perangkat IoT dan Data Definisi Robot

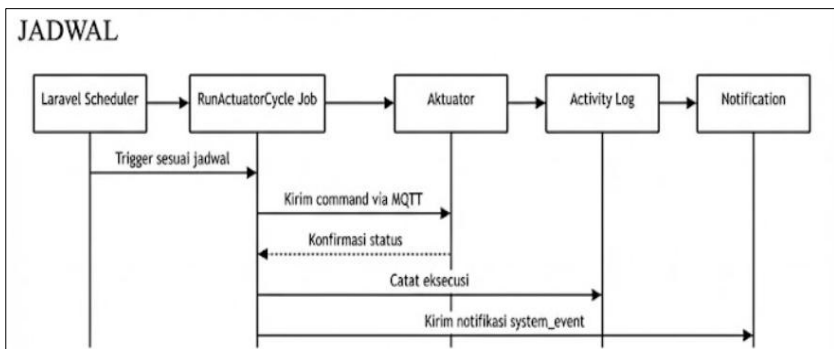
Perangkat	Pola Topik	QoS	Retain
Sensor	<code>iterahero/{location}/sensor/{device}/data</code>	0	False
Aktuator	<code>iterahero/{location}/actuator/{device}/cmd</code>	1	True
Data Definisi Robot	<code>iterahero/{location}/robot/{robot_code}/data/{key}</code>	0	False

Berdasarkan Tabel 3.9, konfigurasi *Quality of Service* (QoS) dan *Retain* disesuaikan dengan krusialitas masing-masing fungsi otomatisasi. *Topic* untuk aktuator menggunakan QoS 1 dan *Retain* bernilai *True* untuk menjamin perintah kontrol dari *Automation Rule* atau *Scheduler* tersampaikan secara pasti dan perangkat segera mengetahui status terakhirnya saat terhubung kembali. Sebaliknya, data sensor dan definisi robot dikonfigurasi dengan QoS

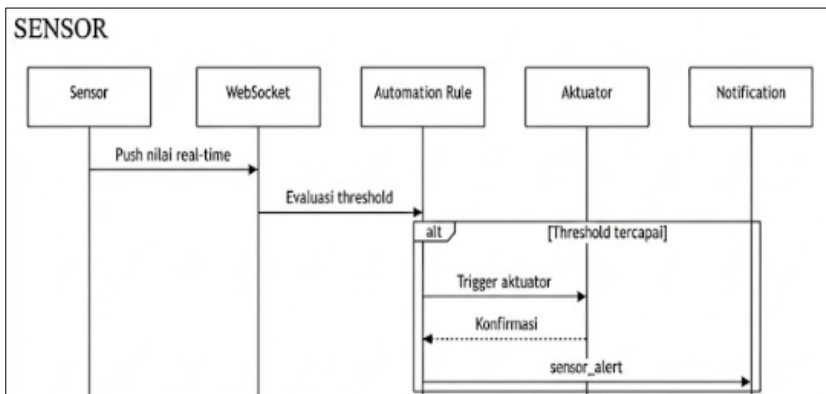
0 dan *Retain* bernilai *False* untuk memprioritaskan latensi rendah dan menghindari pemrosesan data kedaluwarsa yang dapat mendistorsi evaluasi kondisi pada logika otomatisasi *real-time*.

c. *Sequence Diagram*

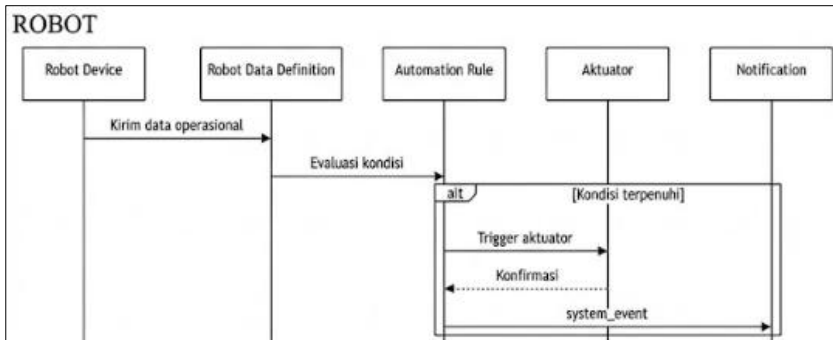
Sequence diagram merupakan salah satu jenis diagram interaksi dalam *Unified Modeling Language* (UML) yang digunakan untuk memodelkan kolaborasi dinamis antara sejumlah objek. Diagram ini berfokus pada visualisasi pertukaran *message* (pesan) antar *lifeline* (garis hidup) objek dalam urutan waktu yang kronologis. Tujuan utama penggunaan *sequence diagram* dalam perancangan ini adalah untuk mengilustrasikan alur logika skenario penggunaan (*use case*) dengan menunjukkan bagaimana berbagai komponen sistem otomatisasi (Penjawalan, Ambang batas sensor, Data definisi robot) perangkat IoT berinteraksi (Lihat Gambar 3.6, 3.7, 3.8).



Gambar 3.6 Diagram Sekuens Otomatisasi Penjadwalan



Gambar 3.7 Diagram Sekuens Otomatisasi Ambang Batas Sensor

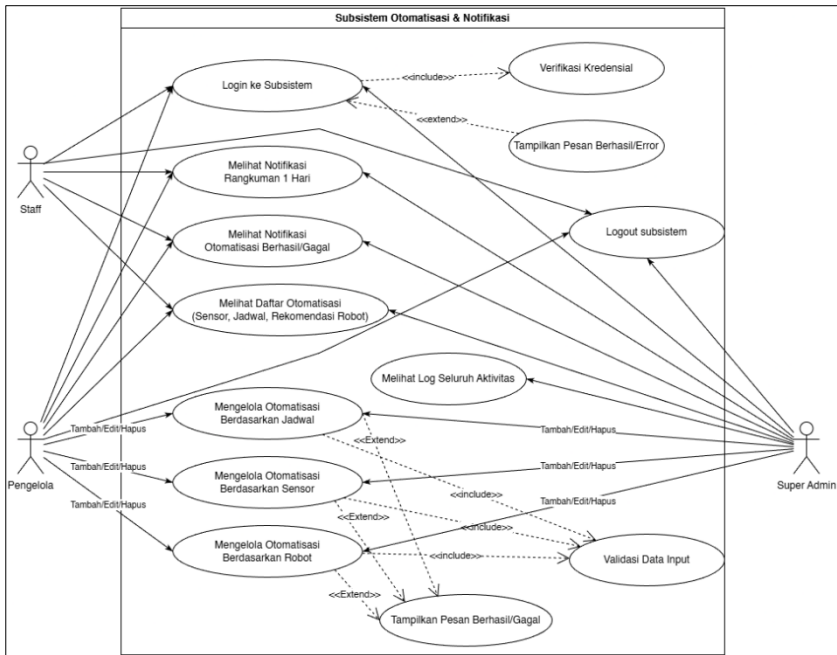


Gambar 3.8 Diagram Sekuens Otomatisasi Data Definisi Robot

Mekanisme otomatisasi yang dirancang terbagi dalam tiga skenario utama yang diilustrasikan pada Gambar 3.6, Gambar 3.7, dan Gambar 3.8 di atas. Skenario pertama (Gambar 3.6) adalah otomatisasi berbasis waktu, di mana *Laravel Scheduler* memicu *job* untuk mengirim perintah MQTT ke *Aktuator*, mencatat log, dan mengirim notifikasi *system_event*. Skenario kedua (Gambar 3.7) merespons data sensor *real-time* via *WebSocket*, di mana *Automation Rule* mengevaluasi ambang batas dan memicu *Aktuator* serta notifikasi *sensor_alert* jika batas tercapai. Skenario terakhir (Gambar 3.8) berbasis kondisi perangkat robotik, di mana data operasional dievaluasi oleh *Automation Rule* untuk memicu *Aktuator* dan mengirim notifikasi *system_event* jika kondisi terpenuhi. Secara keseluruhan, ketiga diagram ini menggambarkan bagaimana sistem secara otomatis merespons berbagai jenis pemicu: waktu, data sensor, dan kondisi perangkat.

d. Desain Diagram Use Case

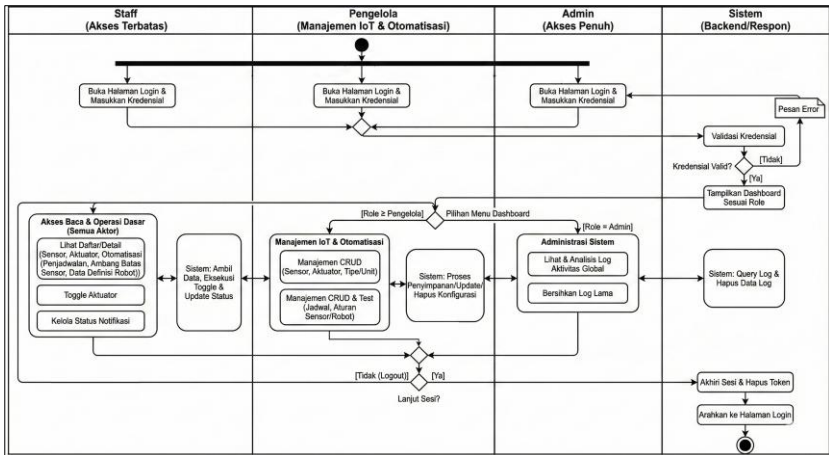
Diagram *Use Case* pada Gambar 3.9 mengilustrasikan interaksi tiga aktor, yakni Staff, Pengelola, dan Admin, yang semuanya memiliki akses dasar untuk *login*, menerima notifikasi operasional, dan *logout*. Aktor Staff memiliki kewenangan terbatas yang hanya mencakup fungsi melihat data pemantauan dan status perangkat di dalam subsistem. Berbeda dengan Pengelola yang memiliki hak tambahan untuk mengelola konfigurasi aturan otomatisasi berdasarkan jadwal, ambang batas *sensor*, dan rekomendasi *robot*. Sementara itu, *Admin* memiliki hak akses penuh yang mencakup seluruh fungsi Pengelola serta kemampuan mengakses riwayat lengkap aktivitas sistem untuk keperluan peninjauan internal.



Gambar 3.9 Use Case Diagram Otomatisasi Perangkat IOT

e. Desain Diagram Aktivitas

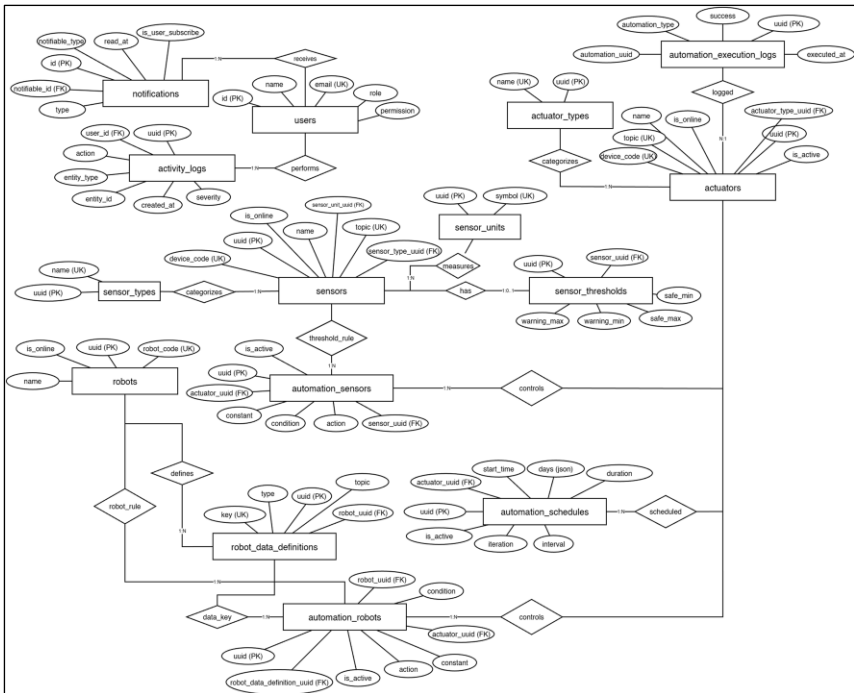
Diagram aktivitas subsistem Otomatisasi Perangkat IOT menggambarkan alur interaksi dinamis antara aktor (Staff, Pengelola, dan Admin) dengan sistem berdasarkan hak akses masing-masing, mencakup proses login, pengelolaan data, hingga logout. Diagram aktivitas ini merepresentasikan alur kerja hierarkis yang diawali dengan validasi kredensial oleh sistem untuk memetakan pengguna ke antarmuka spesifik, di mana Staff terbatas pada akses baca dan kontrol aktuatur dasar, Pengelola memiliki wewenang penuh atas manajemen konfigurasi CRUD (*Create, Read, Update, Delete*) perangkat IoT dan otomatisasi, sedangkan Admin memegang hak akses tertinggi untuk pemeliharaan integritas sistem melalui pengaksesan *log* aktivitas, seperti yang tertera pada Gambar 3.10.



Gambar 3.10 Diagram Aktivitas Subsistem Otomatisasi Perangkat IoT

f. Desain ERD

Desain *Entity Relationship Diagram* (ERD) ini menggambarkan struktur data yang menghubungkan pengguna dengan seluruh ekosistem IoT, mulai dari pemantauan sensor hingga pengendalian aktuator secara otomatis.

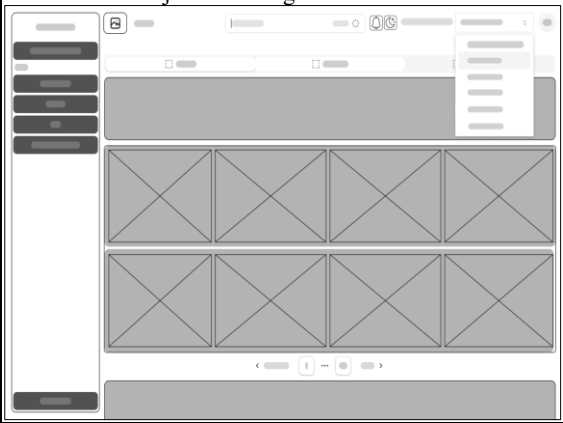


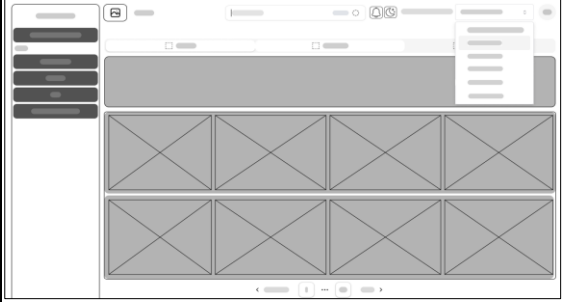
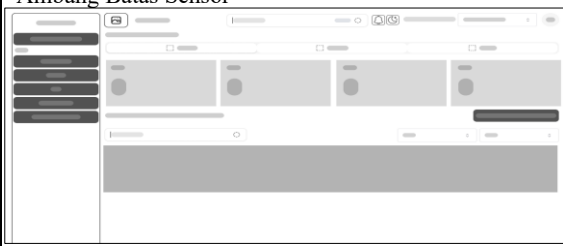
Gambar 3.11 Desain ERD Subsistem Otomatisasi Perangkat IOT

g. **Desain Wireframe**

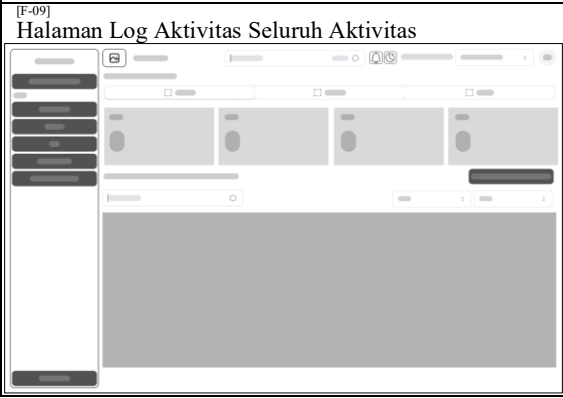
Wireframe berfungsi sebagai rancangan visual awal yang memetakan struktur antarmuka sistem secara mendasar. Tahapan ini berfokus pada penyusunan tata letak elemen, alur navigasi, serta penempatan komponen fungsional sebelum memasuki proses pengembangan antarmuka yang lebih mendetail. Tujuannya adalah memastikan kerangka interaksi telah tersusun secara logis dan sesuai dengan kebutuhan fungsional yang didefinisikan. Secara spesifik, rancangan ini memvisualisasikan tata letak untuk fitur-fitur kunci seperti mekanisme *login*, dasbor manajemen perangkat sensor dan aktuator, antarmuka konfigurasi berbagai jenis otomatisasi, serta tampilan pemantauan melalui notifikasi dan log aktivitas. Rincian rancangan antarmuka untuk setiap subsistem dapat dilihat pada Tabel 3.9 berikut.

Tabel 3.10 Daftar Desain *Wireframe*

[Kode Kebutuhan Fungsional] Nama Desain + Gambar <i>Wireframe</i>	Deskripsi <i>Wireframe</i>
[F-01] Halaman Login 	Halaman ini berfungsi sebagai tempat pengguna masuk ke sistem dengan memasukkan email dan kata sandi. Tampilan dirancang sederhana agar mudah digunakan, serta dilengkapi pesan bantuan jika pengguna salah mengetik atau belum mengisi kolom yang wajib diisi.
[F-02] Halaman Manajemen Perangkat Sensor 	Halaman ini menampilkan daftar seluruh perangkat sensor yang digunakan di lapangan. Pengguna dapat mencari sensor tertentu, menyaring tampilan berdasarkan jenis atau lokasi, dan melihat nilai terkini yang diukur sensor tersebut. Tersedia juga grafik untuk melihat perubahan data sensor berdasarkan tanggal yang dipilih.

[F-03] Halaman Manajemen Perangkat Aktuator 	Halaman ini menampilkan semua perangkat yang berfungsi menggerakkan atau mengalirkan sesuatu, misalnya pompa atau katup. Pengguna dapat melihat status perangkat (menyala atau mati), mengubah mode pengoperasian, serta menyalakan atau mematikan perangkat dengan satu tombol. Setiap tindakan penting akan diminta konfirmasi terlebih dahulu agar lebih aman.
[F-05] Halaman Manajemen Otomatisasi Berdasarkan Ambang Batas Sensor 	Halaman ini menampilkan daftar aturan otomatisasi yang bekerja berdasarkan batas nilai tertentu dari sensor, misalnya ketika suhu terlalu tinggi atau kadar nutrisi terlalu rendah. Pengguna dapat melihat ringkasan jumlah aturan yang aktif, mencari aturan tertentu, dan mengelola seluruh data yang ada.
[F-05] Tambah dan Ubah Data Otomatisasi Berdasarkan Ambang Batas Sensor 	Bagian ini berupa jendela <i>pop-up</i> yang muncul ketika pengguna ingin membuat aturan baru atau mengubah aturan yang sudah ada. Formulir disusun agar mudah dipahami sehingga pengguna dapat mengatur otomatisasi dengan cepat.
[F-04] Halaman Manajemen Otomatisasi Berdasarkan Penjadwalan 	Halaman ini menampilkan daftar otomatisasi yang dijalankan berdasarkan waktu tertentu, misalnya pompa menyala setiap pagi atau setiap dua jam sekali. Pengguna dapat melihat rangkuman statistik, melakukan pencarian, serta menambah, mengubah, atau menghapus jadwal otomatisasi.

[F-04] Tambah dan Ubah Data Otomatisasi Berdasarkan Ambang Batas Sensor 	Jendela <i>pop-up</i> ini digunakan untuk membuat jadwal otomatisasi baru atau memperbarui jadwal yang sudah ada. Seluruh pengaturannya dibuat sederhana agar pengguna mudah menentukan waktu dan aksi yang akan dijalankan.
[F-06] Halaman Manajemen Otomatisasi Berdasarkan Rekomendasi Robot 	Halaman ini menampilkan seluruh aturan otomatisasi yang mengikuti rekomendasi dari sistem robot atau kecerdasan buatan. Pengguna dapat mencari data tertentu, melihat ringkasan kondisi, dan mengelola keseluruhan aturan secara penuh.
[F-06] Tambah dan Ubah Data Otomatisasi Berdasarkan Data Definisi Robot 	Bagian <i>pop-up</i> ini berfungsi untuk membuat atau memperbarui aturan otomatisasi yang mengikuti rekomendasi robot. Pengguna hanya perlu memilih reservoir dan rekomendasi yang ingin diterapkan.
[F-07] & [F-08] Tooltip dan Halaman Notifikasi 	Fitur ini menampilkan informasi singkat mengenai kejadian penting di sistem, misalnya hasil proses otomatisasi atau laporan harian. Notifikasi muncul secara langsung sehingga pengguna dapat memantau kondisi tanpa harus membuka halaman tertentu.

	
[F-09] Halaman Log Aktivitas Seluruh Aktivitas 	Halaman ini berisi daftar lengkap seluruh tindakan atau kejadian yang pernah terjadi di dalam sistem. Catatan ini membantu Admin mengetahui riwayat peristiwa, memeriksa kesalahan, serta melihat siapa yang melakukan suatu tindakan.

3. Implementasi

Tahap implementasi merupakan proses penerjemahan seluruh rancangan desain menjadi kode program yang dapat dijalankan. Pada fase ini, antarmuka pengguna dikembangkan mengacu pada *wireframe*, *sequence*, dan *activity diagram* yang telah dibuat, sementara logika sistem dan struktur basis data dibangun berdasarkan *Use Case Diagram* dan ERD untuk menangani proses otomatisasi serta manajemen data. Fokus utama tahap ini adalah menghasilkan subsistem yang berfungsi penuh sesuai spesifikasi kebutuhan, di mana hasil tampilan antarmuka dan potongan kode program secara rinci akan dipaparkan pada BAB IV, subbab 4.1.

4. Pengujian

a. Fungsional *Blackbox Testing*

Blackbox Testing dilakukan untuk memverifikasi fungsionalitas subsistem tanpa meninjau struktur internal kode program. Fokus utama pengujian ini adalah validasi fungsional berdasarkan fitur-fitur yang telah dikembangkan. Setiap fitur diuji melalui serangkaian skenario yang ditetapkan untuk memastikan subsistem otomatisasi menghasilkan keluaran (*output*) yang

sesuai dengan ekspektasi. Rincian rencana dan skenario pengujian terdapat pada Lampiran F, untuk rangkumannya disajikan pada Tabel 3.11.

Tabel 3.11 Rencana Pengujian *Blackbox Testing*

No	Kode	Kategori Fitur	Jumlah Skenario	Skenario Berhasil	Skenario Gagal	Persentase Keberhasilan
1	BT-01	Login dan Logout	10	Jumlah Berhasil/Gagal		0-100%
2	BT-02	Perangkat Sensor	18			
3	BT-03	Perangkat Aktuator	15			
4	BT-04	Halaman Otomatisasi	8			
5	BT-05	Otomatisasi Jadwal	10			
6	BT-06	Otomatisasi Sensor	9			
7	BT-07	Otomatisasi Robot	10			
8	BT-08	Notifikasi	8			
9	BT-09	Log Aktivitas	11			
10	BT-10	Hak Akses	52			
Total	10 Kategori		151			

Adapun hasil akhir pengujian akan dikalkulasi menggunakan persamaan yang merujuk pada Rumus 2.1, di mana tingkat keberhasilan diperoleh dari perbandingan rasio antara jumlah skenario yang berhasil terhadap total skenario yang diujikan. Berikut contoh perhitungannya jika ada 150 dari 151 pengujian dan persentase keberhasilan mendapatkan hasil 99,33%.

$$\text{Persentase Keberhasilan} = \frac{150}{151} \times 100\% = 99,33\%$$

b. Non-Fungsional *k6*

Rancangan pengujian untuk memenuhi kebutuhan *non-functional* sistem disusun dengan menggunakan *k6* sebagai instrumen utama pengukuran performa dan stabilitas, sebagaimana dirincikan pada Tabel 3.11. Pada tahap awal, aspek *portability* divalidasi menggunakan skenario *smoke test* dengan beban rendah untuk menjamin konsistensi respons API di lintas *platform*, sedangkan aspek *availability* diuji melalui mekanisme *soak test* berdurasi panjang dengan 20 *virtual users* guna mengidentifikasi anomali seperti *memory leak* dan memastikan tingkat kegagalan permintaan tetap minim.

Tabel 3.12 Rencana Pengujian Non-Fungsional

Kebutuhan Non-Fungsional	Jenis Pengujian	Skenario Pengujian	Target Parameter
<i>Functional</i>	<i>Blackbox Testing</i>	% Pengujian Keberhasilan	100%
<i>Portability</i>	<i>Smoke Test</i> (verifikasi cepat kesiapan fungsi dasar)	Simulasi beban ringan dengan 5 <i>virtual users</i> (VU) dalam durasi pendek pada berbagai lingkungan browser.	Tidak ada permintaan gagal; waktu respons < 1.5 detik.

Kebutuhan Non-Fungsional	Jenis Pengujian	Skenario Pengujian	Target Parameter
<i>Availability</i>	<i>Soak Test</i> (uji stabilitas sistem dalam durasi panjang)	Simulasi permintaan kontinu selama beberapa jam dengan 20 VU untuk uji stabilitas jangka panjang.	<i>HTTP request failed</i> < 1%; <i>p95 latency</i> ≤ 2 detik; tidak ada indikasi <i>memory leak</i> .
<i>Security</i>	<i>Unauthorized Access Test & Rate-limit Test</i> (validasi restriksi akses dan pembatasan laju permintaan)	Simulasi akses tanpa token, token tidak valid, akses ke endpoint terbatas, dan uji beban cepat pada <i>endpoint</i> sensitif.	Sistem konsisten memberikan respons 401/403; pembatasan akses berfungsi saat lonjakan permintaan.
<i>Performance</i>	<i>Load Test & Stress Test</i> (simulasi beban operasional normal dan pengujian titik jenuh sistem)	<i>Load test</i> dengan 50 VU (beban normal) dan <i>stress test</i> dengan kenaikan beban bertahap hingga titik jenuh.	Penampilan data < 1 detik; stabilitas pada metrik response time, <i>throughput</i> , dan <i>error rate</i> .

Mengacu pada rincian dalam Tabel 3.11, evaluasi pada aspek *security* diterapkan melalui *unauthorized access test* dan *rate-limit test* untuk memverifikasi bahwa sistem merespons permintaan tidak sah dengan status 401/403 secara konsisten, sehingga mekanisme otorisasi terbukti berfungsi saat terjadi percobaan akses ilegal. Terakhir, validasi *performance* dilaksanakan melalui kombinasi *load test* dan *stress test* yang bertujuan mengukur batas toleransi sistem di bawah tekanan beban, yang mana hal ini dilakukan untuk memastikan data dapat ditampilkan sesuai standar waktu respons yang ditetapkan meskipun sistem beroperasi pada kondisi beban puncak.

c. Pengujian Komunikasi Data dan Otomatisasi

Pengujian ini bertujuan untuk memverifikasi keandalan komunikasi data dua arah antara subsistem dan perangkat *IoT* menggunakan *MQTTX* sebagai alat pemantauan, serta memvalidasi kinerja logika otomatisasi yang diterapkan di *Greenhouse* Kebun Raya ITERA. Proses validasi mencakup pemeriksaan akurasi transmisi pesan pada perangkat sensor aktuator, dan data definisi robot (Tabel 3.13), serta memastikan bahwa seluruh skenario otomatisasi, baik yang dipicu oleh ambang batas sensor, penjadwalan waktu, maupun data definisi robot, berjalan sesuai dengan spesifikasi yang telah ditetapkan (Tabel 3.14). Seluruh parameter pengujian tersebut mengacu pada kebutuhan operasional otomatisasi pada Tabel 3.6.

Tabel 3.13 Skenario Pengujian Komunikasi Data

No	Kode Pengujian	Nama Pengujian + Topik	Payload / Rentang Nilai	Hasil yang Diharapkan
I. Pengujian Manual Perangkat Sensor (<i>QoS 0, Retain= False</i>)				
1	MQTT-01	[S-01]A Suhu Topik: iterahero/kebun-raya-itera/sensor/sensor-ijao/data	Suhu: 20-35 (°C)	100% Sesuai
2	MQTT-02	[S-01]A Kelembapan Topik: iterahero/kebun-raya-itera/sensor/sensor-gxgp/data	Kelembapan: 0-100 (%)	100% Sesuai

No	Kode Pengujian	Nama Pengujian + Topik	Payload / Rentang Nilai	Hasil yang Diharapkan
3	MQTT-03	[S-02] Total Dissolved Solids (TDS) Topik: iterahero/rumah-pemupukan/sensor/sensor-jdtd/data	0-2000 (PPM)	100% Sesuai
4	MQTT-04	[S-03] Level Ketinggian Tandon Air Topik: iterahero/rumah-pemupukan/sensor/sensor-wddu/data	0-100 (%)	100% Sesuai
5	MQTT-05	[S-04] Level Ketinggian Nutrisi Pupuk Topik: iterahero/rumah-pemupukan/sensor/sensor-wddu/data	0-100 (%)	100% Sesuai
II. Pengujian Manual Perangkat Aktuator (<i>QoS 1, Retain= True</i>)				
6	MQTT-06	[A-01] <i>Cooling Pad</i> Topik: iterahero/kebun-roya-itera/actuator/actuator-cneq/cmd	ON / OFF	100% Sesuai
7	MQTT-07	[A-02] <i>Exhaust Fan</i> Topik: iterahero/kebun-roya-itera/actuator/actuator-ct6x/cmd	ON / OFF	100% Sesuai
8	MQTT-08	[A-03] <i>Agitator Pencampuran Utama</i> Topik: iterahero/rumah-pemupukan/actuator/actuator-kp80/cmd	ON / OFF	100% Sesuai
9	MQTT-09	[A-04] <i>Agitator Bahan Nutrisi</i> Topik: iterahero/rumah-pemupukan/actuator/actuator-nkdu/cmd	ON / OFF	100% Sesuai
10	MQTT-10	[A-05] <i>Pompa Listrik Nutrisi</i> Topik: iterahero/rumah-pemupukan/actuator/actuator-pompa-nutrisi/cmd	ON / OFF	100% Sesuai
11	MQTT-11	[A-06] <i>Katup Air Tandon Pencampuran</i> Topik: iterahero/rumah-pemupukan/actuator/actuator-ze4g/cmd	ON / OFF	100% Sesuai
12	MQTT-12	[A-07] <i>Pompa Penyiraman</i> Topik: iterahero/rumah-pemupukan/actuator/actuator-acj8/cmd	ON / OFF	100% Sesuai
III. Pengujian Manual Data Robot (<i>QoS 1, Retain= False</i>)				
13	MQTT-13	[DR-01] Data Definisi Robot (Ml/tanaman) Topik: iterahero/greenhouse/robot/maid-test-1/data/mltanaman	Volume (120-380)	100% Sesuai

Catatan: Angka berdasarkan data wawancara dan observasi lapangan

Tabel 3.14 Pengujian Skenario Otomatisasi Perangkat IoT

No	Kode	Trigger & Skenario Masukan	Hasil yang Diharapkan
I. Otomatisasi Ambang Batas Sensor			
1	[OS-01]	Pengujian Suhu Tinggi 1. Kirim payload data ke [S-01]A bernilai 35°C(Kondisi $\geq 32^{\circ}\text{C}$) 2. Kirim payload data ke [S-01]A bernilai 25°C (Kondisi $\leq 27^{\circ}\text{C}$)	1. [A-01] Exhaust Fan berubah menjadi ON. 2. [A-01] Exhaust Fan berubah menjadi OFF.
2	[OS-02]	Pengujian Kelembapan Rendah 1. Kirim payload data ke [S-01]B bernilai 60% (Kondisi $\leq 65\%$)	1. [A-02] Cooling Pad berubah menjadi ON. 2. [A-02] Cooling Pad berubah menjadi OFF.

No	Kode	Trigger & Skenario Masukan	Hasil yang Diharapkan
		2. Kirim payload data ke [S-01]B bernilai 80% (Kondisi $\geq 75\%$)	
3	[OS-03]	Pengujian Pengisian Air Tandon 1. Kirim payload data ke [S-03] bernilai 5% (Kondisi $\leq 10\%$) 2. Kirim payload data ke [S-03] bernilai 90% (Kondisi $\geq 85\%$)	1. [A-06] Katup Air berubah menjadi ON (Mengisi). 2. [A-06] Katup Air berubah menjadi OFF (Penuh).
4	[OS-04]	Pengujian Konsentrasi Nutrisi (TDS) 1. Kirim payload data ke [S-02] bernilai 100 PPM (Kondisi ≤ 120) 2. Kirim payload data ke [S-02] bernilai 1200 PPM (Kondisi ≥ 1100)	1. [A-05] Pompa Nutrisi berubah menjadi ON. 2. [A-05] Pompa Nutrisi berubah menjadi OFF.
II. Otomatisasi Penjadwalan			
5	[OP-01]	Jadwal <i>Exhaust Fan</i> Waktu sistem menunjukkan pukul 07:00	[A-01] Exhaust Fan menyala otomatis selama 10 jam, kemudian mati.
6	[OP-02]	Jadwal <i>Cooling Pad</i> Waktu sistem menunjukkan pukul 07:00	[A-02] Cooling Pad menyala otomatis selama 10 jam, kemudian mati.
7	[OP-03]	Jadwal Agitator Pencampuran Aktif setiap hari pukul 06:55 [Durasi: 300 detik] [5 pengulangan, interval 2 jam]	[A-03] Agitator Mix menyala otomatis selama 300 detik (5 menit), kemudian mati. Sistem menjadwalkan 4 kali pengulangan berikutnya setiap 2 jam (09:00, 11:00, dst).
8	[OP-04]	Jadwal Agitator Bahan Nutrisi Aktif setiap hari pukul 06:55 [Durasi: 300 detik] [5 pengulangan, interval 2 jam]	[A-04] Agitator Bahan menyala otomatis selama 300 detik (5 menit), kemudian mati. Sistem menjadwalkan 4 kali pengulangan berikutnya setiap 2 jam (09:00, 11:00, dst).
9	[OP-05]	Jadwal Siram Interval Aktif setiap hari pukul 06:55 [Durasi: 300 detik] [5 pengulangan, interval 2 jam]	[A-07] Pompa Siram menyala. Sistem menjadwalkan 4 kali pengulangan berikutnya setiap 2 jam (09:00, 11:00, dst).
III. Otomatisasi Data Definisi Robot			
10	[OR-01]	Penyiraman Presisi Robot Kirim payload: 380 ke topik ml/tanaman robot.	[A-07] Pompa Siram menyala (ON) selama 300 detik (5 menit), lalu mati (OFF) secara otomatis, 4 kali pengulangan berikutnya setiap 2 jam (09:00, 11:00, dst)

5. Deploy

Tahapan *deployment* merupakan fase pemindahan kode program dari lingkungan pengembangan lokal menuju lingkungan server produksi untuk memastikan ketersediaan sistem dalam skala yang lebih luas. Proses ini melibatkan konfigurasi teknis pada infrastruktur server dan jaringan agar aplikasi yang dikembangkan dapat diakses secara daring dan stabil melalui protokol internet. Ketersediaan akses publik ini dirancang secara khusus untuk memfasilitasi keterlibatan pengguna atau klien dalam melakukan interaksi langsung dengan antarmuka sistem yang telah dibangun. Selanjutnya, aktivitas pengujian yang dilakukan pada lingkungan *live* ini bertujuan untuk memverifikasi validitas fungsionalitas perangkat lunak serta memastikan kesesuaiannya dengan spesifikasi kebutuhan yang telah ditetapkan pada tahap perancangan sebelumnya.

6. Review

Tahap tinjauan (*review*) oleh *penanggung jawab greenhouse* Kebun Raya ITERA dilaksanakan untuk memvalidasi kesesuaian hasil pengujian *Blackbox Testing dan Responsive Testing* terhadap spesifikasi kebutuhan yang telah disepakati. Proses ini bertujuan untuk memastikan bahwa seluruh skenario uji dan luaran fungsional sistem benar-benar selaras dengan ekspektasi operasional di *Greenhouse*. Persetujuan formal dari *Penanggung jawab greenhouse* Kebun Raya ITERA pada tahap ini menjadi prasyarat mutlak sebelum sistem dinyatakan layak untuk dilanjutkan ke fase pengujian *usability* dengan pengguna akhir.

3.3.3 Pengujian Usability

Pengujian *usability* dilakukan untuk mengevaluasi tingkat keberterimaan dan kualitas interaksi pengguna terhadap subsistem otomatisasi yang telah dikembangkan. Tahapan ini bertujuan memvalidasi bahwa seluruh otomatisasi perangkat IoT bisa diterima bagi seluruh level pengguna di lingkungan operasional *Greenhouse* Kebun Raya ITERA, dengan menerapkan standar dan instrumen *System Usability Scale (SUS)*.

Jumlah responden yang dilibatkan dalam pengujian *usability* ini adalah 43 pengguna. Angka ini didasarkan pada *purposive sampling* yakni keseluruhan *stakeholder* terkait yang memiliki kewenangan dan keterlibatan langsung dalam penelitian subsistem *greenhouse* Kebun Raya ITERA dan memiliki akses ke website ini, sebagaimana telah didefinisikan dalam Lampiran F. Komposisi responden terdiri dari 30 Staff, 12 Pengelola, dan 1 Admin, yang mencerminkan pembagian peran dari para *stakeholder* tersebut, sebagaimana ditunjukkan pada Tabel 3.15.

Tabel 3.15 Gambaran Responden

Pengguna	Jumlah
Staff	30
Pengelola	12
Admin	1
Total Responden	43

Pengukuran kepuasan subjektif pengguna secara menyeluruh dilakukan menggunakan kuesioner *System Usability Scale (SUS)* yang diadopsi dari John Brooke. Responden akan diminta memberikan penilaian terhadap 10 butir pernyataan standar (lihat Tabel 2.7) dengan skala likert 1 sampai 5 (Lihat Tabel 3.18). Data mentah dari kuesioner tersebut selanjutnya akan dikalkulasi menggunakan aturan perhitungan skor SUS, di mana skor pernyataan ganjil dikurangi 1, dan 5 dikurangi skor pernyataan genap, kemudian totalnya dikalikan 2.5 untuk mendapatkan nilai akhir (0-100). Simulasi perhitungan untuk satu responden dapat dilihat pada Tabel 3.19.

Tabel 3.16 Skala Likert 1-5 [53]

No.	Skor	Keterangan
1.	5	Sangat Setuju (SS)
2.	4	Setuju (S)
3.	3	Netral (N)
4.	2	Tidak Setuju (TS)
5.	1	Sangat Tidak Setuju (STS)

Tabel 3.17 Simulasi Perhitungan 1 Responden SUS

No.	Tipe Pertanyaan	Skor Responden (1-5)	Rumus Perhitungan	Skor Akhir Poin
1.	Ganjil	4	$4-1$	3
2.	Genap	2	$5-2$	3
3.	Ganjil	5	$5-1$	4
4.	Genap	1	$5-1$	4
5.	Ganjil	5	$5-1$	4
6.	Genap	2	$5-2$	3
7.	Ganjil	4	$4-1$	3
8.	Genap	2	$5-2$	3
9.	Ganjil	4	$4-1$	3
10.	Genap	2	$5-2$	3
Total Skor				33
Nilai Akhir SUS			33×2.5	82.5

Berdasarkan hasil simulasi perhitungan pada Tabel 3.13, diperoleh skor akhir *System Usability Scale* (SUS) sebesar 82,5. Untuk menentukan kelayakan sistem, nilai tersebut diinterpretasikan menggunakan pedoman visual penilaian SUS (Brooke, 2013) sebagaimana ditunjukkan pada Gambar 2.5.

Hasil pemetaan skor 82,5 menunjukkan bahwa sistem berada pada kategori Grade A, yang mengindikasikan performa kegunaan sangat tinggi. Secara kualitatif melalui *Adjective Rating*, skor ini diklasifikasikan sebagai "*Excellent*" dan masuk dalam rentang "*Acceptable*" dari sisi keberterimaan (*acceptability*) pengguna. Lebih jauh, jika ditinjau dari perspektif *Net Promoter Score* (NPS), nilai ini menempatkan pengguna pada kategori *Promoter*, yang menyiratkan bahwa pengguna merasa puas dan memiliki kecenderungan kuat untuk merekomendasikan sistem ini kepada pihak lain.

3.3.4 Analisis dan Kesimpulan

Tahap akhir penelitian ini berfokus pada analisis terhadap seluruh data yang diperoleh dari pengujian Fungsional, Non-Fungsional, Komunikasi Data dan Otomatisasi, serta *Usability*. Hasil analisis tersebut disintesis untuk mengevaluasi apakah fitur otomatisasi yang dibangun telah menjawab rumusan masalah, serta menarik kesimpulan mengenai kelayakan teknis dan

operasional sistem sebelum diserahkan sepenuhnya kepada pihak penanggung jawab *greenhouse* Kebun Raya ITERA.

BAB IV

HASIL DAN PEMBAHASAN

4.1 Implementasi Hasil

Tahap implementasi merupakan realisasi teknis dari rancangan desain yang telah didefinisikan pada BAB III, mengubah *wireframe* dan skema logika menjadi kode program yang fungsional. Seluruh antarmuka dikembangkan untuk memastikan interaksi pengguna yang fungsional dan , sementara logika *backend* dibangun untuk menangani komunikasi data *real-time* melalui protokol MQTT dan *WebSocket*. Rincian lengkap mengenai komponen fitur yang berhasil diimplementasikan beserta deskripsi fungsi utamanya disajikan sebagai berikut.

4.1.1 Implementasi Arsitektur

Arsitektur *backend* yang dikembangkan pada subsistem otomatisasi perangkat IoT dibangun melalui integrasi strategis beberapa pendekatan teknis yang saling melengkapi. Mekanisme *event-driven* diterapkan secara spesifik untuk menangani respons terhadap aliran data perangkat secara reaktif, sedangkan penggunaan *message queue* bertujuan untuk mengelola antrian perintah aktuator agar dapat diproses secara *asynchronous* tanpa membebani proses utama pada subsistem. Selain itu, metode *scheduled tasks* diimplementasikan guna menjalankan logika otomatisasi berbasis waktu secara periodik dan terstruktur sesuai kebutuhan sistem. Rincian komponen arsitektur yang diterapkan dalam implementasi subsistem ini disajikan secara lengkap pada Tabel 4.1.

Tabel 4.1 Komponen Arsitektur Subsistem Otomatisasi Perangkat IoT

No.	Komponen	Penjelasan
1.	<i>Event-Driven</i>	Data sensor dan robot memicu <i>event</i> yang diproses untuk menjalankan aturan otomatisasi
2.	<i>Message Queue</i>	Perintah aktuator diproses melalui <i>Laravel Queue</i> secara <i>asynchronous</i>
3.	<i>Scheduled Tasks</i>	<i>Cron job</i> memeriksa jadwal otomatisasi setiap menit
4.	<i>Real-time Broadcasting</i>	Pembaruan status disiarkan melalui <i>WebSocket</i> menggunakan <i>Laravel Reverb</i>
5.	<i>Publish-Subscribe</i>	Protokol MQTT digunakan untuk komunikasi dengan perangkat IoT

Bagian berikut menjelaskan implementasi komponen arsitektur yang mendukung tiga jenis otomatisasi: berbasis penjadwalan, berbasis ambang batas sensor, dan berbasis data definisi robot.

A. Penerimaan Data Perangkat melalui MQTT

Sistem menerima data dari perangkat IoT melalui protokol MQTT. Protokol ini memiliki *overhead* rendah sehingga cocok untuk perangkat

dengan sumber daya terbatas. Penerimaan data dijalankan melalui *Artisan command* yang berjalan terus-menerus sebagai *long-running process* pada sisi server. *Command* tersebut berlangganan ke beberapa *topic* sekaligus dan mendelegasikan pemrosesan ke *handler* yang sesuai (Lihat Kode 4.1).

Kode 4.1 Pendaftaran *handler topic* MQTT

```
// app/Console/Commands/MqttSubscribe.php
protected function registerTopicHandlers(): void
{
    $this->topicHandlers = [
        // Topic untuk perintah aktuator
        'iterahero/+/actuator/+/cmd' => fn($topic, $message)
            => $this->handleActuatorCommandTopic($topic, $message),

        // Topic untuk data sensor
        'iterahero/+/sensor/+/data' => fn($topic, $message)
            => $this->handleSensorDataTopic($topic, $message),

        // Topic untuk status robot
        'iterahero/robot/+/status' => fn($topic, $message)
            => $this->handleRobotStatusTopic($topic, $message),

        // Topic untuk data dinamis robot
        'iterahero/+/robot/+/data/+' => fn($topic, $message)
            => $this->handleRobotDynamicDataTopic($topic, $message),
    ];
}
```

B. Pemrosesan Data Sensor

Data sensor yang diterima melalui MQTT, sistem menjalankan beberapa proses secara berurutan. Pertama, sistem mengekstrak kode perangkat dari *topic* dan mencari sensor yang sesuai di basis data. Kedua, nilai mentah dikalibrasi menggunakan rumus yang dikonfigurasi pada sensor. Ketiga, data disimpan ke basis data dan disiarkan ke antarmuka pengguna melalui *WebSocket*. Terakhir, sistem mengevaluasi aturan otomatisasi berbasis ambang batas sensor (Lihat Kode 4.2).

Kode 4.2 Alur pemrosesan data sensor

```
// app/Console/Commands/MqttSubscribe.php
public function handleSensorDataTopic(string $topic, string $message): void
{
    // Ekstrak kode perangkat dari topic
    preg_match('/^iterahero\/([^\/]+)\/sensor\/([^\/]+)\/data$/', $topic,
    $matches);
    [$location, $deviceCode] = [$matches[1], $matches[2]];

    // Cari sensor berdasarkan kode perangkat
    $sensor = Sensor::byDeviceCode($deviceCode)->with('threshold')->first();

    // Kalibrasi nilai mentah
    $rawValue = (float) $message;
    $calibrated = $sensor->calibrate($rawValue);

    // Siarkan ke frontend melalui WebSocket
    event(new SensorValueUpdated($sensor, $rawValue, $calibrated, true));

    // Simpan ke basis data
    SensorLog::create([
        'sensor uuid' => $sensor->uuid,
        'raw value' => $rawValue,
        'value' => $calibrated,
        'recorded_at' => now()->utc(),
    ]);

    // Perbarui status sensor
    $sensor->update([
        'last seen at' => now()->utc(),
        'is online' => true,
        'last_value' => $calibrated,
    ]);

    // Proses aturan otomatisasi berbasis sensor
    $this->processSensorAutomations($sensor, $calibrated);
}
```

C. Evaluasi Kondisi Otomatisasi Berbasis Ambang Batas Sensor

Otomatisasi berbasis sensor bekerja dengan membandingkan nilai sensor terhadap konstanta ambang batas. Sistem mengambil semua aturan otomatisasi yang aktif untuk sensor tersebut dari basis data. Setiap aturan dievaluasi menggunakan *operator* perbandingan yang dikonfigurasi, seperti lebih besar (>), lebih kecil (<), sama dengan (==), dan tidak sama dengan (!=). Jika kondisi terpenuhi dan aktuator tidak sedang dikontrol otomatisasi lain, sistem mengirim perintah ke aktuator (Lihat Kode 4.3).

Kode 4.3 Evaluasi kondisi otomatisasi berbasis sensor

```
// app/Console/Commands/MqttSubscribe.php
protected function registerTopicHandlers(): void
{
    $this->topicHandlers = [
        // Topic untuk perintah aktuator
        'iterahero/+actuator/+cmd' => fn($topic, $message)
            => $this->handleActuatorCommandTopic($topic, $message),
        // Topic untuk data sensor
        'iterahero/+sensor/+data' => fn($topic, $message)
            => $this->handleSensorDataTopic($topic, $message),
        // Topic untuk status robot
        'iterahero/robot/+status' => fn($topic, $message)
            => $this->handleRobotStatusTopic($topic, $message),
        // Topic untuk data dinamis robot
        'iterahero/+robot/+data/+*' => fn($topic, $message)
            => $this->handleRobotDynamicDataTopic($topic, $message),
    ];
}
```

D. Pemrosesan Data Robot

Sistem juga menerima data dari robot melalui MQTT dengan format *topic* yang berbeda dari sensor. Data robot memiliki tipe yang beragam, meliputi angka, teks, *boolean*, dan enumerasi. Ketika data diterima, sistem memvalidasi nilai sesuai tipe data yang didefinisikan, kemudian menyimpan ke basis data. Setelah penyimpanan, sistem mengevaluasi aturan otomatisasi berbasis data robot yang aktif (Lihat Kode 4.4).

Kode 4.4 Alur pemrosesan data robot

```
// app/Console/Commands/MqttSubscribe.php
public function handleRobotDynamicDataTopic(string $topic, string $message): void
{
    // Ekstrak informasi dari topic
    preg_match('/^iterahero\/([^\/]*)\/robot\/([^\/]*)\/data\/([^\/]*)$/',
    $topic, $matches);
    [$location, $robotCode, $dataKey] = [$matches[1], $matches[2], $matches[3]];

    // Cari definisi data robot
    $robotDataDefinition = RobotDataDefinition::where('topic', $topic)
        ->with('robot')
        ->first();
    $robot = $robotDataDefinition->robot;

    // Validasi nilai enum
    $dataDefinition = RobotDataDefinition::where('robot_uuid', $robot->uuid)
        ->where('key', $dataKey)
        ->first();

    if ($dataDefinition && $dataDefinition->type === 'enum') {
        $enumOptions = $dataDefinition->enumOptions;
        if (!in_array($message, $enumOptions)) {
            return;
        }
    }

    // Simpan log data robot
    RobotDataLog::create([
        'robot_uuid' => $robot->uuid,
        'key' => $dataKey,
        'value' => $message,
        'recorded_at' => now(),
    ]);

    // Siarkan pembaruan status robot
    event(new RobotStatusUpdated($robot));

    // Proses aturan otomatisasi berbasis data robot
    $this->processRobotAutomations($robot, $dataKey, $message);
}
```

E. Evaluasi Kondisi Otomatisasi Berbasis Data Robot

Evaluasi kondisi untuk data robot mendukung berbagai tipe data. Untuk tipe angka, sistem menggunakan *operator* perbandingan numerik standar. Untuk tipe teks dan enumerasi, sistem mendukung *operator* tambahan seperti *contains*, *startsWith*, dan *endsWith*. Jika aturan memiliki konfigurasi iterasi, eksekusi didelegasikan ke *job* terpisah. Jika tidak, aktuator dieksekusi langsung dengan opsi penjadwalan *auto-off* sesuai durasi (Lihat Kode 4.5).

Kode 4.5 Evaluasi kondisi otomatisasi berbasis data robot

```
// app/Console/Commands/MqttSubscribe.php
public function processRobotAutomations(Robot $robot, string $key, mixed $value):
void
{
    // Ambil aturan otomatisasi aktif untuk robot dan key data ini
    $automations = AutomationRobot::with(['dataDefinition', 'actuators'])
        ->where('robot uuid', $robot->uuid)
        ->whereHas('dataDefinition', fn($q) => $q->where('key', $key))
        ->where('is_active', true)
        ->get();

    foreach ($automations as $automation) {
        $dataDefinition = $automation->dataDefinition;

        // Evaluasi kondisi berdasarkan tipe data
        $conditionMet = $this->evaluateRobotCondition(
            $value,
            $automation->condition,
            $automation->constant,
            $dataDefinition->type
        );

        if ($conditionMet) {
            $actuators = $automation->actuators;
            $hasIterations = $automation->interval && $automation->iterations
                && $automation->interval > 0 && $automation->iterations > 0;

            if ($hasIterations && $automation->action && $automation->duration >
0) {
                RunRobotIterationsJob::dispatch($automation->uuid);
            } else {
                $this->dispatchActuatorAction($actuators->uuid, $automation-
>action, $robot->name);
                if ($automation->action && $automation->duration > 0) {
                    TurnActuatorOff::dispatch($actuators->uuid, false, $actuators-
>name)
                        ->delay(now()->addSeconds($automation->duration));
                }
            }
            $automation->update([
                'last triggered at' => now(),
                'trigger_count' => $automation->trigger_count + 1,
            ]);
        }
    }
}

public function evaluateRobotCondition(mixed $value, string $operator, mixed
$constant, string $type): bool
{
    return match ($type) {
        'number', 'integer', 'float' => match ($operator) {
            '>' => (float) $value > (float) $constant,
            '<' => (float) $value < (float) $constant,
            '==' => (float) $value == (float) $constant,
            '!=' => (float) $value != (float) $constant,
            default => false,
        },
        'string', 'enum' => match ($operator) {
            '==' => (string) $value === (string) $constant,
            '!=' => (string) $value !== (string) $constant,
            'contains' => str_contains(strtolower($value),
strtolower($constant)),
            'startsWith' => str_starts_with(strtolower($value),
strtolower($constant)),
            'endsWith' => str_ends_with(strtolower($value),
strtolower($constant)),
            default => false,
        },
    };
}
```

F. Penjadwalan Otomatisasi Berbasis Waktu

Otomatisasi berbasis waktu menggunakan *task scheduling* yang berjalan setiap menit melalui *cron*. Sistem memeriksa jadwal yang aktif dan mencocokkan dengan hari serta waktu saat ini. Jadwal yang sudah dijalankan

pada hari yang sama tidak akan dieksekusi ulang. Untuk jadwal dengan konfigurasi iterasi, eksekusi didelegasikan ke *job* terpisah (Lihat Kode 4.6).

Kode 4.6 Konfigurasi *task scheduling*

```
// routes/console.php
use Illuminate\Support\Facades\Schedule;

// Periksa jadwal otomatisasi setiap menit
Schedule::command('automation:run-schedules')->everyMinute();

// Periksa status perangkat setiap menit
Schedule::command('devices:check-status')->everyMinute();
```

Kode 4.7 menunjukkan logika pemeriksaan dan eksekusi jadwal otomatisasi. *Command* ini dijalankan setiap menit oleh *scheduler* dan memeriksa jadwal yang sesuai dengan hari dan waktu saat ini.

Kode 4.7 Pemeriksaan dan eksekusi jadwal otomatisasi

```
// app/Console/Commands/RunAutomationSchedules.php
public function handle()
{
    $now = Carbon::now('Asia/Jakarta');
    $day = $now->dayOfWeek;
    $time = $now->format('H:i');

    // Ambil jadwal aktif yang sesuai hari dan belum dijalankan hari ini
    $schedules = AutomationSchedule::with('actuator')
        ->where('is active', true)
        ->whereJsonContains('days', $day)
        ->where('start time', '<=', $time)
        ->where(function ($query) use ($now) {
            $query->whereNull('last_run_at')
                ->orWhereDate('last_run_at', '<', $now->toDateString());
        })
        ->get();

    foreach ($schedules as $schedule) {
        $actuator = $schedule->actuator;

        if (is_null($schedule->interval) || is_null($schedule->iterations)) {
            // Mode eksekusi tunggal
            dispatch(new TurnActuatorOn($actuator->uuid, false, $actuator->name));
            dispatch(new TurnActuatorOff($actuator->uuid, false, $actuator->name))
                ->delay($schedule->duration);
        } else {
            // Mode iterasi
            dispatch(new RunScheduleIterationsJob($schedule->uuid));
        }

        $schedule->update(['last_run_at' => now()]);
    }
}
```

G. Eksekusi Iterasi Penjadwalan

Jadwal dengan konfigurasi iterasi diproses oleh *job* terpisah yang menjadwalkan beberapa siklus nyala-mati. Setiap siklus terdiri dari durasi aktuator menyala ditambah interval jeda sebelum siklus berikutnya (Lihat Kode 4.8). Sistem memeriksa *lock* aktuator sebelum menjadwalkan siklus untuk mencegah konflik. Waktu yang sudah terlewat lebih dari 10 detik akan dilewati.

Kode 4.8 *Job* eksekusi iterasi penjadwalan


```
// app/Jobs/RunScheduleIterationsJob.php
// Menangani penjadwalan iterasi aktuatur dengan validasi lock dan perhitungan
siklus waktu
class RunScheduleIterationsJob implements ShouldQueue
{
    use Dispatchable, InteractsWithQueue, Queueable, SerializesModels;

    public function __construct(public string $scheduleUuid)
    {
        $this->onQueue('schedules');
    }

    public function handle(): void
    {
        $schedule = AutomationSchedule::with('actuator')->findOrFail($this->scheduleUuid);
        $lock = ActuatorLockManager::isLocked($schedule->actuator->uuid);
        if ($lock['is locked'] && $lock['lock_info']['lock_type'] !== ActuatorLockManager::LOCK_TYPE_SCHEDULE) return;

        $startTime = Carbon::createFromFormat('H:i', $schedule->start_time, 'Asia/Jakarta')->setDateFrom(now());
        $cycle = $schedule->duration + $schedule->interval;

        for ($i = 0; $i < $schedule->iterations; $i++) {
            $delay = $startTime->copy()->addSeconds($i * $cycle);
            if ($delay->lt(now()) && $delay->diffInSeconds(now()) > 10) continue;

            RunActuatorCycle::dispatch($schedule->actuator->uuid, $schedule->duration, $delay, $schedule->actuator->name, $schedule->uuid->delay($delay);
        }
    }
}
```

H. Eksekusi Siklus Aktuatur

Setiap siklus aktuatur dieksekusi oleh *job* terpisah yang menyalakan perangkat aktuatur serta bertugas untuk menjadwalkan waktu mati. Sistem mengakuisisi *lock* dengan *TTL (Time to Live)* sesuai durasi ditambah *buffer* 30 detik. Jika *lock* gagal diperoleh, siklus dilewati untuk mencegah konflik. Setelah *lock* berhasil, perintah nyala dikirim dan perintah mati dijadwalkan sesuai durasi (Lihat Kode 4.9).

Kode 4.9 Job eksekusi siklus aktuatur

```
// app/Jobs/RunActuatorCycle.php
class RunActuatorCycle implements ShouldQueue
{
    use Dispatchable, InteractsWithQueue, Queueable, SerializesModels;

    public string $actuatorUuid;
    public int $duration;
    public Carbon $startTime;
    public string $actuatorName;
    public ?string $scheduleUuid;

    public function construct(
        string $actuatorUuid,
        int $duration,
        Carbon $startTime,
        string $actuatorName = '',
        ?string $scheduleUuid = null
    ) {
        $this->actuatorUuid = $actuatorUuid;
        $this->duration = $duration;
        $this->startTime = $startTime;
        $this->actuatorName = $actuatorName;
        $this->scheduleUuid = $scheduleUuid;
        $this->onQueue('schedules');
    }

    public function handle(): void
    {
        $lockTtl = $this->duration + 30;
        $lockResult = ActuatorLockManager::acquireLock(
            $this->actuatorUuid,
            ActuatorLockManager::LOCK_TYPE_CYCLE,
            $this->scheduleUuid ?? 'cycle-' . $this->actuatorUuid,
            $lockTtl
        );
        if (!$lockResult['success']) {
            return;
        }

        dispatch(new TurnActuatorOn($this->actuatorUuid, false, $this->actuatorName));
        dispatch(new TurnActuatorOff($this->actuatorUuid, false, $this->actuatorName))
            ->delay($this->duration);
    }
}
```

I. Eksekusi Perintah Aktuatur melalui Antrian

Perintah untuk menyalakan atau mematikan aktuator diproses melalui sistem antrian (*queue*) (Lihat Kode 4.10).

Kode 4.10 *Job* untuk menyalakan aktuator

```
// app/Jobs/TurnActuatorOn.php
class TurnActuatorOn implements ShouldQueue, ShouldBeUnique
{
    use Dispatchable, InteractsWithQueue, Queueable, SerializesModels;

    public function __construct(
        public string $actuatorUuid,
        public bool $isManual,
        public string $actuatorName
    ) {
        $this->onQueue('actuators');
    }

    public function handle(): void
    {
        $actuator = Actuator::where('uuid', $this->actuatorUuid)->firstOrFail();
        if ($actuator->is_active) {
            return;
        }

        // Kirim perintah melalui MQTT
        $mqttClient = new MqttClient($server, $port, $clientId);
        $mqttClient->connect();
        $mqttClient->publish($actuator->topic, 'ON', 1, true);
        $mqttClient->disconnect();

        $actuator->update(['is active' => true]);
        broadcast(new ActuatorToggled($actuator));

        ActuatorLog::create([
            'actuator uuid' => $actuator->uuid,
            'value' => 'ON',
            'is manual' => $this->isManual,
            'recorded_at' => now()->utc(),
        ]);
    }
}
```

Pendekatan ini memastikan proses utama tidak terhambat oleh kemungkinan latensi jaringan yang terjadi pada saat proses komunikasi *MQTT*.

J. Pencegahan Konflik Antar Otomatisasi

Ketika beberapa aturan otomatisasi menargetkan aktuator yang sama, sistem menggunakan mekanisme *lock* berbasis *cache*. Setiap jenis otomatisasi memiliki tipe *lock* tersendiri: *schedule*, *sensor_threshold*, *robot_automation*, dan *cycle*. Sistem hanya mengizinkan otomatisasi dengan sumber yang sama untuk memperbarui *lock* yang sudah ada. *Lock* memiliki *TTL* yang akan kedaluwarsa secara otomatis jika tidak diperbarui (Lihat Kode 4.11). Validasi kesesuaian sumber ini berfungsi mencegah intervensi perintah dari otomatisasi lain selama durasi penguncian masih berlaku.

Kode 4.11 Manajemen *lock* aktuator

```
// app/Services/Automation/ActuatorLockManager.php
class ActuatorLockManager
{
    public const LOCK_TYPE_SCHEDULE = 'schedule';
    public const LOCK_TYPE_SENSOR = 'sensor threshold';
    public const LOCK_TYPE_ROBOT = 'robot automation';
    public const LOCK_TYPE_CYCLE = 'cycle';

    public static function acquireLock(
        string $actuatorUuid,
        string $lockType,
        string $sourceId,
        ?int $ttl = 300
    ): array {
        $lockKey = "actuator_lock:{$actuatorUuid}";
        $existingLock = Cache::get($lockKey);

        if ($existingLock && $existingLock['source_id'] !== $sourceId) {
            return [
                'success' => false,
                'lock_info' => $existingLock,
            ];
        }

        $lockInfo = [
            'actuator_uuid' => $actuatorUuid,
            'lock_type' => $lockType,
            'source_id' => $sourceId,
            'locked_at' => now()->toIso8601String(),
            'expires_at' => now()->addSeconds($ttl)->toIso8601String(),
        ];

        Cache::put($lockKey, $lockInfo, $ttl);
        return ['success' => true, 'lock_info' => $lockInfo];
    }

    public static function isLocked(string $actuatorUuid): array
    {
        $lockKey = "actuator_lock:{$actuatorUuid}";
        $lockInfo = Cache::get($lockKey);

        return [
            'is_locked' => $lockInfo !== null,
            'lock_info' => $lockInfo,
        ];
    }
}
```

K. Penyiaran Status Perangkat melalui *WebSocket*

Setiap perubahan status perangkat disiarkan ke antarmuka pengguna melalui *WebSocket*. Sistem menggunakan *Laravel Reverb* sebagai *WebSocket server* dan *Laravel Echo* yang bertugas menangani sisi *frontend* (Lihat Kode 4.12). *Event* mengimplementasikan *interface ShouldBroadcastNow* sebagai penyiaran langsung tanpa melalui antrian. *Channel* dibuat berdasarkan UUID perangkat dan juga lokasi perangkat untuk mendukung langganan terpadu. Metode *broadcastWith* secara spesifik menyusun struktur data yang dikirimkan agar memuat informasi esensial, meliputi identitas perangkat, nilai sensor, status koneksi, serta penanda waktu kejadian. Format data yang terstruktur ini memungkinkan sisi *frontend* untuk segera memperbarui elemen visual tanpa memerlukan permintaan data tambahan ke basis data.

Kode 4.12 *Event broadcast* status sensor

```
// app/Events/SensorValueUpdated.php
class SensorValueUpdated implements ShouldBroadcastNow
{
    use SerializesModels;

    public function __construct(
        public Sensor $sensor,
        public float $raw,
        public float $value,
        public bool $is_online
    ) {}

    public function broadcastOn(): array
    {
        $channels = [
            new PrivateChannel('sensor.' . $this->sensor->uuid),
        ];
        if ($this->sensor->sensorable type === 'App\\Models\\Greenhouse') {
            $channels[] = new PrivateChannel('greenhouse.' . $this->sensor->sensorable_id);
        }
        return $channels;
    }

    public function broadcastAs(): string
    {
        return 'sensor.value.updated';
    }

    public function broadcastWith(): array
    {
        return [
            'sensor_uuid' => $this->sensor->uuid,
            'device_code' => $this->sensor->device_code,
            'raw_value' => $this->raw,
            'value' => $this->value,
            'is_online' => $this->is_online,
            'timestamp' => now()->toISOString()
        ];
    }
}
```

L. Penerimaan Pembaruan *Real-time* di *Frontend*

Antarmuka pengguna akan otomatis menerima data pembaruan status melalui *WebSocket* menggunakan *Laravel Echo*. Sistem menggunakan pola *subscription manager* untuk mengelola koneksi. Kelas ini menyimpan referensi *channel* yang sudah dibuat untuk mencegah duplikasi langganan. Ketika data status ON/OFF diterima, *state* diperbarui melalui *store* yang dikelola oleh *Zustand*. Kode 4.13 menunjukkan implementasi penerimaan pembaruan status aktuator di sisi *frontend*.

Kode 4.13 Penerimaan pembaruan status aktuator di *frontend*

```
// resources/js/hooks/use-actuator-subscription.ts
class ActuatorSubscriptionManager {
    private channels = new Map<string, ReturnType<typeof window.Echo.private>>();

    subscribe(actuatorUuid: string): void {
        if (this.channels.has(actuatorUuid)) {
            return;
        }

        const channelName = `actuator.${actuatorUuid}`;
        const channel = window.Echo.private(channelName);

        channel.listen(
            'actuator.toggled',
            (data: { uuid: string; is_active: boolean; last_active?: string }) => {
                useActuatorSubscriptionStore.getState().updateState(actuatorUuid, {
                    uuid: actuatorUuid,
                    is_active: data.is_active,
                    last_active: data.last_active,
                });
            }
        );
        this.channels.set(actuatorUuid, channel);
    }

    unsubscribe(actuatorUuid: string): void {
        if (this.channels.has(actuatorUuid)) {
            const channelName = `actuator.${actuatorUuid}`;
            window.Echo.leave(channelName);
            this.channels.delete(actuatorUuid);
        }
    }
}

export const actuatorSubscriptionManager = new ActuatorSubscriptionManager();
```

M. Otorisasi *Channel WebSocket*

Setiap *channel WebSocket* memerlukan otorisasi untuk memastikan hanya pengguna yang berhak dapat menerima pembaruan. Sistem menggunakan mekanisme *callback* yang memvalidasi identitas pengguna sebelum mengizinkan akses ke *channel*. Konfigurasi otorisasi didefinisikan pada berkas *routes*, lalu kemudian diproses oleh *Laravel Broadcasting*. Untuk *channel* umum seperti sensor dan aktuator, semua pengguna terautentikasi diizinkan berlangganan (Lihat Kode 4.14).

Kode 4.14 Konfigurasi otorisasi *channel WebSocket*

```
// routes/channels.php
use Illuminate\Support\Facades\Broadcast;

Broadcast::channel('sensor.{uuid}', function ($user, $uuid) {
    return $user !== null;
});

Broadcast::channel('actuator.{uuid}', function ($user, $uuid) {
    return $user !== null;
});

Broadcast::channel('greenhouse.{uuid}', function ($user, $uuid) {
    return $user !== null;
});

Broadcast::channel('notifications.{userId}', function ($user, $userId) {
    return (int) $user->id === (int) $userId;
});
```

N. Notifikasi *Push* Otomatisasi

Setiap eksekusi otomatisasi menghasilkan notifikasi *push* yang dikirim ke pengguna yang berwenang. Sistem menggunakan *Web Push Notification* yang memanfaatkan *Service Worker* di sisi browser. Notifikasi otomatisasi pada subsistem dikirim melalui dua *channel*: *WebPush* untuk notifikasi browser dan *BroadcastDatabaseChannel* untuk penyimpanan dan penyiaran *real-time*. Mekanisme *throttling* berbasis *cache* digunakan dengan tujuan untuk mencegah pengiriman notifikasi berulang dalam interval 60 detik (Lihat Kode 4.15).

Kode 4.15 *Service* pengiriman notifikasi otomatisasi

```
// app/Services/Automation/AutomationNotificationService.php
final class AutomationNotificationService
{
    public const TYPE_SCHEDULE_SUCCESS = 'automation schedule success';
    public const TYPE_SENSOR_TRIGGERED = 'automation sensor triggered';
    public const TYPE_ROBOT_TRIGGERED = 'automation robot triggered';

    // Menangani notifikasi sensor trigger dengan mekanisme throttling dan logging
    public static function notifySensorTriggered(string $uuid, string $sensor,
    string $actuator, float $val, string $cond, float $limit, bool $action, ?string
    $unit = ''): void
    {
        $key = "automation notif:sensor:{$uuid}:{$actuator}";
        if (Cache::has($key)) return;

        $body = sprintf("%s = %s (%s %s), %s %s", $sensor, $val, $cond, $unit,
        self::formatCondition($cond), $limit, $unit, $action ? 'menyalakan' : 'mematikan',
        $actuator);

        self::broadcast('Sensor Otomasi Terpicu', $body, '/icons/gauge.svg',
        'info'); self::logExecution('sensor', $uuid, "{$sensor} -> {$actuator}", true,
        ['current value' => $val, 'condition' => $cond, 'threshold' => $limit, 'action' =>
        $action]);

        Cache::put($key, true, 60);
    }

    // Mengirim broadcast notifikasi ke role tertentu menggunakan Facade
    private static function broadcast(string $title, string $body, ?string $icon,
    ?string $type): void
    {
        $users = User::whereHas('roles', fn($q) => $q->whereIn('name', ['admin',
        'operator', 'super-admin']))->get();
        Notification::send($users, new WebPushNotification($title, $body,
        route('dashboard.automation.index'), $icon, null, $type));
    }
}
```

Kode 4.16 menunjukkan implementasi kelas notifikasi yang mendefinisikan format pesan untuk *Web Push* dan penyimpanan basis data.

Kode 4.16 Kelas *WebPushNotification*

```
// app/Notifications/WebPushNotification.php
class WebPushNotification extends Notification
{
    use Queueable;

    public function __construct(
        private string $title,
        private string $body,
        private ?string $actionUrl = null,
        private ?string $icon = null,
        private ?string $image = null,
        private ?string $type = null
    ) {}

    public function via($notifiable): array
    {
        return [WebPushChannel::class, BroadcastDatabaseChannel::class];
    }

    public function toWebPush($notifiable, $notification): WebPushMessage
    {
        $message = (new WebPushMessage)
            ->title($this->title)
            ->body($this->body)
            ->icon($this->icon ?? '/logo.png');

        if ($this->actionUrl) {
            $message->action('Open Link', 'open_url_action')
                ->data(['url' => $this->actionUrl]);
        }

        return $message;
    }

    public function toDatabase($notifiable): array
    {
        return [
            'title' => $this->title,
            'description' => $this->body,
            'action url' => $this->actionUrl,
            'icon' => $this->icon,
            'type' => $this->type ?? 'info'
        ];
    }
}
```

O. Pencatatan Log Aktivitas Otomatisasi

Setiap eksekusi otomatisasi dicatat dalam dua jenis log: log eksekusi otomatisasi dan juga log aktivitas umum. Log eksekusi otomatisasi (*AutomationExecutionLog*) menyimpan informasi spesifik seperti tipe otomatisasi, status keberhasilan, dan metadata eksekusi. Log aktivitas umum (*ActivityLog*) berfungsi menyimpan catatan, termasuk informasi pengguna, alamat IP, dan kategori aksi. Pemisahan ini memungkinkan agregasi data untuk ringkasan harian sekaligus pelacakan aktivitas sistem secara menyeluruh (Lihat Kode 4.17).

Kode 4.17 Pencatatan log eksekusi otomatisasi

```
// app/Services/Automation/AutomationNotificationService.php
private static function logExecution(
    string $type,
    string $uid,
    string $name,
    bool $success,
    array $metadata = []
): void {
    AutomationExecutionLog::create([
        'automation type' => $type,
        'automation uid' => $uid,
        'automation name' => $name,
        'success' => $success,
        'metadata' => $metadata,
        'executed_at' => now(),
    ]);
}
```

Kode 4.18 menunjukkan implementasi pencatatan log aktivitas umum yang digunakan untuk berbagai jenis aktivitas termasuk otomatisasi.

Kode 4.18 *Service* pencatatan log aktivitas

```
// app/Services/ActivityLogger.php
class ActivityLogger
{
    public static function log(array $data): ActivityLog
    {
        $user = Auth::user();
        $request = request();

        return ActivityLog::create([
            'user_id' => $user->id,
            'user_name' => $user->name,
            'user_email' => $user->email,
            'action' => $data['action'],
            'entity_type' => $data['entity_type'],
            'entity_id' => $data['entity_id'] ?? null,
            'entity_name' => $data['entity_name'] ?? null,
            'description' => $data['description'] ?? null,
            'changes' => $data['changes'] ?? null,
            'metadata' => $data['metadata'] ?? null,
            'ip_address' => $request->ip(),
            'user_agent' => $request->userAgent(),
            'severity' => $data['severity'] ?? 'info',
            'category' => $data['category'] ?? null,
        ]);
    }

    public static function logAutomation(string $automationType, string $name,
    array $metadata = []): ActivityLog
    {
        return self::log([
            'action' => 'automation',
            'entity_type' => $automationType,
            'entity_name' => $name,
            'description' => "Automation triggered: {$name}",
            'category' => 'automation',
            'severity' => 'info',
            'metadata' => $metadata,
        ]);
    }

    public static function logSensorAlert(string $sensorName, string $alertType,
    array $data): ActivityLog
    {
        return self::log([
            'action' => 'alert',
            'entity_type' => 'sensor',
            'entity_name' => $sensorName,
            'description' => "Sensor alert: {$sensorName} - {$alertType}",
            'category' => 'alert',
            'severity' => 'critical',
            'metadata' => $data,
        ]);
    }
}
```

4.1.2 Implementasi Backend

Implementasi *backend* dibangun menggunakan kerangka kerja Laravel dengan menerapkan pola desain *Service Layer* untuk memisahkan logika bisnis dari *controller*. Fokus utama pengembangan ini mencakup penanganan fungsionalitas inti seperti manajemen perangkat sensor dan aktuator, serta eksekusi berbagai skenario otomatisasi cerdas. Sistem ini juga mengintegrasikan fitur keamanan menyeluruh yang meliputi otentikasi pengguna, pencatatan *log* aktivitas, dan pembagian hak akses berbasis peran. Rincian lengkap mengenai komponen teknis berupa *class*, *file*, dan *route* yang mendukung seluruh fungsi tersebut disajikan dalam Tabel 4.2.

Tabel 4.2 Implementasi *Backend* Subsistem Otomatisasi Perangkat IoT

No	Fitur	Komponen (Class/File) dan Route	Deskripsi Fungsi
1	Login	<i>AuthenticatedSessionController</i> , <i>LoginRequest</i> , <i>User (Model)</i> , <i>Authenticate (Middleware)</i> , <i>RedirectIfAuthenticated (Middleware)</i> , <i>ActivityLogger (Service)</i>	Menangani proses otentikasi pengguna termasuk validasi kredensial, manajemen sesi, dan pencatatan aktivitas <i>login/logout</i> . <i>Middleware</i> memastikan hanya pengguna terautentikasi yang dapat mengakses subsistem.
		Route	
		GET /auth/login	Menampilkan halaman <i>login</i>

No	Fitur	Komponen (Class/File) dan Route	Deskripsi Fungsi
		<i>POST /auth/login</i>	Proses autentikasi kredensial
		<i>POST /auth/logout</i>	Proses <i>logout</i> dan hapus sesi
2	Perangkat Sensor	<i>SensorController, SensorApiController, SensorService, SensorThresholdService, Sensor (Model), SensorLog (Model), SensorType (Model), SensorUnit (Model), SensorThreshold (Model), SensorResource, SensorLogResource, SensorObserver, SensorLogObserver, SensorValueUpdated (Event), StoreSensorRequest, UpdateSensorRequest, StoreSensorTypeRequest, StoreSensorUnitRequest, UpdateSensorThresholdRequest</i>	Mengelola operasi tambah, ubah, dan hapus sensor, penyimpanan riwayat nilai, konfigurasi tipe dan satuan, serta <i>broadcast</i> nilai terbaru ke <i>frontend</i> melalui <i>WebSocket</i> . Data sensor diperbarui secara <i>realtime</i> saat diterima via MQTT.
		Route	
		<i>GET /dashboard/monitoring</i>	Daftar semua sensor dengan <i>filter</i>
		<i>GET /dashboard/monitoring/create</i>	Form tambah sensor baru
		<i>POST /dashboard/monitoring/create</i>	Simpan sensor baru
		<i>GET /dashboard/monitoring/{uuid}</i>	Detail sensor dan grafik
		<i>GET /dashboard/monitoring/{uuid}/edit</i>	Form edit sensor
		<i>PUT /dashboard/monitoring/{uuid}</i>	Update data sensor
		<i>DELETE /dashboard/monitoring/{uuid}</i>	Hapus sensor
		<i>GET /dashboard/monitoring/{uuid}/logs</i>	Riwayat <i>log</i> sensor
		<i>GET /dashboard/monitoring/{uuid}/logs/download</i>	Unduh <i>log</i> sensor (CSV)
		<i>GET /dashboard/monitoring/list/types</i>	Daftar tipe sensor
		<i>GET /dashboard/monitoring/api/data</i>	API data daftar sensor
		<i>POST /dashboard/monitoring/api/sensor-logs/range</i>	API <i>log</i> sensor dari <i>range</i>
		<i>GET /dashboard/monitoring/api/{uuid}/logs</i>	API data <i>log</i> sensor
		<i>GET /dashboard/monitoring/api/{uuid}/stats</i>	API <i>statistik</i> sensor
		<i>POST /dashboard/monitoring/types</i>	Tambah tipe sensor
		<i>DELETE /dashboard/monitoring/types/{uuid}</i>	Hapus tipe sensor
		<i>POST /dashboard/monitoring/unit</i>	Tambah satuan sensor
		<i>DELETE /dashboard/monitoring/unit/{uuid}</i>	Hapus satuan sensor
3	Perangkat Aktuator	<i>ActuatorController, ActuatorService, TopicGenerator (Service), Actuator (Model), ActuatorLog (Model), ActuatorType (Model), ActuatorResource, ActuatorObserver, ActuatorLogObserver, ActuatorToggled (Event), ActuatorException, StoreActuatorRequest, UpdateActuatorRequest, StoreActuatorTypeRequest, TurnActuatorOn (Job), TurnActuatorOff (Job), TurnActuatorOffDelayed (Job)</i>	Mengelola operasi tambah, ubah, dan hapus aktuator, <i>toggle</i> status ON/OFF secara manual, penyimpanan riwayat perubahan status, dan komunikasi dengan perangkat melalui protokol MQTT. Perubahan status di- <i>broadcast</i> ke <i>frontend</i> via <i>WebSocket</i> .
		Route	
		<i>GET /dashboard/controlling</i>	Daftar semua aktuator
		<i>GET /dashboard/controlling/create</i>	Form tambah aktuator
		<i>POST /dashboard/controlling/create</i>	Simpan aktuator baru
		<i>GET /dashboard/controlling/{uuid}</i>	Detail aktuator
		<i>GET /dashboard/controlling/{uuid}/edit</i>	Form ubah aktuator
		<i>POST /dashboard/controlling/{uuid}/edit</i>	Simpan ubah aktuator
		<i>DELETE /dashboard/controlling/{uuid}</i>	Hapus aktuator
		<i>POST /dashboard/controlling/{uuid}/toggle</i>	<i>Toggle</i> ON/OFF aktuator
		<i>GET /dashboard/controlling/{uuid}/logs</i>	Riwayat <i>log</i> aktuator
		<i>GET /dashboard/controlling/actuator/list/types</i>	Daftar tipe aktuator
		<i>GET /dashboard/controlling/api/data</i>	API data aktuator
		<i>GET /dashboard/controlling/api/{uuid}/logs</i>	API data <i>log</i> aktuator
		<i>GET /dashboard/controlling/api/{uuid}/stats</i>	API <i>statistik</i> aktuator
		<i>POST /dashboard/controlling/types</i>	Tambah tipe aktuator

No	Fitur	Komponen (Class/File) dan Route	Deskripsi Fungsi
4	Otomatisasi Berdasarkan Penjadwalan	<i>DELETE /dashboard/monitoring/types/{uuid}</i>	Hapus tipe aktuator
		<i>AutomationController, AutomationScheduleController, AutomationScheduleService, ActuatorLockManager (Service), AutomationConflictDetector (Service), AutomationNotificationService (Service), AutomationSchedule (Model), AutomationExecutionLog (Model), StoreAutomationScheduleRequest, UpdateAutomationScheduleRequest, RunAutomationSchedules (Command), SendDailyAutomationSummary (Command), RunScheduleIterationsJob (Job), RunActuatorCycle (Job), AutomationUpdated (Event)</i>	Mengelola aturan otomatisasi berbasis waktu dengan konfigurasi durasi, waktu mulai, hari aktif, interval, dan iterasi. <i>Command</i> dijalankan setiap menit oleh <i>cron</i> untuk mengeksekusi jadwal yang sesuai. Sistem <i>lock</i> mencegah konflik antar otomatisasi.
		Route	
		<i>GET /dashboard/automation</i>	Halaman utama otomatisasi
		<i>GET /dashboard/automation/schedules/create</i>	Form tambah jadwal
		<i>POST /dashboard/automation/schedules</i>	Simpan jadwal baru
		<i>GET /dashboard/automation/schedules/{uuid}</i>	Detail jadwal
		<i>GET /dashboard/automation/schedules/{uuid}/edit</i>	Form edit jadwal
		<i>PUT /dashboard/automation/schedules/{uuid}</i>	Ubah jadwal
		<i>DELETE /dashboard/automation/schedules/{uuid}</i>	Hapus jadwal
		<i>POST /dashboard/automation/schedules/{uuid}/toggle</i>	Toggle aktif/nonaktif
		<i>POST /dashboard/automation/schedules/{uuid}/test</i>	Uji coba eksekusi jadwal
5	Otomatisasi Berdasarkan Ambang Batas Sensor	<i>AutomationSensorController, AutomationSensorService, SensorThresholdService (Service), ActuatorLockManager (Service), AutomationSensor (Model), StoreAutomationSensorRequest, UpdateAutomationSensorRequest, MqttSubscribe (Command)</i>	Mengelola aturan otomatisasi berbasis kondisi sensor dengan konfigurasi operator perbandingan (>, <, >=, <=, ==, !=), nilai ambang, dan aksi. Evaluasi dilakukan saat data sensor diterima via MQTT, kemudian aktuator dieksekusi jika kondisi terpenuhi.
		Route	
		<i>GET /dashboard/automation/sensor/create</i>	Form tambah aturan
		<i>POST /dashboard/automation/sensor</i>	Simpan aturan baru
		<i>GET /dashboard/automation/sensor/{uuid}</i>	Detail aturan
		<i>GET /dashboard/automation/sensor/{uuid}/edit</i>	Form edit aturan
		<i>PUT /dashboard/automation/sensor/{uuid}</i>	Simpan perubahan aturan
		<i>DELETE /dashboard/automation/sensor/{uuid}</i>	Hapus aturan
6	Otomatisasi Data Robot	<i>AutomationRobotController, AutomationRobotService, ActuatorLockManager (Service), AutomationRobot (Model), StoreAutomationRobotRequest, UpdateAutomationRobotRequest, MqttSubscribe (Command), RunRobotIterationsJob (Job), RunRobotActuatorCycle (Job), ProcessRobotAutomationCondition (Job), RobotAutomationTriggered (Event)</i>	Mengelola aturan otomatisasi berbasis data robot dengan konfigurasi definisi data, <i>operator</i> kondisi (termasuk <i>contains</i> , <i>startsWith</i> , <i>endsWith</i> untuk tipe <i>string</i>), nilai, aksi, durasi, <i>interval</i> , dan iterasi. Evaluasi dilakukan saat data robot diterima via MQTT.
		Route	
		<i>GET /dashboard/automation/robot/create</i>	Form tambah otomatisasi robot

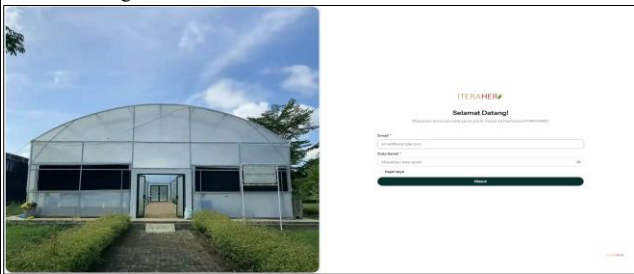
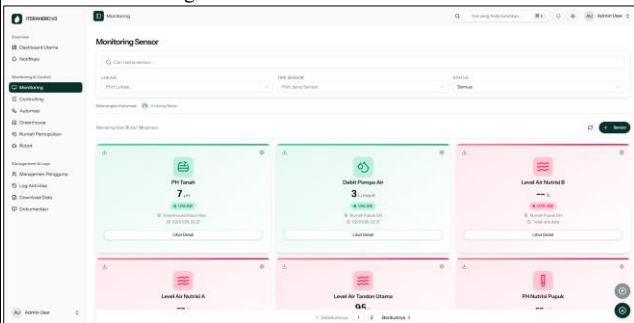
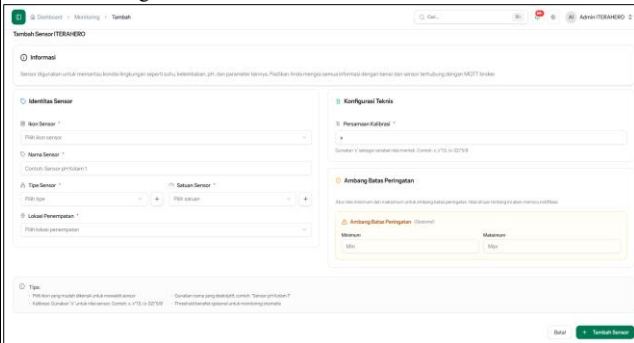
No	Fitur	Komponen (Class/File) dan Route	Deskripsi Fungsi
		<i>POST /dashboard/automation/robot</i>	Simpan otomatisasi robot baru
		<i>GET /dashboard/automation/robot/{uuid}</i>	<i>Detail</i> otomatisasi robot
		<i>GET /dashboard/automation/robot/{uuid}/edit</i>	<i>Form</i> ubah otomatisasi robot
		<i>PUT /dashboard/automation/robot/{uuid}</i>	Simpan perubahan otomatisasi robot
		<i>DELETE /dashboard/automation/robot/{uuid}</i>	Hapus otomatisasi robot
		<i>POST /dashboard/automation/robot/{uuid}/toggle</i>	<i>Toggle</i> aktif/nonaktif
7	Notifikasi	<i>NotificationController, PushSubscriptionController, WebPushNotification (Notification), DatabaseNotification (Model), NotificationType (Enum), NewNotificationEvent (Event), DailySummaryService (Service), AutomationNotificationService (Service), SendDailySummary (Command), AutomationExecutionLog (Model)</i>	Mengelola notifikasi sistem termasuk penyimpanan ke basis data, <i>push notification</i> ke perangkat pengguna, dan rangkuman aktivitas harian. Notifikasi otomatis dikirim saat otomatisasi berhasil/gagal dieksekusi.
		Route	
		<i>GET /dashboard/notifications</i>	Daftar notifikasi
		<i>POST /dashboard/notifications/{id}/read</i>	Tandai dibaca
		<i>DELETE /dashboard/notifications/{id}</i>	Hapus notifikasi
		<i>POST /dashboard/notifications/mark-all-read</i>	Tandai semua dibaca
		<i>POST /dashboard/notifications/destroy-all</i>	Hapus semua
		<i>GET /dashboard/notifications/api/notification</i>	<i>API infinite scroll</i>
		<i>POST /dashboard/notifications/push/subscriptions</i>	Daftar <i>push notification</i>
		<i>DELETE /dashboard/notifications/push/subscriptions</i>	Batalan langganan
		<i>GET /dashboard/notifications/push/vapid-public-key</i>	Ambil <i>VAPID key</i>
8	Log Aktivitas	<i>ActivityLogController, ActivityLog (Model), AutomationExecutionLog (Model), ActivityLogger (Service), DailySummaryService (Service)</i>	Mencatat jejak audit seluruh aktivitas sistem termasuk operasi tambah, ubah, hapus, kontrol aktuator, eksekusi otomatisasi, dan <i>login/logout</i> . Menyediakan filter berdasarkan aksi, entitas, kategori, <i>severity</i> , tanggal, dan pengguna.
		Route	
		<i>GET /dashboard/activity-logs</i>	Daftar log aktivitas lengkap dengan filter
		<i>GET /dashboard/activity-logs/api/insights</i>	
		<i>GET /dashboard/activity-logs/{id}</i>	Detail log aktivitas
		<i>POST /dashboard/activity-logs/cleanup</i>	Hapus log lama
9	Hak Akses	<i>Role (Model), Permission (Model), Permission (Enum), RoleName (Enum), RolePermissionSeeder, PermissionSeeder, PermissionMiddleware (Spatie), RoleMiddleware (Spatie), HasRoles (Trait), HandleInertiaRequests (Middleware)</i>	Implementasi RBAC berbasis <i>Spatie</i> . <i>Permission</i> mengatur akses fitur IoT melalui validasi <i>middleware</i> untuk tiga peran berbeda yaitu Admin, Pengelola, dan Staff

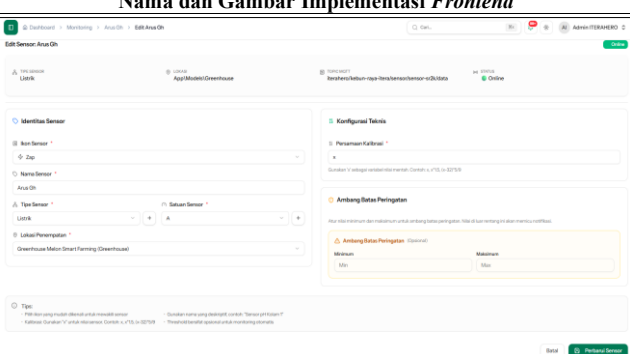
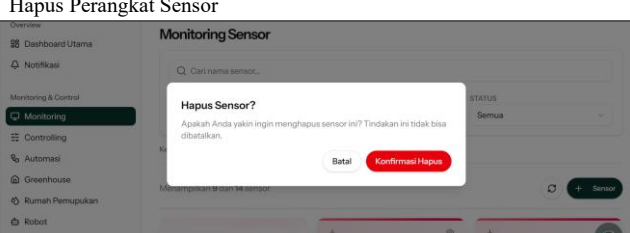
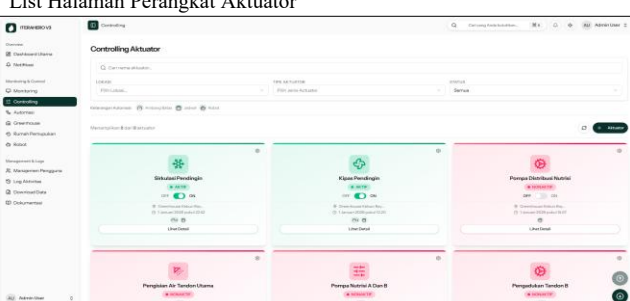
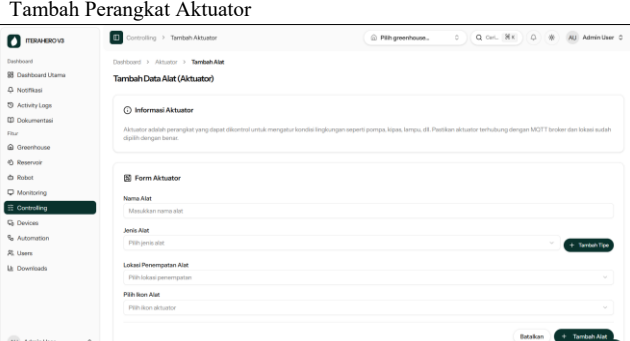
4.1.3 Implementasi *Frontend*

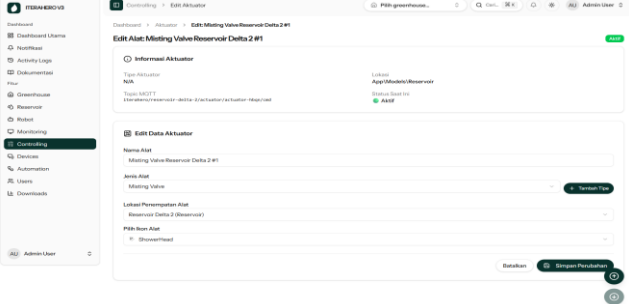
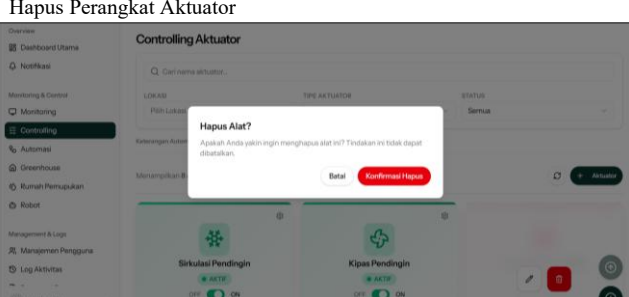
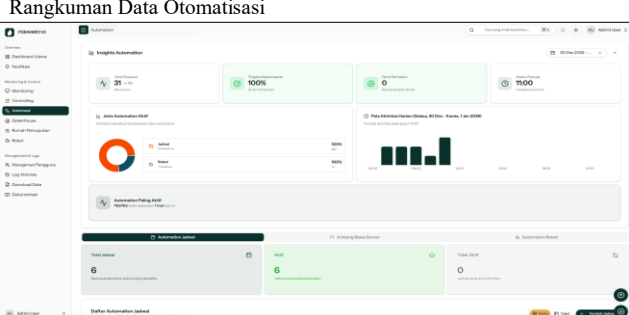
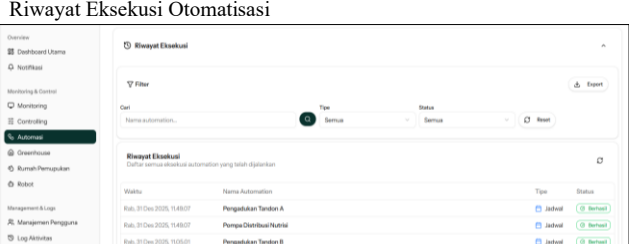
Implementasi *frontend* pada subsistem direalisasikan dengan cara mengadopsi *Inertia.js* untuk membangun antarmuka yang terintegrasi erat dengan logika *backend* tanpa kompleksitas pengelolaan API terpisah. Dalam arsitektur ini, antarmuka disusun sebagai kumpulan komponen yang

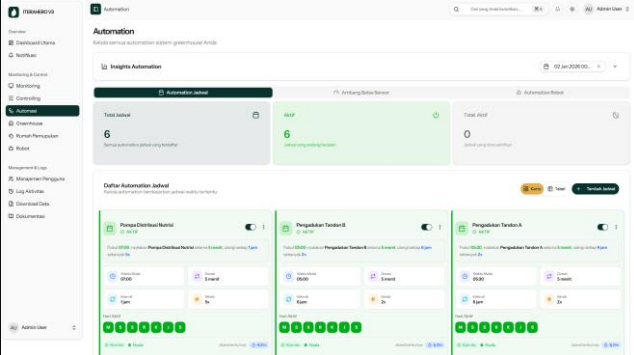
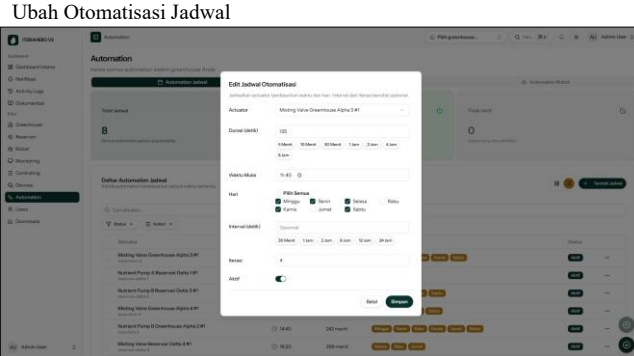
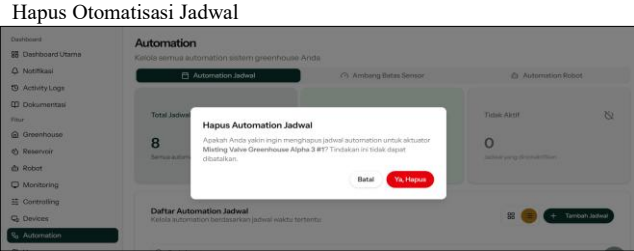
menerima data operasional secara langsung melalui mekanisme injeksi properti (*props*) dari peladen. Implementasi *frontend* mencakup sembilan fitur utama, mulai dari *login* berbasis peran hingga manajemen sensor dan aktuator yang mendukung pembaruan *real-time* via *WebSocket*. Selain empat jenis otomatisasi, tersedia fitur notifikasi dengan *push notification* serta *log* aktivitas untuk *monitoring* dan audit, sebagaimana dirinci dalam Tabel 4.3.

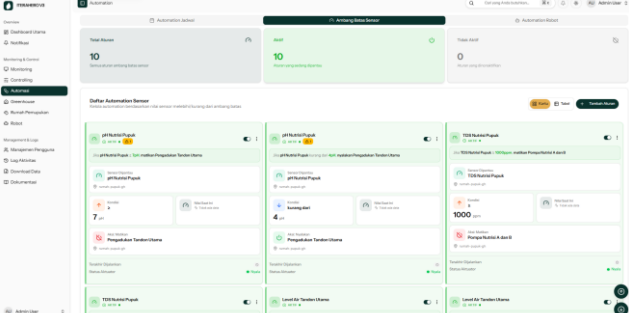
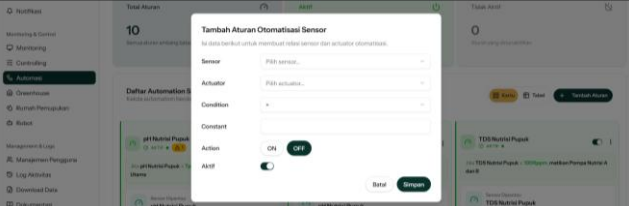
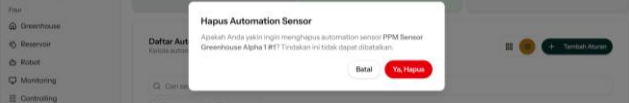
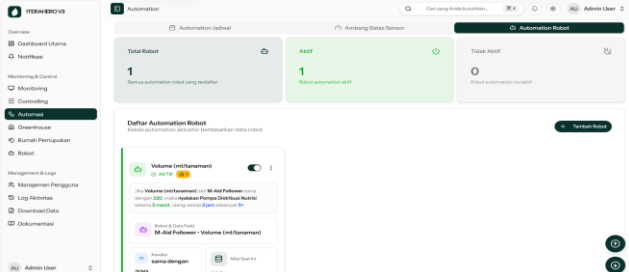
Tabel 4.3 Hasil Implementasi *Frontend*

Nama dan Gambar Implementasi <i>Frontend</i>		Deskripsi
Fitur Login		
Halaman Login 		Halaman login digunakan untuk otentikasi guna memverifikasi identitas pengguna yang memiliki akses ke subsistem (Staff, Pengelola, dan Admin) untuk menjamin hak dan keamanan akses data.
Fitur Perangkat Sensor		
List Halaman Perangkat Sensor 		Menampilkan daftar seluruh sensor (nama, nilai dan satuan, status, lokasi, dan nilai terakhir yang dikirim), tombol pengaturan untuk edit dan hapus, <i>filter</i> , paginasi, serta nilai angka parameter terkini dari sensor yang dapat diperbarui secara <i>real-time</i> .
Tambah Perangkat Sensor 		Halaman untuk menambahkan perangkat sensor, dengan mengisi <i>icon</i> perangkat, nama sensor, persamaan kalibrasi, tipe sensor, satuan sensor, lokasi, dan juga ambang batas sensor.
Ubah Perangkat Sensor		Halaman untuk mengedit

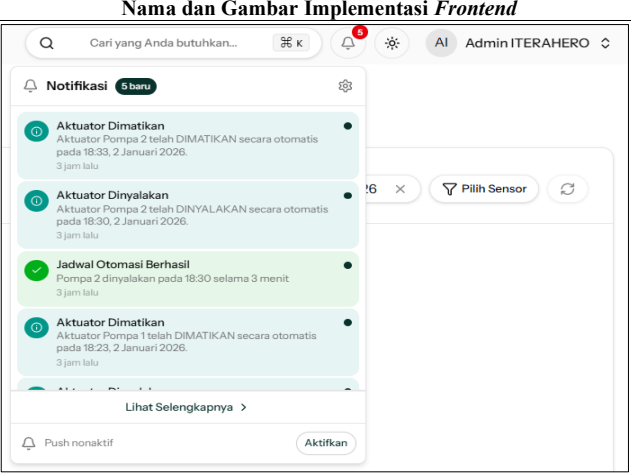
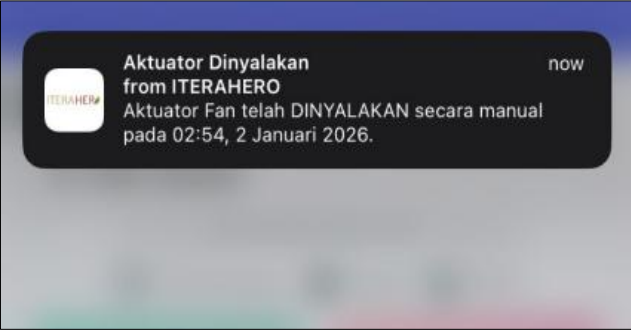
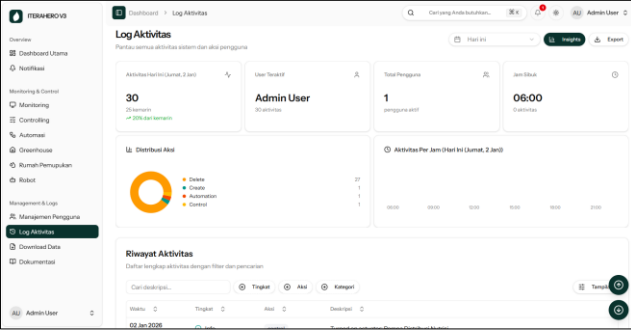
Nama dan Gambar Implementasi <i>Frontend</i> 	Deskripsi perangkat sensor, dengan mengisi <i>icon</i> perangkat, nama sensor, persamaan kalibrasi, tipe sensor, satuan sensor, lokasi, dan juga ambang batas sensor yang perlu diubah.
Hapus Perangkat Sensor 	Fitur untuk menghapus perangkat sensor dari data, yang dilengkapi dengan konfirmasi hapus supaya mencegah kasus pengguna salah menekan tombol.
Fitur Perangkat Aktuator	
List Halaman Perangkat Aktuator 	Menampilkan daftar seluruh perangkat aktuator, dengan <i>filter</i>, paginasi, serta status perangkat (aktif/nonaktif) yang bisa diperbarui secara <i>real-time</i>.
Tambah Perangkat Aktuator 	Halaman untuk menambahkan perangkat aktuator ke sistem, dengan mengisi nama alat, jenis alat, lokasi, dan icon alat.
Ubah Perangkat Aktuator	Halaman yang digunakan untuk ubah perangkat

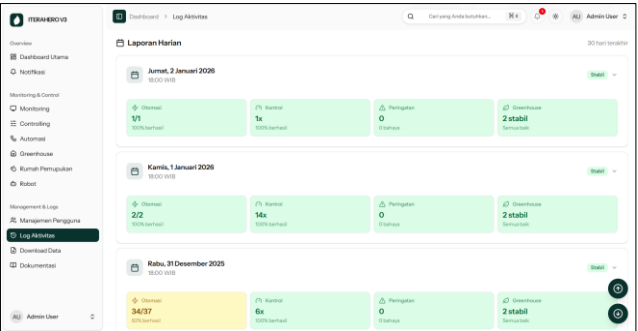
Nama dan Gambar Implementasi <i>Frontend</i>	Deskripsi
	aktuator yang sudah ada pada sistem, dengan mengisi nama alat, jenis alat, lokasi, dan <i>icon</i> alat yang perlu diubah.
	Fitur untuk menghapus perangkat aktuator dari data, yang dilengkapi dengan konfirmasi hapus supaya mencegah kasus pengguna salah menekan tombol.
Halaman Otomatisasi	
	Data rangkuman otomatisasi yang dijalankan oleh sistem, dilengkapi dengan <i>filter</i> tanggal, grafik pola aktivitas perangkat IoT pada otomatisasi.
	Riwayat lengkap otomatisasi yang dijalankan oleh sistem mengenai waktu eksekusi perangkat IoT pada otomatisasi berhasil/gagal.
Fitur Otomatisasi Jadwal	
	Menampilkan daftar seluruh aturan otomatisasi berbasis

Nama dan Gambar Implementasi <i>Frontend</i>	Deskripsi
 The screenshot shows the 'Automation' section of the application. It displays a list of automation rules with columns for 'Total Jadwal', 'Status', and 'Aksi'. The 'Total Jadwal' column shows a count of 6. The 'Status' column shows a green indicator for 'Aktif' (Active). The 'Aksi' column shows a list of actions: 'Mendinginkan ruangan', 'Menyalakan lampu', and 'Menyalakan kipas'. The interface includes a sidebar with navigation options like 'Dashboard Utama', 'Notifikasi', 'Monitoring & Log', 'Pengaturan', 'Ruang Persepsi', 'Robot', 'Manajemen User', 'Log Aktivitas', 'Download Data', and 'Dokumentasi'.	penjadwalan harian, dengan jam waktu eksekusinya, <i>filter</i>, paginasi, serta status aturan (aktif/nonaktif). Informasi dibuat dalam cerita supaya pengguna mudah memahami informasi.
 The screenshot shows the 'Tambah Jadwal Otomatisasi' (Add Automation Schedule) form. It includes fields for 'Aksi' (Action), 'Durasi (detik)' (Duration in seconds), 'Interval (detik)' (Interval in seconds), and 'Aktif' (Active). The 'Interval' field is set to 1000. The 'Aktif' checkbox is checked. The form also includes a 'Simpan' (Save) button.	Halaman untuk menambahkan aturan otomatisasi jadwal baru, dengan memilih aktuator, mengatur durasi, waktu mulai, hari-hari aktif, serta iterasi dan <i>interval</i> pengu-langan.
 The screenshot shows the 'Edit Jadwal Otomatisasi' (Edit Automation Schedule) form. It includes fields for 'Aksi' (Action), 'Durasi (detik)' (Duration in seconds), 'Interval (detik)' (Interval in seconds), and 'Aktif' (Active). The 'Interval' field is set to 1000. The 'Aktif' checkbox is checked. The form also includes a 'Simpan' (Save) button.	Halaman untuk mengedit aturan otomatisasi jadwal yang sudah ada, dengan mengubah aktuator, durasi, waktu mulai, hari-hari aktif, atau iterasi dan <i>interval</i> yang perlu diubah.
 The screenshot shows the 'Hapus Automation Jadwal' (Delete Automation Schedule) confirmation dialog. It asks the user to confirm the deletion of the automation rule. The dialog includes a 'Batal' (Cancel) button and a 'Ya, Hapus' (Yes, Delete) button.	Fitur untuk menghapus aturan otomatisasi jadwal dari data, yang dilengkapi dengan konfirmasi hapus untuk mencegah pengguna yang salah menekan tombol.
Fitur Otomatisasi Ambang Batas Sensor	
Halaman Otomatisasi Ambang Batas Sensor	Menampilkan daftar seluruh

Nama dan Gambar Implementasi <i>Frontend</i>	Deskripsi
	aturan otomatisasi berbasis ambang batas sensor, dilengkapi <i>filter</i>, paginasi, serta status aturan (aktif/nonaktif).
Tambah Otomatisasi Ambang Batas Sensor 	Halaman untuk menambahkan aturan otomatisasi baru, dengan memilih sensor, aktuator, kondisi <i>operator</i>, nilai ambang, dan aksi yang dijalankan.
Ubah Otomatisasi Ambang Batas Sensor 	Halaman untuk mengedit aturan otomatisasi yang sudah ada, dengan mengubah sensor, aktuator, kondisi <i>operator</i>, nilai ambang, atau aksi yang perlu diubah.
Hapus Otomatisasi Ambang Batas Sensor 	Fitur untuk menghapus aturan otomatisasi dari data server, yang dilengkapi dengan konfirmasi hapus.
Fitur Otomatisasi Rekomendasi Robot	
Halaman Otomatisasi Data Robot 	Menampilkan daftar seluruh aturan otomatisasi berbasis data robot, dilengkapi <i>filter</i>, paginasi, serta status aturan (aktif/nonaktif).
Tambah Otomatisasi Data Robot	Fitur ini memfasilitasi

Nama dan Gambar Implementasi <i>Frontend</i>	Deskripsi
	penambahan aturan otomatisasi baru melalui konfigurasi robot, definisi data, aktuator, operator, nilai, aksi, serta pengaturan durasi, interval, dan iterasi.
	Fitur ini digunakan untuk mengedit aturan otomatisasi yang sudah ada, dengan mengubah robot, definisi data, aktuator, kondisi <i>operator</i>, nilai, aksi, durasi, <i>interval</i>, atau iterasi yang perlu diubah.
	Fitur untuk menghapus aturan otomatisasi dari data, yang dilengkapi dengan konfirmasi hapus supaya mencegah kasus pengguna salah menekan tombol.
Fitur Notifikasi	
	Menampilkan seluruh notifikasi yang diterima pengguna, dilengkapi <i>filter</i> berdasarkan tipe notifikasi dan status baca/belum dibaca.
	Jendela <i>pop-up</i> ringkas yang menyajikan detail cepat terkait pesan notifikasi yang dikirim ke setiap pengguna.

Nama dan Gambar Implementasi <i>Frontend</i>		Deskripsi
		
Notifikasi Muncul di Perangkat 		Pesan peringatan instan (<i>push notification</i>) yang muncul di layar perangkat pengguna untuk memberitahu kejadian secara seketika.
Fitur Log Aktivitas Perangkat IoT		
Halaman Log Aktivitas Perangkat IoT 		Rekaman jejak informasi digital yang mencatat seluruh kronologi tindakan pengguna dan respons sistem untuk keperluan monitoring dan evaluasi sistem.

Nama dan Gambar Implementasi <i>Frontend</i>		Deskripsi
		

4.2 Deploy

Tahapan *deployment* merupakan proses pemindahan sistem dari lingkungan pengembangan lokal menuju peladen (*server*) produksi untuk memastikan aplikasi dapat diakses secara publik melalui jaringan internet. Konfigurasi infrastruktur dan instalasi dependensi perangkat lunak dilakukan agar seluruh fitur dapat berjalan serta berfungsi saat digunakan oleh pengguna akhir. Proses *deployment* dilakukan menggunakan teknologi kontainerisasi Docker untuk menjaga konsistensi lingkungan antara tahap pengembangan dan produksi. Aplikasi dijalankan menggunakan Laravel Octane dengan server aplikasi FrankenPHP yang terintegrasi dengan Caddy Web Server, memberikan performa tinggi untuk menangani koneksi *concurrent* yang dibutuhkan oleh *WebSockets* (Lihat Kode 4.19).

Kode 4.19 Konfigurasi *Deploy* Menggunakan Docker

```

services:
# Laravel Octane + FrankenPHP + Caddy
app:
  build:
    context: .
    dockerfile: docker/8.4/production/Dockerfile
  image: itera-hero:latest
  ports:
    - '80:80' # HTTP
    - '443:443' # HTTPS
    - '443:443/udp' # HTTP/3 QUIC
  environment:
    SUPERVISOR_PHP_COMMAND: >
    php artisan Octane:frankenphp
    --https --host=0.0.0.0 --port=443
    --http-redirect --workers=8
    --max-requests=1000
  depends_on:
    - db
    - redis
    - mosquitto
  restart: unless-stopped
  networks:
    - backend

# PostgreSQL dengan TimescaleDB
db:
  image: timescale/timescaledb:latest-pg17
  environment:
    POSTGRES_DB: '${DB_DATABASE}'
    POSTGRES_USER: '${DB_USERNAME}'
    POSTGRES_PASSWORD: '${DB_PASSWORD}'
  volumes:
    - db-data:/var/lib/postgresql/data
  healthcheck:
    test: ["CMD", "pg_isready", "-q", "-d", "${DB_DATABASE}", "-U",
"${DB_USERNAME}"]

# Redis untuk Cache dan Queue
redis:
  image: redis:alpine
  command: redis-server --appendonly yes
  volumes:
    - redis-data:/data

# MQTT Broker: Mosquitto
mosquitto:
  image: eclipse-mosquitto:2.0.21
  ports:
    - '1883:1883' # MQTT
    - '8083:8083' # WebSocket
  volumes:
    - ./docker/mosquitto/config:/mosquitto/config
    - ./docker/mosquitto/data:/mosquitto/data

networks:
  backend:
    driver: bridge

volumes:
  db-data:
  redis-data:

```

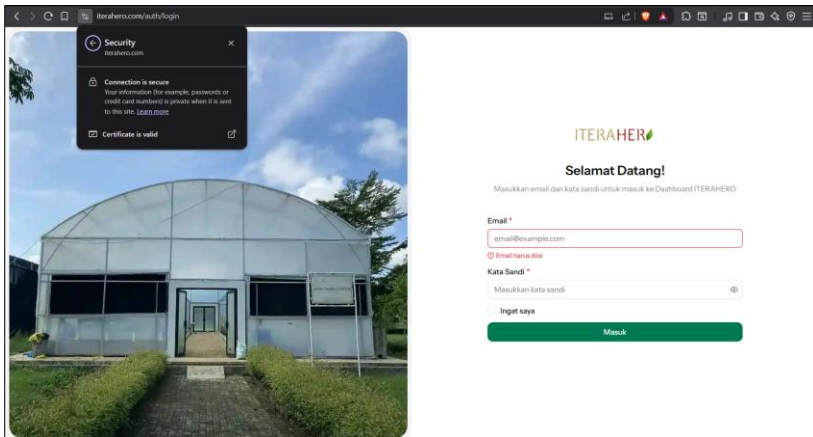
Arsitektur tersebut mengorkestrasi empat layanan utama yang beroperasi dalam jaringan internal terisolasi. Rincian peran teknis dari setiap layanan yang dikerahkan dalam arsitektur ini dijelaskan pada Tabel 4.4.

Tabel 4.4 Spesifikasi Lingkungan Produksi dan Status Akses

Komponen	Spesifikasi/Keterangan	Status
Server OS	Ubuntu 22.04 LTS (Linux)	Terkonfigurasi
Web Server	Caddy (via FrankenPHP)	Aktif (HTTPS/HTTP3)
Database	PostgreSQL 17	Terhubung
Cache & Queue	Redis (Alpine)	Berjalan
MQTT Broker	Eclipse Mosquitto 2.0.21	Aktif
Runtime	PHP 8.4 (Laravel Octane)	Optimal
Domain Akses	https://iterahero.com	Dapat Diakses

Mengacu pada spesifikasi dalam Tabel 4.4, integrasi sistem operasi Ubuntu 22.04 LTS dengan Caddy *web server* melalui *runtime* FrankenPHP menghasilkan lingkungan *production* yang mendukung protokol HTTP/3, sehingga akses aplikasi melalui domain <https://iterahero.com> terjamin keamanannya melalui enkripsi SSL otomatis (Lihat Gambar 4.1). Status terhubung pada layanan PostgreSQL 17 dan Redis mengindikasikan bahwa mekanisme persistensi serta *caching* data telah berfungsi untuk menangani

beban transaksi sistem, sementara aktifnya layanan Eclipse Mosquitto memastikan infrastruktur *broker* siap memfasilitasi pertukaran pesan antar perangkat IoT secara *real-time*.



Gambar 4.1 Hasil Deployment

4.3 Pengujian

Tahap pengujian dilaksanakan untuk memverifikasi kesesuaian operasional subsistem dengan spesifikasi kebutuhan yang telah ditetapkan. Rangkaian evaluasi ini mencakup pengujian fungsional menggunakan metode *black box testing*, pengujian komunikasi data dan otomatisasi menggunakan protokol *MQTT*, serta pengujian aspek non-fungsional sistem. Ringkasan metode dan hasil dari setiap skenario tersebut diuraikan pada sub-bab berikut.

4.2.1 Pengujian Fungsional *Blackbox Testing*

Pengujian *blackbox testing* difokuskan pada validasi keluaran (*output*) fungsional sistem berdasarkan skenario masukan (*input*) tertentu tanpa meninjau struktur kode internal program. Evaluasi ini mencakup verifikasi terhadap fitur manajemen perangkat, eksekusi logika otomatisasi, mekanisme notifikasi, dan log aktivitas untuk memastikan tidak adanya kegagalan fungsi pada operasi utama. Tabel 4.5 menyajikan rekapitulasi tingkat keberhasilan pengujian, sementara rincian lengkap setiap skenario uji dapat dilihat pada Lampiran F.

Tabel 4.5 Rekapitulasi Hasil Pengujian *Blackbox Testing*

No	Kode	Kategori Fitur	Jumlah Skenario	Skenario Berhasil	Skenario Gagal	Persentase Keberhasilan
1	BT-01	Login dan Logout	10	10	0	100%
2	BT-02	Perangkat Sensor	18	18	0	100%

No	Kode	Kategori Fitur	Jumlah Skenario	Skenario Berhasil	Skenario Gagal	Persentase Keberhasilan
3	BT-03	Perangkat Aktuator	15	15	0	100%
4	BT-04	Halaman Otomatisasi	8	8	0	100%
5	BT-05	Otomatisasi Jadwal	10	10	0	100%
6	BT-06	Otomatisasi Sensor	9	9	0	100%
7	BT-07	Otomatisasi Robot	10	10	0	100%
8	BT-08	Notifikasi	8	8	0	100%
9	BT-09	Log Aktivitas	11	11	0	100%
10	BT-10	Hak Akses	52	52	0	100%
Total	10 Kategori		151	151	0	100%

4.2.2 Pengujian Komunikasi Data dan Otomatisasi

Pengujian komunikasi data dilaksanakan untuk memverifikasi reliabilitas transmisi pesan antara subsistem *server* dan perangkat IoT menggunakan protokol MQTT. Fokus utama pengujian ini adalah validasi pengiriman paket data dengan standar *Quality of Service* (QoS) level 0 (*at most once*) untuk sensor dan data definisi robot, lalu 1 (*At least once*) untuk aktuator, yang menjamin setiap instruksi otomatisasi terkirim secara pasti ke *broker* tanpa risiko kehilangan data (*packet loss*). Pengamatan dilakukan secara *real-time* menggunakan perangkat lunak MQTTX untuk memantau kesesuaian topik (*topic*) dan isi pesan (*payload*) yang diterima oleh perangkat target. Hasil pengujian jalur komunikasi untuk berbagai skenario otomatisasi dirangkum dalam Tabel 4.6 untuk rinciannya ada pada Lampiran G.

Tabel 4.6 Ringkasan Hasil Pengujian Komunikasi Data dan Otomatisasi

No	Kategori	Jumlah Skenario	Status	Rata-rata Waktu Respon	Hasil
Komunikasi Data					
1	Sensor (<i>Manual Read</i>) QoS 0	4	100%	106 ms	Sukses
	Aktuator (<i>Manual Control</i>) QoS 1	7	100%	106 ms	Sukses
	Data Definisi Robot (<i>Manual Transfer</i>) QoS 0	1	100%	105 ms	Sukses
Otomatisasi Perangkat IoT Aktuator					
2	Ambang Batas (<i>Threshold</i>)	8 (ON/OFF)	100%	1.174 ms	Sukses
	Penjadwalan (<i>Scheduling</i>)	10 (ON/OFF)	100%	1.663 ms	Sukses
	Data Definisi Robot	2 (ON/OFF)	100%	1.506 ms	Sukses

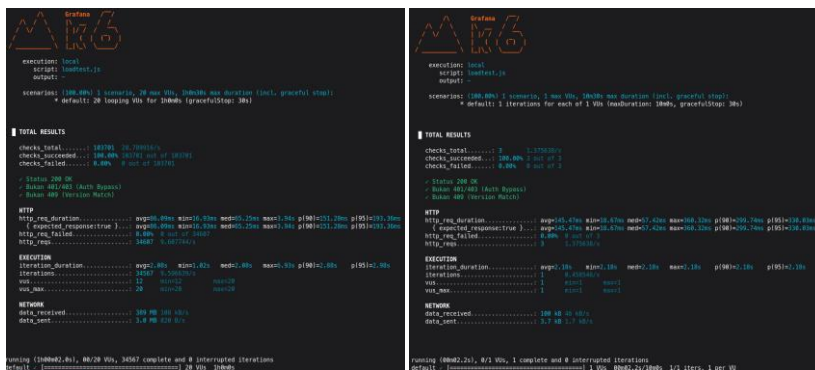
Mengacu pada Tabel 4.6, kategori Komunikasi Data Manual menunjukkan rata-rata respons 105–106 ms. Dalam perspektif teori Nielsen (1993), angka ini berada sedikit di atas limit 0.1 detik (*Instant Feel*) namun

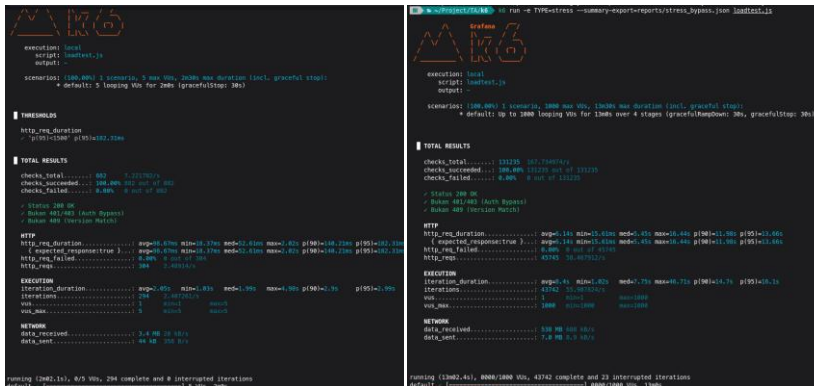
sangat aman di bawah limit 1.0 detik (*Flow of Thought*). Capaian ini mengindikasikan bahwa implementasi *WebSocket* berhasil memfasilitasi interaksi *real-time* yang menjaga alur kognitif pengguna tetap mulus saat memantau sensor, meskipun secara perseptual terdapat jeda mikro yang nyaris tidak terdeteksi dibandingkan reaksi instan perangkat fisik.

Sebaliknya, kategori Otomatisasi mencatat rata-rata 1.174 ms hingga 1.663 ms, yang melampaui batas 1.0 detik Nielsen sehingga pengguna akan menyadari adanya proses komputasi komputer. Keterlambatan ini merupakan konsekuensi teknis penerapan protokol MQTT QoS 1 dan antrian proses *server* (*Laravel Queue*) demi menjamin validitas instruksi. Meskipun demikian, kinerja ini dinilai sukses karena seluruh skenario—termasuk *peak latency* 4.950 ms saat *cold start*—tetap konsisten berada jauh di bawah batas kritis 10 detik (*User Attention*), sehingga menjamin pengguna tetap fokus menunggu penyelesaian proses tanpa risiko meninggalkan antarmuka.

4.2.3 Pengujian Non-Fungsional

Evaluasi terhadap aspek *non-functional* sistem dilaksanakan dengan merujuk pada skenario yang telah ditetapkan sebelumnya, di mana hasil eksekusi direkam menggunakan perangkat lunak *k6*. Sebagaimana divisualisasikan pada Gambar 4.2 (a)-(d), pengujian mencakup empat dimensi utama: stabilitas jangka panjang (*soak test*), keamanan akses (*security test*), validasi awal (*smoke test*), dan ketahanan terhadap beban puncak (*stress test*). Seluruh data statistik yang diperoleh dari metrik *checks*, *http_req_duration*, dan *iterations* telah direkapitulasi untuk memetakan perilaku sistem di bawah berbagai kondisi operasional, sebagaimana dirincikan dalam Tabel 4.7.





(c) *Smoke Test* (d) *Stress Test*

Gambar 4.2 Kumpulan Hasil Pengujian Non-Fungsional k6

Tabel 4.7 Hasil Pengujian Non-Fungsional

Jenis Pengujian	Parameter Beban	Metrik Kunci (Average)	Metrik Ekstrem (p95)	Tingkat Keberhasilan	Status
Smoke Test	5 VUs (2 menit)	98.67 ms	182.31 ms	100%	Passed
Soak Test	20 VUs (1 jam)	86.09 ms	193.36 ms	100%	Passed
Security Test	1 VU (Validasi Akses)	145.47 ms	330.03 ms	100%	Passed
Stress Test	1000 VUs (Ramping)	6.14 s	13.66 s	100%	Passed w/ Note

Mengacu pada data hasil pengujian Tabel 4.7, terlihat bahwa sistem mampu mempertahankan latensi yang rendah pada kondisi beban ringan hingga menengah, terbukti dari hasil *smoke test* dan *soak test* yang mencatatkan waktu respons rata-rata di bawah 100 ms dengan tingkat kegagalan 0%. Stabilitas ini mengindikasikan bahwa manajemen memori dan alokasi sumber daya berjalan optimal selama durasi pengujian satu jam tanpa adanya degradasi performa (*memory leak*). Sebaliknya, pada skenario *stress test* dengan beban ekstrem mencapai 1000 *virtual users*, terjadi lonjakan latensi yang signifikan hingga mencapai rata-rata 6,14 detik dan *p95* 13,66 detik, yang disebabkan oleh antrean permintaan (*request queuing*) yang melebihi kapasitas pemrosesan konkuren server. Meskipun terjadi penurunan kecepatan respons, sistem menunjukkan tingkat ketahanan (*resilience*) yang tinggi karena tidak ditemukan adanya *error* atau *request dropped* (tingkat keberhasilan tetap 100%), yang menegaskan bahwa mekanisme penanganan beban berlebih berfungsi menjaga ketersediaan layanan (*availability*) meski beroperasi dalam kondisi saturasi.

4.4 Review

Tahap tinjauan (*review*) dilaksanakan bersama penanggung jawab *Greenhouse* Kebun Raya ITERA, Ibu Zunanik Mufidah, S.TP., M.Si., guna memverifikasi kesiapan operasional sistem pada seluruh fitur yang dikembangkan. Validasi difokuskan pada pengujian fungsi autentikasi, manajemen perangkat sensor dan aktuator, serta akurasi eksekusi tiga skenario otomatisasi utama yang meliputi penjadwalan, ambang batas, dan definisi data robot. Kinerja mekanisme distribusi notifikasi serta ketelusuran data pada *log* aktivitas turut dievaluasi untuk memastikan integritas pemantauan di lapangan. Temuan hasil validasi beserta rekomendasi tindak lanjut perbaikan dirangkum dalam Tabel 4.8.

Tabel 4.8 Tahapan *Review* Hasil Implementasi

No.	Fitur Skenario	Hasil Validasi	Catatan/Tindak Lanjut
1.	Login/Logout	Sesuai	-
2.	Perangkat Sensor	Sesuai	-
3.	Perangkat Aktuator	Sesuai	-
4.	Otomatisasi Penjadwalan	Sesuai, perlu perbaikan minor	Penambahan <i>filter</i> lokasi dan pencarian pada daftar otomatisasi serta form pemilihan aktuator.
5.	Otomatisasi Ambang Batas Sensor	Sesuai, perlu perbaikan minor	Penambahan <i>filter</i> lokasi dan pencarian pada daftar otomatisasi serta form pemilihan sensor dan aktuator.
6.	Otomatisasi Data Robot	Sesuai, perlu perbaikan minor	Penambahan <i>filter</i> lokasi dan pencarian pada daftar otomatisasi serta form pemilihan data robot dan aktuator.
7.	Notifikasi	Sesuai, perlu perbaikan minor	Implementasi <i>filter</i> kategori notifikasi untuk meminimalisir penumpukan informasi (<i>spam</i>).
8.	Log Aktivitas	Sesuai, perlu perbaikan minor	Pencantuman identitas pengguna (nama dan email) pada log riwayat aktivitas.

A. Implementasi Perbaikan

Merespons temuan dan catatan tindak lanjut pada Tabel 4.8, proses modifikasi sistem dilakukan untuk meningkatkan efisiensi navigasi data serta aspek transparansi aktivitas pengguna. Implementasi perbaikan ini dieksekusi secara terstruktur pada dua lapisan arsitektur utama, yaitu penyesuaian *query* dan API di sisi *backend* serta integrasi komponen interaktif di sisi *frontend*, guna memastikan seluruh kebutuhan operasional tambahan dapat terakomodasi tanpa mendegradasi performa sistem yang telah berjalan.

1. Implementasi Perbaikan *Backend*

Menindaklanjuti temuan validasi operasional pada Tabel 4.8, perbaikan sistem difokuskan pada pengayaan logika pemrosesan data di sisi *backend* guna mendukung kebutuhan interaksi yang lebih spesifik. Modifikasi ini mencakup implementasi parameter *query* tambahan pada *endpoint* API otomatisasi untuk memfasilitasi fungsi pencarian dan penyaringan (*filtering*) berbasis lokasi, serta penyesuaian struktur respon data pada modul notifikasi dan *log* aktivitas agar memuat atribut kategori prioritas dan identitas pengguna secara eksplisit. Rincian teknis mengenai

perubahan kode dan logika yang diterapkan pada lapisan server disajikan dalam Tabel 4.9.

Tabel 4.9 Implementasi Perbaikan *Backend*

No.	Temuan Validasi	File Perbaikan	Deskripsi Implementasi
1	Filter lokasi dan pencarian pada otomatisasi penjadwalan	<i>AutomationScheduleService.php</i>	Penambahan parameter search dan location pada query builder untuk memfilter data berdasarkan nama aktuator dan kode lokasi
2	Filter lokasi dan pencarian pada otomatisasi ambang batas sensor	<i>AutomationSensorService.php</i>	Implementasi filter ganda pada relasi sensor dan actuato dengan dukungan pencarian berdasarkan nama perangkat
3	Filter lokasi dan pencarian pada otomatisasi data robot	<i>AutomationRobotService.php</i>	Penambahan query scope untuk memfilter berdasarkan nama robot, nama aktuator, dan definisi data
4	Filter kategori notifikasi	<i>NotificationController.php, notification-utils.ts</i>	Implementasi deteksi kategori otomatis berdasarkan pattern matching pada judul dan deskripsi notifikasi
5	Identitas pengguna pada log aktivitas	<i>ActivityLogger.php, activity_logs migration</i>	Pencantuman kolom user name dan user_email pada tabel log untuk menjaga ketelusuran meskipun data pengguna dihapus

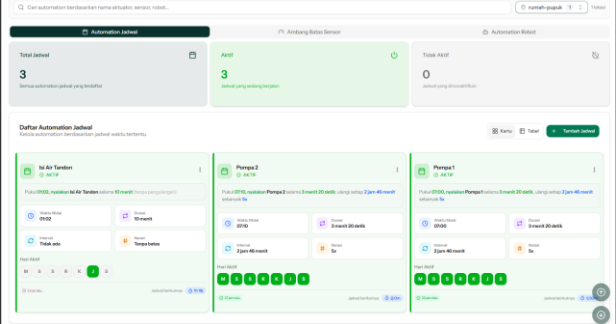
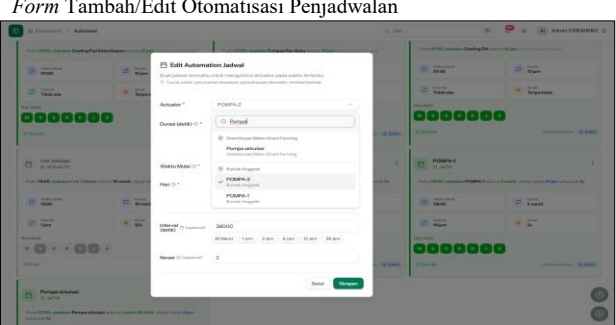
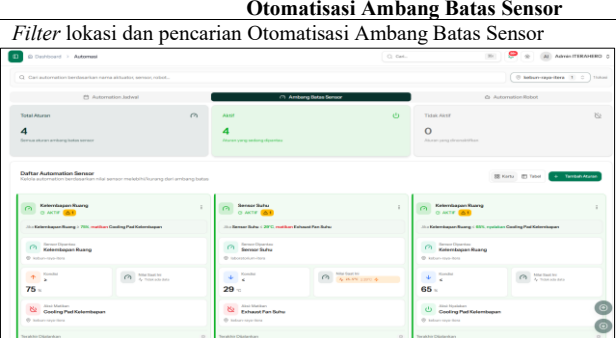
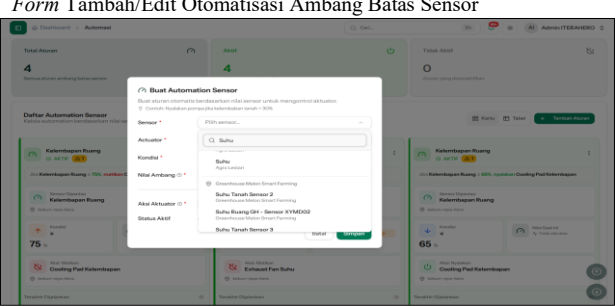
Modifikasi pada tiga poin pertama berfokus pada penerapan logika pencarian nama dan penyaringan lokasi di lapisan layanan aplikasi. Pendekatan ini memungkinkan sistem menampilkan data yang spesifik sesuai masukan pengguna secara akurat. Selanjutnya, fitur pengelompokan notifikasi dikembangkan menggunakan deteksi kata kunci sederhana pada judul dan deskripsi untuk menentukan kategori pesan secara otomatis. Terakhir, struktur tabel aktivitas diperbarui dengan menyimpan informasi nama dan *email* pengguna secara langsung, yang bertujuan menjaga kelengkapan riwayat penggunaan sistem meskipun akun pengguna terkait telah dihapus dari basis data.

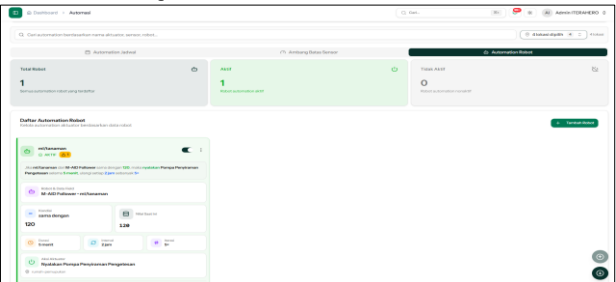
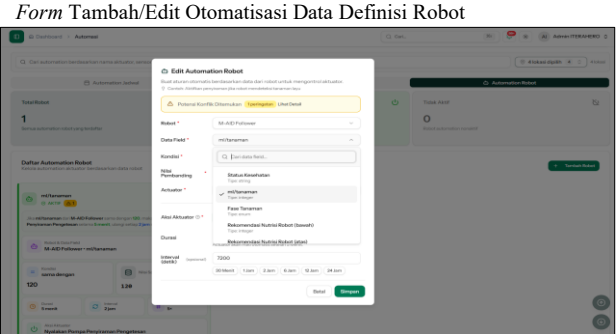
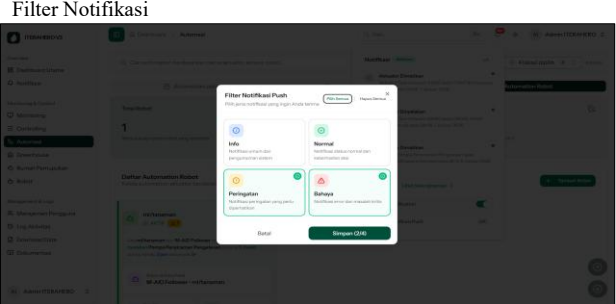
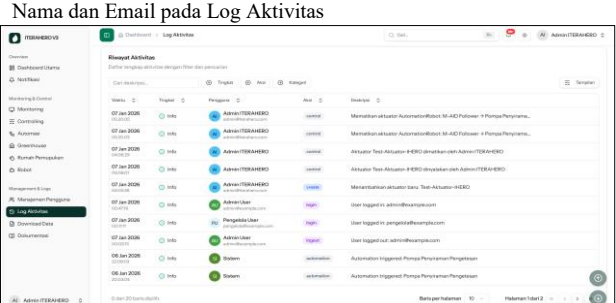
2. Implementasi Perbaikan *Frontend*

Melengkapi perbaikan yang telah dilakukan pada sisi server, modifikasi antarmuka (*frontend*) diterapkan untuk memfasilitasi akses pengguna terhadap fitur baru secara langsung. Fokus pengembangan diarahkan pada integrasi elemen navigasi seperti kolom pencarian dan filter lokasi di setiap modul otomatisasi, serta penyesuaian tampilan notifikasi agar kategori pesan mudah diidentifikasi. Seluruh perubahan elemen visual yang diterapkan untuk memenuhi catatan validasi tersebut didokumentasikan dalam Tabel 4.10.

Tabel 4.10 Implementasi Perbaikan *Frontend*

Nama dan Gambar Implementasi Perbaikan <i>Frontend</i>	Deskripsi
Otomatisasi Penjadwalan	
<i>Filter</i> lokasi dan pencarian Otomatisasi Penjadwalan	Penambahan fitur filter lokasi dan pencarian (<i>search</i>)

Nama dan Gambar Implementasi Perbaikan <i>Frontend</i>	Deskripsi
	bertujuan memudahkan pengguna menemukan data jadwal spesifik, sehingga waktu penelusuran informasi menjadi lebih singkat.
Form Tambah/Edit Otomatisasi Penjadwalan 	Implementasi pencarian (<i>search</i>) pada formulir pemilihan aktuator dilakukan untuk mempercepat proses seleksi perangkat, sehingga meminimalisir kesalahan pemilihan akibat banyaknya daftar perangkat.
Otomatisasi Ambang Batas Sensor Filter lokasi dan pencarian Otomatisasi Ambang Batas Sensor 	Penerapan filter lokasi dan fitur pencarian (<i>search</i>) membantu pengguna memilah aturan sensor berdasarkan area tertentu, yang berdampak pada kemudahan pengelolaan data pemantauan.
Form Tambah/Edit Otomatisasi Ambang Batas Sensor 	Penambahan fitur pencarian (<i>search</i>) pada pilihan sensor dan aktuator bertujuan membantu pengguna memilih perangkat dengan tepat, terutama saat jumlah perangkat dalam daftar cukup banyak.
Otomatisasi Data Definisi Robot	

Nama dan Gambar Implementasi Perbaikan <i>Frontend</i>	Deskripsi
Filter lokasi dan pencarian Otomatisasi Data Definisi Robot 	Penggunaan filter lokasi dan pencarian (<i>search</i>) memudahkan pengguna memantau konfigurasi robot pada zona spesifik, sehingga fokus pemantauan menjadi lebih terarah.
Form Tambah/Edit Otomatisasi Data Definisi Robot 	Fitur pencarian (<i>search</i>) pada formulir pemilihan data robot ditambahkan untuk mempercepat akses ke definisi variabel yang dibutuhkan, sehingga proses konfigurasi logika berjalan lebih lancar.
Notifikasi	
Filter Notifikasi 	Penambahan filter kategori memungkinkan pengguna memilih jenis informasi yang ingin ditampilkan, sehingga interaksi pengguna tidak terganggu oleh penumpukan pesan yang kurang relevan.
Log Aktivitas	
Nama dan Email pada Log Aktivitas 	Penambahan informasi nama dan email pada riwayat aktivitas bertujuan memperjelas identitas pelaksana tindakan, sehingga penelusuran tanggung jawab penggunaan sistem menjadi lebih akurat.

B. Pengujian Perbaikan

Pengujian perbaikan dilaksanakan menggunakan metode *black box testing* untuk memverifikasi fungsionalitas fitur pasca modifikasi antarmuka. Pendekatan ini berfokus pada validasi kesesuaian *input* dan *output* pada modul otomatisasi, notifikasi, dan pencatatan aktivitas tanpa meninjau struktur kode internal. Skenario pengujian dirancang spesifik untuk menguji respons sistem terhadap penggunaan filter lokasi, fungsi pencarian data, serta representasi informasi khusus admin yang menampilkan identitas pengguna pada log riwayat aktivitas sebagaimana dijabarkan pada Tabel 4.11.

Tabel 4.11 Hasil Pengujian *Black Box* Implementasi Perbaikan

No	Kode	Fitur	Skenario Pengujian	Ekspektasi	Hasil
1	P-01	Otomatisasi Penjadwalan	Memasukkan kata kunci pada kolom pencarian dan memilih filter lokasi pada daftar jadwal.	Sistem hanya menampilkan data jadwal yang mengandung kata kunci dan lokasi yang dipilih.	Berhasil
2	P-02	Form Ambang Batas Sensor	Mengetik nama sensor/aktuator pada kolom pencarian saat tambah/edit data.	<i>Dropdown menu menyaring daftar perangkat secara real-time sesuai karakter yang diketik.</i>	Berhasil
3	P-03	Otomatisasi Data Robot	Mengaktifkan filter lokasi pada halaman daftar definisi robot.	Tabel data diperbarui secara dinamis menampilkan hanya konfigurasi robot pada lokasi terkait.	Berhasil
4	P-04	Notifikasi	Memilih kategori "Bahaya" pada menu filter notifikasi.	Daftar notifikasi hanya memuat pesan dengan label prioritas "Bahaya" dan menyembunyikan kategori lain.	Berhasil
5	P-05	<i>Log Aktivitas</i>	Melakukan aksi perubahan data (edit/hapus) dan memeriksa tabel riwayat.	Baris data log terbaru memuat atribut nama lengkap dan alamat email pengguna yang melakukan aksi.	Berhasil
Persentase Berhasil					100%

Mengacu pada rekapitulasi hasil dalam Tabel 4.11, seluruh skenario pengujian terhadap fitur perbaikan menunjukkan status valid. Responsivitas fitur pencarian dan filter pada modul otomatisasi membuktikan bahwa logika seleksi data pada sisi antarmuka (*frontend*) telah berfungsi optimal dalam memilah informasi sesuai parameter masukan. Lebih lanjut, efektivitas filter kategori notifikasi serta penambahan atribut identitas pada *log* aktivitas menegaskan bahwa mekanisme penyajian data telah diperbarui untuk mereduksi kepadatan informasi serta memenuhi aspek keterlacakan pengguna yang menjadi catatan evaluasi sebelumnya.

C. Review Perbaikan

Tahap validasi ulang dilaksanakan pasca-implementasi perbaikan antarmuka guna memverifikasi fungsionalitas fitur yang telah dimodifikasi. Fokus pengujian diarahkan secara spesifik pada kinerja mekanisme filter dan pencarian pada seluruh modul otomatisasi, serta efektivitas pengelompokan notifikasi dan kelengkapan atribut data pada riwayat aktivitas. Hasil

rekapitulasi pengujian tahap akhir ini disajikan secara ringkas dalam Tabel 4.12.

Tabel 4.12 Tahapan Review Hasil Setelah Perbaikan

No.	Fitur Skenario	Hasil Validasi	Catatan/Tindak Lanjut
1.	Otomatisasi Penjadwalan	Sesuai	-
2.	Otomatisasi Ambang Batas Sensor	Sesuai	-
3.	Otomatisasi Data Robot	Sesuai	-
4.	Notifikasi	Sesuai	-
5.	Log Aktivitas	Sesuai	-

Mengacu pada hasil validasi dalam Tabel 4.10, seluruh fitur yang sebelumnya memiliki catatan kini telah berstatus sesuai tanpa rekomendasi revisi lanjutan. Keberhasilan ini mengindikasikan bahwa integrasi fungsi filter dan pencarian pada modul otomatisasi berfungsi optimal dalam memilah data. Selain itu, mekanisme kategorisasi notifikasi terbukti mampu menyajikan informasi yang relevan sesuai prioritas, sementara penambahan identitas pengguna pada *log* aktivitas mengonfirmasi bahwa sistem telah mengakomodasi seluruh catatan perbaikan yang direkomendasikan di Tabel 4.8 pada tahap *review* awal. Dengan tidak adanya temuan isu baru, sistem dinyatakan telah memenuhi spesifikasi kebutuhan operasional pengguna secara utuh.

4.5 Pengujian *Usability*

Pengujian *usability* dilakukan setelah *review perbaikan* dilakukan untuk mengukur kualitas interaksi pengguna terhadap subsistem dengan melibatkan 43 responden yang terdiri dari staf, pengelola, dan admin sistem. Evaluasi ini menerapkan kuesioner *System Usability Scale* (SUS) untuk mendapatkan skor kelayakan dan kebergunaan sistem. Rekapitulasi penilaian subjektif responden terhadap sepuluh indikator SUS disajikan secara rinci pada Tabel 4.13.

Tabel 4.13 Hasil Pengujian SUS

R	Pertanyaan										Jumlah	Nilai SUS
	Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8	Q9	Q10		
R1	4	2	3	1	4	4	3	4	3	1	29	72.5
R2	4	1	3	4	3	3	3	4	4	1	30	75
R3	2	3	4	2	4	3	3	3	4	3	31	77.5
R4	4	3	3	3	4	3	4	3	4	3	34	85
R5	4	1	3	4	3	2	4	3	3	1	28	70
R6	4	4	4	4	4	4	4	4	4	4	40	100
R7	4	4	4	4	4	4	4	4	4	4	40	100
R8	4	3	4	4	4	3	4	4	4	3	37	92.5
R9	4	3	4	2	4	3	4	4	4	3	35	87.5
R10	4	2	4	0	4	2	4	4	4	2	30	75
R11	4	2	4	3	3	3	4	4	4	1	32	80
R12	3	3	4	2	3	2	3	4	3	0	27	67.5
R13	1	2	2	4	0	2	1	1	1	3	17	42.5
R14	3	3	3	1	3	3	2	3	3	3	27	67.5

R	Pertanyaan										Jumlah	Nilai SUS
	Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8	Q9	Q10		
R15	4	0	4	4	4	0	4	4	4	0	28	70
R16	3	3	3	3	4	3	2	2	3	3	29	72.5
R17	3	2	4	1	3	2	3	4	3	3	28	70
R18	3	4	4	4	3	4	4	4	4	4	38	95
R19	4	4	4	4	4	4	4	4	4	3	39	97.5
R20	3	2	2	0	4	3	3	2	4	2	25	62.5
R21	4	3	4	2	3	2	4	3	3	2	30	75
R22	3	3	3	0	4	3	4	4	4	0	28	70
R23	3	3	4	2	4	3	4	4	4	1	32	80
R24	4	4	3	3	3	3	3	4	3	1	31	77.5
R25	0	4	0	1	0	1	0	1	0	1	8	20
R26	3	4	4	4	3	1	3	4	3	4	33	82.5
R27	4	1	4	3	3	3	4	4	3	0	29	72.5
R28	3	4	4	3	4	4	3	3	4	4	36	90
R29	4	3	3	4	3	4	4	3	4	4	36	90
R30	3	3	4	4	4	3	1	1	0	0	23	57.5
R31	3	3	4	3	3	4	3	4	3	4	34	85
R32	3	4	4	4	4	4	4	3	3	4	37	92.5
R33	3	4	3	4	4	4	3	4	2	3	34	85
R34	4	2	2	4	3	4	4	4	3	4	34	85
R35	4	4	3	4	3	3	3	4	4	4	36	90
R36	4	4	3	4	4	2	4	4	3	3	35	87.5
R37	4	4	4	4	3	3	3	4	2	3	34	85
R38	3	4	3	3	3	4	3	4	2	4	33	82.5
R39	4	4	4	4	3	3	4	3	3	4	36	90
R40	3	3	3	3	4	4	3	4	3	4	34	85
R41	4	4	4	4	4	4	3	3	3	4	37	92.5
R42	3	4	3	3	2	4	4	4	4	3	34	85
R43	4	4	4	4	3	3	2	2	4	4	34	85
$\bar{x}$	3.40	3.05	3.42	2.98	3.33	3.02	3.26	3.42	3.21	2.60	Skor SUS	79.19

Keterangan: R=Responden, Q ganjil=(Jawaban Pengguna – 1), Q genap=(Jawaban Pengguna – 1)

Evaluasi terhadap nilai rata-rata per indikator memperlihatkan dominasi skor pada aspek kemudahan penggunaan (Q3) dan kenyamanan interaksi (Q8) dengan nilai identik 3,42, yang mengimplikasikan bahwa responden dapat mengoperasikan sistem tanpa hambatan kognitif signifikan. Tingginya skor pada aspek preferensi penggunaan rutin (Q1) sebesar 3,40 turut memvalidasi relevansi fungsional sistem terhadap kebutuhan operasional pengguna. Meskipun skor terendah pada aspek prasyarat belajar (Q10) sebesar 2,60 mengindikasikan adanya kurva pembelajaran (*learning curve*) minor di tahap awal, hal tersebut masih dalam batas wajar mengingat kompleksitas fitur yang ditawarkan. Mengacu pada standar instrumen penilaian yang tersaji dalam Gambar 2.6 dan rincian kuantitatif pada Gambar 2.7, akumulasi skor akhir 79,19 *Good* menempatkan subsistem pada kategori *acceptable* dengan predikat *Grade A-* penilaian *adjective Good*, serta klasifikasi NPS Promoter (Tinggiat pengguna merekomendasikan subsistem ini ke oranglain) sesuai standar pada Gambar 2.7. Klasifikasi ini menegaskan

bahwa tingkat *usability* sistem telah memenuhi standar kelayakan interaksi untuk diimplementasikan ke lingkungan produksi.

4.6 Analisis

Implementasi subsistem otomatisasi perangkat *IoT* dengan arsitektur *event-driven* terbukti mampu menjaga stabilitas layanan dan integritas data. Hasil pengujian *black box* pada 151 skenario menunjukkan keberhasilan penuh dalam memvalidasi logika bisnis tanpa adanya kegagalan fungsi. Ketahanan infrastruktur juga terkonfirmasi melalui *stress test* yang tetap mempertahankan tingkat keberhasilan transaksi 100 persen, meskipun terjadi lonjakan latensi rata-rata hingga 6,14 detik saat beban mencapai 1000 *virtual users*. Perbedaan waktu respons antara kendali manual yang cepat (106 ms) dan eksekusi otomatisasi yang lebih lambat (1,1–1,6 detik) terjadi akibat penerapan mekanisme *queue* dan *Quality of Service* (QoS) level 1 pada protokol MQTT. Langkah teknis ini diterapkan untuk mencegah risiko kehilangan data (*packet loss*) dan memastikan instruksi aktuator diproses secara asinkron tanpa membebani proses utama, sehingga performa sistem tetap stabil dalam penggunaan jangka panjang seperti yang ditunjukkan hasil *soak test*.

Penerimaan pengguna terhadap antarmuka sistem tercermin dari skor *System Usability Scale* (SUS) sebesar 79,19 yang menempatkan subsistem pada kategori *Acceptable* dengan predikat *Grade A-*. Tingginya skor pada aspek kemudahan penggunaan mengindikasikan bahwa fitur visualisasi data dan filter pencarian efektif dalam membantu navigasi operasional pengguna. Skor yang sedikit lebih rendah pada indikator prasyarat belajar dinilai wajar mengingat sistem memiliki fitur teknis yang cukup padat, seperti konfigurasi logika robot dan ambang batas sensor. Gabungan data hasil pengujian fungsional yang valid, ketahanan sistem di bawah beban tinggi, serta respon positif dari pengguna menegaskan bahwa subsistem ini layak diterapkan pada lingkungan produksi *Greenhouse* Kebun Raya ITERA dengan tetap memerlukan pemeliharaan rutin untuk mengantisipasi potensi antrean data saat beban memuncak.

BAB V

KESIMPULAN

5.1 Kesimpulan

Berdasarkan seluruh tahapan penelitian yang telah dilaksanakan, mulai dari analisis kebutuhan hingga pengujian akhir terhadap subsistem otomatisasi, dapat ditarik kesimpulan sebagai berikut:

1. Rancang bangun subsistem otomatisasi perangkat *IoT* berbasis *website* telah berhasil diimplementasikan melalui penerapan metode *Agile Kanban* yang menghasilkan arsitektur sistem berbasis *event-driven*. Subsistem ini dikembangkan menggunakan kerangka kerja Laravel dan Inertia.js yang terintegrasi dengan protokol MQTT dan *WebSockets* untuk menangani komunikasi data secara *real-time*. Hasil implementasi mencakup fitur manajemen perangkat sensor dan aktuator, serta tiga mekanisme otomatisasi utama yaitu berbasis penjadwalan (*scheduling*), ambang batas sensor (*threshold*), dan rekomendasi data robot, yang seluruhnya telah beroperasi pada lingkungan produksi menggunakan teknologi kontainerisasi Docker.
2. Pengukuran kinerja subsistem menunjukkan hasil yang valid dan layak pada ketiga aspek pengujian. Pengujian fungsional menggunakan *Black-Box Testing* pada 151 skenario menghasilkan tingkat keberhasilan 100 persen. Pengujian komunikasi data mencatat waktu respons kendali manual rata-rata 105-106 milidetik dan eksekusi otomatisasi 1,1 hingga 1,6 detik, di mana latensi pada otomatisasi merupakan konsekuensi teknis penerapan mekanisme antrian (*queue*) dan *Quality of Service* (QoS) level 1 untuk menjamin pengiriman data tanpa kegagalan (*packet loss*). Sementara itu, evaluasi tingkat kebergunaan (*usability*) menggunakan metode *System Usability Scale* (SUS) terhadap 43 responden menghasilkan skor akhir 79,19 yang masuk dalam kategori *Acceptable* dengan predikat *Grade A-*, mengindikasikan bahwa sistem dapat diterima dan dioperasikan dengan baik oleh pengguna.

5.2 Saran

Optimalisasi dan pengembangan keberlanjutan subsistem untuk mendukung operasional otomatisasi *greenhouse* di masa mendatang memerlukan implementasi dari beberapa rekomendasi strategis berikut:

1. Integrasi *Artificial Intelligence* (AI) Disarankan menerapkan algoritma *Machine Learning* untuk menganalisis pola data historis sensor. Hal ini bertujuan agar otomatisasi dapat berjalan berdasarkan prediksi kondisi

lingkungan (*predictive analytics*), tidak hanya bergantung pada aturan ambang batas statis.

2. Peningkatan Skalabilitas Infrastruktur Mengingat adanya peningkatan latensi pada beban tinggi, disarankan memisahkan layanan *broker MQTT* dan *worker queue* ke infrastruktur server yang berbeda atau menerapkan arsitektur *microservices* guna menjaga performa saat pengguna bertambah.
3. Penerapan *Edge Computing* Disarankan untuk menanamkan logika otomatisasi kritis langsung pada mikrokontroler lokal. Langkah ini penting untuk menjamin otomatisasi tetap berjalan (*fail-safe*) meskipun koneksi internet atau server pusat mengalami gangguan.
4. Pengembangan Aplikasi *Mobile Native* Perlu dikembangkan aplikasi berbasis Android atau iOS untuk memberikan pengalaman pengguna yang lebih responsif dibandingkan *browser*, serta memaksimalkan keandalan fitur *push notification* bagi petugas di lapangan.
5. Pemanfaatan *Workflow Automation* Eksternal Disarankan mengintegrasikan *tools* seperti *n8n* untuk mempermudah konektivitas sistem dengan aplikasi pihak ketiga (seperti Telegram, Google Sheets, atau Slack). Hal ini akan memperluas fungsionalitas pelaporan dan notifikasi tanpa menambah kompleksitas kode di sisi *backend*.

DAFTAR PUSTAKA

- [1] Presiden Republik Indonesia, “Peraturan Presiden (PERPRES) Nomor 83 Tahun 2023 tentang Penyelenggaraan Kebun Raya,” 2023. [Online]. Available: <https://peraturan.bpk.go.id/Details/274235/perpres-no-83-tahun-2023>
- [2] Pemerintah Pusat, “Undang-undang (UU) Nomor 11 Tahun 2019 tentang Sistem Nasional Ilmu Pengetahuan dan Teknologi,” 2019. [Online]. Available: <https://peraturan.bpk.go.id/Details/117023/uu-no-11-tahun-2019>
- [3] Institut Teknologi Sumatera, “Rencana Strategis (Renstra) Institut Teknologi Sumatera 2025-2029,” 2025, *Lampung Selatan*.
- [4] Rudiyanasyah, “Golden Melon Tumbuh Subur di Smart Farming System ITERA,” Jan. 2023. [Online]. Available: <https://www.itera.ac.id/golden-melon-tumbuh-subur-di-smart-farming-system-itera/>
- [5] D. Y. Setyawan, W. Warsito, R. Marjunus, N. Nurfiana, and R. Syahputri, “A Systematic Literature Review: Internet of Things on Smart Greenhouse,” *Int J Adv Comput Sci Appl*, vol. 13, no. 12, p. 672, 2022, [Online]. Available: www.ijacsa.thesai.org
- [6] F. Abou-Mehdi-Hassani, A. Zaguia, H. Ait Bouh, and A. Mkhida, “Systematic literature review of smart greenhouse monitoring,” *SN Comput Sci*, vol. 6, no. 1, p. 95, 2025, doi: 10.1007/s42979-024-03640-4.
- [7] V. Jagatha, A. Khamesipour, and S. Chung, “Cross-Platform App Development: A Comparative Study of PWAs and React Native Mobile Apps,” in *2023 Proceedings of the ISCAP Conference*, Albuquerque, NM: ISCAP (Information Systems and Computing Academic Professionals), 2023. [Online]. Available: <https://iscap.us/proceedings/>
- [8] R. A. Nugraha, “Sistem Monitoring Smart Greenhouse Berbasis IoT Dengan Menggunakan Metode Agile Kanban (Studi Kasus Greenhouse Melon ITERA),” Institut Teknologi Sumatera, Lampung Selatan, 2023.
- [9] R. Appelqvist and O. Örmmyr, “Performance comparison of XHR polling, Long polling, Server sent events and Websockets,” Blekinge Institute of Technology, Faculty of Computing, Karlskrona, Sweden, 2017.
- [10] M. Kleppmann, *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O’Reilly Media, 2017.
- [11] Suhandi, S. Purnama, R. Ahsanitaqwm, and S. Nurm, “The Role of Automation and IoT in Enhancing Operational Efficiency: Evidence from PLS-SEM Analysis,” *APTISI Transactions on Management (ATM)*, vol. 9, no. 1, pp. 72–81, 2025, doi: 10.33050.
- [12] S. A. Khajeh, M. Saberikamarposhti, and A. M. Rahmani, “Real-Time Scheduling in IoT Applications: A Systematic Review,” *Sensors*, vol. 23, no. 1, p. 232, 2023, doi: 10.3390/s23010232.
- [13] M. I. Hossain, “Software Development Life Cycle (SDLC) Methodologies for Information Systems Project Management,” *International Journal for Multidisciplinary Research (IJFMR)*, vol. 5, no. 5, 2023, doi: 10.36948/ijfmr.2023.v05i05.6228.
- [14] B. D. Magar, “Comparative Study of Agile Methodologies: Scrum, Kanban, and Extreme Programming (XP),” Vaasan Ammattikorkeakoulu University of Applied Sciences, 2025.
- [15] E. Brechner, *Agile Project Management with Kanban*. Redmond, Washington: Microsoft Press, 2015.

- [16] Tim Penyusun, “Pedoman Tugas Akhir Capstone Program Studi Teknik Informatika,” 2024, *Lampung Selatan*.
- [17] M. S. Tuloli, R. Patalangi, and R. Takdir, “Pengukuran Tingkat Usability Sistem Aplikasi e-Rapor Menggunakan Metode Usability Testing dan SUS,” *Jambura Journal of Informatics*, vol. 4, no. 1, pp. 14–26, 2022, doi: 10.37905/jji.v4i1.13411.
- [18] “ISO 9241-11:2018 Ergonomics of human-system interaction – Part 11: Usability: Definitions and concepts,” 2018, *Geneva, Switzerland*.
- [19] M. A. Memon, R. Thurasamy, H. Ting, and J.-H. Cheah, “Purposive Sampling: A Review and Guidelines for Quantitative Research,” *Journal of Applied Structural Equation Modeling*, vol. 9, no. 1, pp. 1–23, 2025, doi: 10.47263/JASEM.9(1)01.
- [20] I. Etikan, S. A. Musa, and R. S. Alkassim, “Comparison of Convenience Sampling and Purposive Sampling,” *American Journal of Theoretical and Applied Statistics*, vol. 5, no. 1, pp. 1–4, 2016, doi: 10.11648/j.ajtas.20160501.11.
- [21] A. Morchid, R. Jebabra, H. Qjidaa, R. El Alami, and M. O. Jamil, “Agri-tech innovations for sustainability: A fire detection system based on MQTT broker and IoT to improve environmental risk management,” *Results in Engineering*, vol. 24, Dec. 2024, doi: 10.1016/j.rineng.2024.103683.
- [22] B. Hardika *et al.*, “Pengujian Blackbox Testing Website Garuda Farm Menggunakan Teknik Equivalence Partitioning.”
- [23] S. Sari and O. C. R. Rachmawati, “The Implementation of Agile Kanban in the Development of an IoT-Based Sugarcane Growth Monitoring System,” *G-Tech: Jurnal Teknologi Terapan*, vol. 9, no. 4, pp. 1858–1867, 2025, doi: 10.70609/g-tech.v9i4.7845.
- [24] H. Y. Riskiawan *et al.*, “Artificial Intelligence Enabled Smart Monitoring and Controlling of IoT-Green House,” *Arab J Sci Eng*, 2023, doi: 10.1007/s13369-023-07887-6.
- [25] Maryamah, L. Hasibuan, K. Anwar, and A. F. R. Ahmad, “Sistem Informasi Pendidikan,” *Milenial: Journal for Teachers and Learning*, vol. 2, no. 1, pp. 1–11, 2021, [Online]. Available: <https://ejournal.anotero.org/index.php/milenial>
- [26] A. Hidayat and A. Buana, “SISTEM INFORMASI PERPUSTAKAAN DIGITAL BERBASIS WEB MENGGUNAKAN FRAMEWORK SLIM CENDANA,” *Jurnal Manajemen Informatika (JUMIKA)*, vol. 5, no. 1, 2018, [Online]. Available: <http://jurnal.stmik-dci.ac.id/index.php/jumika/>
- [27] C. Chastro and E. Darmawan H., “Perbandingan Pengembangan Front End Menggunakan Blade Template dan Vue Js,” *Jurnal Strategi*, vol. 2, no. 2, Nov. 2020.
- [28] D. N. Rizkiani, A. Sumadyo, and A. Marlina, “Greenhouse Sebagai Wadah Penelitian Hortikultura Pada Balai Penelitian Dan Pengembangan Tanaman Pangan Di Pematang,” *SENTHONG: Jurnal Ilmiah Mahasiswa Arsitektur*, vol. 3, no. 2, pp. 461–470, Jul. 2020, [Online]. Available: <https://jurnal.ft.uns.ac.id/index.php/senthong/index>
- [29] S. Megawati and A. Lawi, “Pengembangan Sistem Teknologi Internet of Things Yang Perlu Dikembangkan Negara Indonesia,” *JEET: Journal Information Engineering and Educational Technology*, vol. 5, no. 1, 2021.
- [30] Y. Efendi, “INTERNET OF THINGS (IOT) SISTEM PENGENDALIAN LAMPU MENGGUNAKAN RASPBERRY PI BERBASIS MOBILE,” *Jurnal Ilmiah Ilmu Komputer*, vol. 4, no. 1, 2018.

- [31] S. N. Malini and E. Gusmira M.Si, "LITERATUR REVIEW: PENGGUNAAN INTERNET OF THINGS (IOT) DALAM PEMANTAUAN SUHU DAN KELEMBAPAN MENGGUNAKAN SENSOR DHT11," *JSSIT: Jurnal Sains dan Sains Terapan*, vol. 2, no. 2, Nov. 2024.
- [32] F. P. E. Putra, F. Muslim, N. Hasanah, Holipah, R. Paradina, and R. Alim, "Analisis Komparasi Protokol Websocket dan MQTT Dalam Proses Push Notification," *Jurnal Sistim Informasi dan Teknologi*, vol. 5, no. 4, pp. 63–72, Jan. 2023, doi: 10.60083/jsisfotek.v5i4.325.
- [33] M. Diono, A. D. Putri, H. Azwar, and W. Khabzli, "Sistem Monitoring Jaringan Sensor Node Berbasis Protokol MQTT," *Jurnal ELEMENTER*, vol. 7, no. 2, pp. 120–126, Nov. 2021, [Online]. Available: <https://jurnal.pcr.ac.id/index.php/elementer>
- [34] A. Rahman and A. N. Salim, "Sistem Kendali pH dan Kekeruhan Air Aquascape Menggunakan Wemos D1 Mini ESP8266 Berbasis IoT," *Jurnal Teknologi Terpadu*, vol. 8, no. 1, pp. 22–30, 2022.
- [35] G. Yudha Saputra, A. Denhas Afrizal, F. Khusnu Reza Mahfud, F. Angga Pribadi, and F. Jati Pamungkas, "PENERAPAN PROTOKOL MQTT PADA TEKNOLOGI WAN (STUDI KASUS SISTEM PARKIR UNIVERISTAS BRAWIJAYA)," 2017.
- [36] P. Sethi and S. R. Sarangi, "Internet of Things: Architectures, protocols, and applications," *Journal of Electrical and Computer Engineering*, vol. 2017, 2017, doi: 10.1155/2017/9324035.
- [37] M. Javaid, A. Haleem, R. P. Singh, and S. Rab, "Significance of sensors for Industry 4.0: Roles, capabilities, and applications," *Sensors International*, vol. 2, p. 100110, 2021, doi: 10.1016/j.sintl.2021.100110.
- [38] J. Katare, "Agile Marketing as a Key Driver to Increasing Operational Efficiencies and Speed to Market," *International Journal of Business Administration*, vol. 13, no. 2, 2022.
- [39] GeeksforGeeks, "Kanban Agile Methodology," 2024. [Online]. Available: <https://www.geeksforgeeks.org/software-engineering/kanban-agile-methodology/>
- [40] S. AKHBARONA, "PENGEMBANGAN APLIKASI INVENTORY MANAGEMENT BERBASIS WEBSITE DENGAN BARCODE MENGGUNAKAN MODEL PERSONAL EXTREME PROGRAMMING (STUDI KASUS: KEBUN RAYA ITERA)," Institut Teknologi Sumatera (ITERA), Lampung, 2023. [Online]. Available: <https://repo.itera.ac.id/depan/submission/SB2505280052>
- [41] S. J. Putri, D. G. P. Putri, and W. H. N. Putra, "Analisis Komparasi pada Teknik Black Box Testing (Studi Kasus: Website Lars)," *Journal of Internet and Software Engineering (JISE)*, vol. 5, no. 1, p. 23, May 2024.
- [42] J. Brooke, "SUS: A Retrospective," *J Usability Stud*, vol. 8, no. 2, pp. 29–40, Feb. 2013.
- [43] J. Sauro, "5 Ways to Interpret a SUS Score," 2018. [Online]. Available: <https://measuringu.com/interpret-sus-score/>
- [44] A. Aurellia, S. Nooriansyah, and Y. Amrozi, "PEMANFAATAN UML DALAM PERANCANGAN SISTEM INFORMASI PRODUK KREATIF DAUR ULANG SAMPAH BERBASIS WEB," *JITET (Jurnal Informatika dan Teknik Elektro Terapan)*, vol. 13, no. 3S1, pp. 1739–1748, 2025, doi: 10.23960/jitet.v13i3S1.8073.

- [45] dheaprianiblog, “Entity Relationship Model,” Nov. 2015. [Online]. Available: <https://dheaprianiblog.wordpress.com/2015/11/04/entity-relationship-model/>
- [46] R. Appelqvist and O. Örnmyr, “Performance comparison of XHR polling, Long polling, Server sent events and Websockets,” Blekinge Institute of Technology, Faculty of Computing, Karlskrona, Sweden, 2017.
- [47] Laravel, “Laravel Introduction,” 2025. [Online]. Available: <https://laravel.com/docs/12.x/installation>
- [48] Laravel, “Laravel Reverb,” 2025. [Online]. Available: <https://laravel.com/docs/12.x/reverb>
- [49] Laravel, “Laravel Horizon,” 2025. [Online]. Available: <https://laravel.com/docs/12.x/horizon>
- [50] Laravel, “Laravel Notifications,” 2025. [Online]. Available: <https://laravel.com/docs/12.x/notifications>
- [51] A. I. Sanka, M. H. Chowdhury, and R. C. C. Cheung, “Efficient High-Performance FPGA-Redis Hybrid NoSQL Caching System for Blockchain Scalability,” *Comput Commun*, vol. 169, pp. 81–91, 2021, doi: 10.1016/j.comcom.2021.01.017.
- [52] B. Michelson, “Event-Driven Architecture Overview,” Boston, Massachusetts, Feb. 2006. doi: 10.1571/bda2-2-06cc.

LAMPIRAN

A. Dokumen-Dokumen *Capstone*



KEMENTERIAN PENDIDIKAN TINGGI, SAINS,
DAN TEKNOLOGI
INSTITUT TEKNOLOGI SUMATERA
LEMBAGA PENELITIAN DAN PENGABDIAN KEPADA MASYARAKAT
Jalan Terusan Ryacudu Way Hui, Kecamatan Jati Agung, Lampung Selatan 35365
Telepon: (0721) 8030188
Email: ippm@itera.ac.id, Website : <http://ippm.itera.ac.id>

SURAT TUGAS
Nomor: 4809/IT9.2.1/PT.01.05/2025

Kepala Lembaga Penelitian dan Pengabdian kepada Masyarakat Institut Teknologi Sumatera dengan ini memberi tugas kepada:

No	Nama	NIDN/NIM	Program Studi	Keterangan
1.	Zunanik Mufidah, M.Si.	0015059206	Teknik Biosistem	Ketua
2.	Kisna Pertiwi, S.Si. M.Si.	0001059402	Rekayasa Instrumentasi dan Automasi	Anggota
3.	Winda Yulita, M.Cs.	0027079304	Teknik Informatika	Anggota
4.	Gerry Romora Tarigan	122310072	Teknik Biosistem	Mahasiswa
5.	Istifa Zuriyanti	122310002	Teknik Biosistem	Mahasiswa
6.	Amas Muda Luckyto	122310055	Teknik Biosistem	Mahasiswa
7.	Iqbal Darwis	122310019	Teknik Biosistem	Mahasiswa
8.	Cornelius Linux	122140079	Teknik Informatika	Mahasiswa
9.	Muhammad Yusuf	122140193	Teknik Informatika	Mahasiswa
10.	Ade Muharom	122490029	Rekayasa Instrumentasi dan Automasi	Mahasiswa
11.	Muslimin	122490019	Rekayasa Instrumentasi dan Automasi	Mahasiswa

Untuk melaksanakan kegiatan "Penelitian" skema Hilirisasi Riset-Pengujian Model Dan Prototype Dana BOPTN Direktorat Penelitian dan Pengabdian kepada Masyarakat Tahun Anggaran 2025 yang berjudul "Transformasi Digital Pemupukan: Implementasi IoT dalam Sistem Peracik Pupuk Otomatis di Lahan Pertanian", pada tanggal 23 Juli s.d. 9 Desember 2025 di Institut Teknologi Sumatera dan Lampung Timur.

Demikian surat tugas ini dibuat untuk dilaksanakan dengan penuh tanggung jawab.

7 November 2025
Kepala

Muhammad Fatikhul Arif
NIP. 198111152020121002

Tembusan:

1. Wakil Rektor Bidang Akademik dan Kemahasiswaan;
2. Dekan Fakultas Teknologi Industri;
3. Koordinator Prodi Teknik Biosistem;
4. Koordinator Prodi Rekayasa Instrumentasi dan Automasi;
5. Koordinator Prodi Teknik Informatika;
6. Arsip.

(1) Surat Tugas Ibu Zunaik Mufidah, M.Si.


ITERAHERO AGROEDUWISATA

Jl. DUSUN 1 SIDODADI, Desa/Kelurahan Sidodadi, Kec. Semaka,
Kab. Tanggamus, Provinsi Lampung.
Telp. 082131073247, Email : iteraheroeduwisata@gmail.com

FORM C-1
SURAT PERNYATAAN KESANGGUPAN MITRA

Yang bertanda tangan di bawah ini:

- | | |
|-------------------------|--|
| 1. Nama | : Zunanik Mufidah, S.TP., M.Si. |
| 2. Jabatan | : <i>Product Owner</i> |
| 3. Nama Mitra | : ITERAHERO |
| 4. Bidang Usaha Mitra | : Pertanian dan Edukasi |
| 5. Nomor Induk Berusaha | : 3110250108659 |
| 6. Alamat | : Jl. Ronggohadi RT 02, RW 01, Desa Keblan Kulon
Kecamatan Sekaran Kabupaten Lamongan |

Menyatakan bersedia untuk menjadi mitra dalam **penelitian tugas akhir mahasiswa**, guna menerapkan IPTEK dengan tujuan mengembangkan produk/jasa atau target lainnya, dengan anggota mahasiswa terlibat:

- | | |
|-------------------------|---|
| Nama mahasiswa 1 | : Muhammad Yusuf |
| NIM | : 122140193 |
| Rancangan Judul Skripsi | : RANCANG BANGUN SUBSISTEM OTOMATISASI
PERANGKAT IOT BERBASIS WEBSITE (STUDI KASUS :
GREENHOUSE KEBUN RAYA ITERA) |
| Nama mahasiswa 2 | : Cornelius Linux |
| NIM | : 122140079 |
| Rancangan Judul Skripsi | : RANCANG BANGUN SUBSISTEM RUMAH
PEMUPUKAN DAN KENDALI ROBOT BERBASIS
WEBSITE (STUDI KASUS: GREENHOUSE KEBUN
RAYA ITERA) |

Bersama ini pula kami menyatakan dengan sebenarnya bahwa di antara Usaha Kecil/Menengah atau Kelompok dan Pelaksana Kegiatan Program tidak terdapat ikatan kekeluargaan dan usaha dalam wujud apapun juga.

Demikian Surat Pernyataan ini dibuat dengan penuh kesadaran dan tanggung jawab tanpa ada unsur paksaan di dalam pembuatannya untuk dapat digunakan sebagaimana mestinya.

Lampung Selatan, 1 November 2025
Yang membuat pernyataan



Zunanik Mufidah, S.TP., M.Si.
NIP. 1992051520202166

(2) Form C1 Kesanggupan Mitra



KEMENTERIAN PENDIDIKAN DAN KEBUDAYAAN
 INSTITUT TEKNOLOGI SUMATERA
 LEMBAGA PENELITIAN, PENGABDIAN KEPADA MASYARAKAT, DAN
 PENJAMINAN MUTU PENDIDIKAN
 Jalan Terusan Riyasudin, Way Hui, Jati Agung, Lampung Selatan 35365
 Telp. 081386267380, 085896093964, Email: lp3@itera.ac.id
 www.lp3.itera.ac.id

FORM-C2
SURAT PERNYATAAN KESANGGUPAN TIM
SKEMA TUGAS AKHIR KELOMPOK/ CAPSTONE

Yang bertanda tangan di bawah ini:

1. Nama Anggota 1 : Cornelius Linux
2. NIM Anggota 1 : 122140079
3. Rancangan judul skripsi : RANCANG BANGUN SUBSISTEM RUMAH
 PEMUPUKAN DAN KENDALI ROBOT BERBASIS
 WEBSITE (STUDI KASUS: GREENHOUSE
 KEBUN RAYA ITERA)

Timeline Kegiatan Anggota 1

Rencana Kegiatan	November	Desember	Januari
Riset Masalah Lapangan	Tanggal 1 sd. 10		
Penentuan Scope Proyek	Tanggal 11 sd. 25		
Pelaksanaan Proyek		Tanggal 1 s.d 31	
Pelaksanaan Proyek		Tanggal 1 s.d 31	
Pelaksanaan Proyek		Tanggal 1 s.d 31	
Presentasi Proyek			Tanggal 1 s.d. 6
Penyerahan Proyek			Tanggal 1 s.d. 6

1. Nama Anggota 2 : Muhammad Yusuf
2. NIM Anggota 2 : 122140193
3. Rancangan judul skripsi : RANCANG BANGUN SUBSISTEM
 OTOMATISASI PERANGKAT IOT BERBASIS
 WEBSITE (STUDI KASUS : GREENHOUSE
 KEBUN RAYA ITERA)

Timeline Kegiatan Anggota 2

Rencana Kegiatan	November	Desember	Januari
Riset Masalah Lapangan	Tanggal 1 sd. 10		
Penentuan Scope Proyek	Tanggal 11 sd. 25		
Pelaksanaan Proyek		Tanggal 1 s.d 31	
Pelaksanaan Proyek		Tanggal 1 s.d 31	
Pelaksanaan Proyek		Tanggal 1 s.d 31	
Presentasi Proyek			Tanggal 1 s.d. 6
Penyerahan Proyek			Tanggal 1 s.d. 6

(3) (A) Form C2 Kesanggupan TIM



KEMENTERIAN PENDIDIKAN DAN KEBUDAYAAN
 INSTITUT TEKNOLOGI SUMATERA
 LEMBAGA PENELITIAN, PENGABDIAN KEPADA MASYARAKAT, DAN
 PENJAMINAN MUTU PENDIDIKAN
 Jalan Terusan Ryacudu, Way Hui, Jati Agung, Lampung Selatan 35365
 Telp. 081386267380, 085896093964, Email: lp3@itera.ac.id
 www.lp3.itera.ac.id

Kami yang mengisikan pernyataan menyatakan kesanggupan dan berkomitmen untuk menyelesaikan tugas akhir sesuai dengan timeline yang sudah dibuat.

Apabila mahasiswa tidak sanggup menyelesaikan atau melanjutkan sesuai dengan timeline yang diajukan pada semester berjalan maka

- Mahasiswa **harus** mengajukan rekomendasi pengganti untuk menyelesaikan tugas yang dibebankan sebelumnya.
- Mahasiswa tidak diperbolehkan mengambil TA capstone kembali.
- Mahasiswa wajib mengulang kembali TA1 dan atau TA2 di semester berikutnya, mahasiswa diberikan **penilaian E** pada TA1 atau TA2.

Demikian Surat Pernyataan ini dibuat dengan penuh kesadaran dan tanggung jawab tanpa ada unsur paksaan di dalam pembuatannya untuk dapat digunakan sebagaimana mestinya.

Lampung Selatan, 1 November 2025
 Yang membuat pernyataan

Mahasiswa 1

Cornelius Linux
 (NIM. 122140193)

Mahasiswa 2

Muhammad Yusuf
 (NIM. 122140193)

Mengetahui,

Calon Pembimbing 1

Muhammad Habib Algifari, S.Kom., M.T.I.
 Pembimbing 1
 NIP. 199105252022031002

Calon Pembimbing 2

Winda Yulita, M.Cs.
 Pembimbing 2
 NIP. 199307272022032022

Calon Mitra

Zunanik Mufidah, S.TP., M.Si.
 NIP. 1992051520202166

(3) (A) Form C2 Kesanggupan TIM



ITERAHERO AGROEDUWISATA
 JL. DUSUN 1 SIDODADI, Desa/Kelurahan Sidodadi, Kec. Semaka,
 Kab. Tanggamus, Provinsi Lampung.
 Telp. 082131073247, Email : iteraheroeduwisata@gmail.com

FORM-C3 SURAT PERNYATAAN PENYELESAIAN TA

Yang bertanda tangan di bawah ini:

1. Nama : Zunanik Mufidah, S.TP., M.Si.
2. Jabatan : *Product Owner*

Dengan ini menyatakan bahwasanya mengizinkan kepada:

1. Nama mahasiswa 1 : Muhammad Yusuf
2. NIM mahasiswa 1 : 122140193
3. Judul TA Akhir : RANCANG BANGUN SUBSISTEM OTOMATISASI
 PERANGKAT IOT BERBASIS WEBSITE (STUDI KASUS :
 GREENHOUSE KEBUN RAYA ITERA)
1. Nama mahasiswa 2 : Cornelius Linux
2. NIM mahasiswa 2 : 122140079
3. Judul TA Akhir : RANCANG BANGUN SUBSISTEM RUMAH
 PEMUPUKAN DAN KENDALI ROBOT BERBASIS
 WEBSITE (STUDI KASUS: GREENHOUSE KEBUN
 RAYA ITERA)

Untuk dapat mengikuti pelaksanaan sidang akhir yang akan dilaksanakan pada 9 Januari 2026. Mahasiswa tersebut dinyatakan telah siap mengikuti sidang akhir, dikarenakan telah menyelesaikan seluruh kegiatan penelitian dan telah memaparkan hasilnya kepada mitra. Demikian surat pernyataan ini dibuat tanpa dengan sebenar-benarnya dan bisa dipertanggungjawabkan.

Lampung Selatan, 5 Januari 2026
 Yang membuat pernyataan

Zunanik Mufidah, S.TP., M.Si.
 NIP. 1992051520202166

(4) Form C3 Penyelesaian TA

(5) Manual *Book*

(6) Video

(7) Poster

(8) Link Github : <https://github.com/krossmanzs/ITERA-Hero-Web>

B. Hasil Wawancara

No	Daftar Pertanyaan dan Jawaban
1	Bagaimana kendala yang Ibu alami selama mengoperasikan sistem monitoring dan pengendalian greenhouse dalam dua tahun terakhir? Jawaban Ibu Zuna: Iya dek, sebenarnya sistem otomatisasi yang lama itu sudah cukup membantu kami, cuma di lapangan sering terasa agak lambat dan kadang tidak langsung bereaksi begitu kondisi di greenhouse berubah. Bagian-bagian seperti pengaturan suhu, penyiraman, dan pemupukan itu juga masih berjalan sendiri-sendiri, jadi kami harus cek satu-satu, dan itu bikin pekerjaan kurang efisien. Notifikasinya pun sering telat datang atau infonya tidak lengkap, jadinya kami tidak cepat tahu apakah proses otomatisasinya benar-benar berhasil atau malah gagal. Untuk tampilan juga kami berharap bisa dibuat lebih modern dan konsisten, biar data pentingnya langsung terlihat dan lebih mudah dipahami staf di lapangan. Selain itu dek, ke depannya kami ada rencana pakai robot keliling untuk cek kondisi tanaman, jadi akan sangat membantu kalau subsistem yang baru ini sudah disiapkan supaya bisa tersambung dengan robot itu dan bisa pakai rekomendasinya untuk mengatur perangkat di greenhouse/rumah pemupukan.
2	Dalam kondisi apa perangkat yang ada di greenhouse seharusnya menyala atau mati secara otomatis? Misalnya ketika suhu terlalu tinggi atau air habis. Jawaban Ibu Zuna: Kalau suhu sudah melewati batas tertentu, misalnya lebih dari 31 derajat, kipas dan cooling pad itu harus menyala sendiri. Kalau suhu sudah kembali normal, ya harus mati sendiri juga. Begitu juga dengan kelembapan, kalau rendah harus otomatis dinaikkan.
3	Apakah ada kegiatan rutin seperti penyiraman, pengadukan nutrisi, atau pendinginan yang sebaiknya berjalan otomatis berdasarkan waktu tertentu? Kapan saja jadwal yang Ibu perlukan? Jawaban Ibu Zuna: Beberapa alat itu perlu dijalankan berdasarkan jadwal. Misalnya pompa untuk penyiraman pagi dan sore. Atau pengadukan tandon setiap hari jam tertentu. Saya ingin pengaturannya mudah. Saya tidak mau harus minta programmer setiap kali ada perubahan jadwal.
4	Apakah sistem yang sebelumnya sudah mendukung otomatisasi jadwal atau hanya monitoring? Apa kelemahan yang paling terasa? Jawaban Ibu Zuna: Kadang jadwal penyiraman telat atau tidak jalan, terutama kalau server lagi berat. Jadi sistemnya tidak bisa diandalkan kalau waktunya pas-pasan. Ini sangat merugikan karena tanaman butuh konsistensi.
5	Saya dengar greenhouse berencana menggunakan robot pemantau tanaman. Informasi apa saja yang perlu dikirim robot ke sistem agar bisa membantu otomatisasi, misalnya rekomendasi nutrisi atau kondisi tanaman? Jawaban Ibu Zuna: Kami juga ada rencana pakai robot keliling. Kalau robot melihat ada tanaman yang kuning atau kurang nutrisi, sistem harus bisa menerima

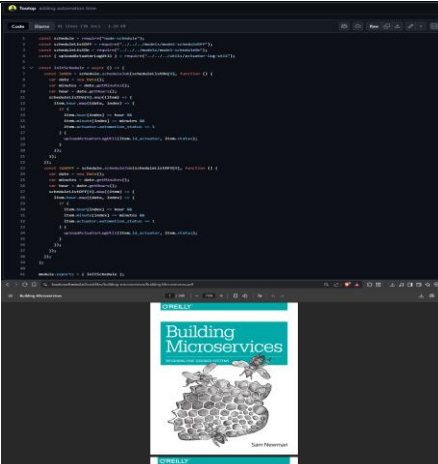
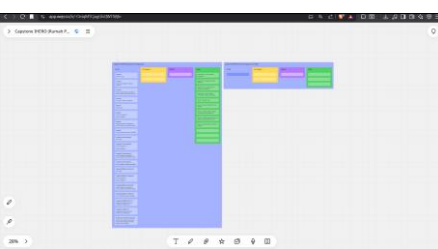
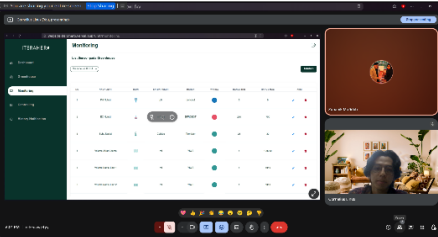
	informasi itu dan menyesuaikan dosis nutrisi otomatis. Saya ingin sistem ini bisa terhubung dengan robot, bukan robot berjalan sendiri tanpa koneksi. Untuk rencana data yang dikumpulkan itu adalah kategori tanaman (sehat, kurang sehat, dan sakit), lalu fase dari tanaman tersebut masuk ke vegetatif atau generatif, lalu ada nilai konsentrasi nutrisi ppm, dan rekomendasi volume tanaman si dek.
6	Informasi apa saja yang Ibu ingin dapatkan sebagai notifikasi? Apakah perlu ringkasan otomatisasi harian atau peringatan kalau ada kegagalan? Jawaban Ibu Zuna: Saya ingin ada notifikasi kalau otomatisasi berhasil atau gagal. Kalau pagi ini penyiraman tidak jalan, saya harus tahu supaya bisa segera diperbaiki. Notifikasi juga harus muncul kalau nilai sensor keluar batas, atau stok nutrisi mau habis.
7	Oke Ibu, apakah ada tambahan lainnya? Jawaban Ibu Zuna: Oiya dek, 1 lagi saya ingin tahu perangkat mana yang nyala, kapan nyala, dan siapa yang mengubah aturannya. Ini penting untuk memastikan semuanya berjalan benar. Sama tampilan si paling dek, kami sering memantau lewat HP sambil keliling. Tampilan di HP harus enak dilihat, jangan besar-besar dan jangan keluar layar bisa discroll seperti yang lama.

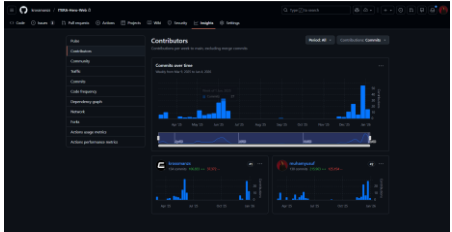
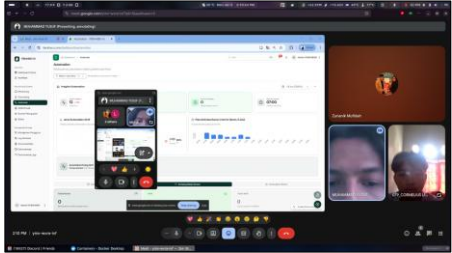
Tertanda Tangan,
Lampung Selatan, Rabu, 19-11-2025



Zunanik Mufidah, S.TP., M.Si.

C. Daftar Bukti Dokumentasi Penelitian *Capstone*

Bukti Foto	Detail Informasi
	Wawancara dengan Ibu Zunanik Mufidah, S.TP., M.Si. selaku penanggung jawab greenhouse Kebun Raya ITERA Pelaksanaan wawancara mendalam dengan pemangku kepentingan utama guna mengidentifikasi kendala operasional serta memetakan kebutuhan fungsional sistem otomatisasi greenhouse. Timestamp: 18-11-2025 Pukul 17:00-18:00 (GKU1 ITERA)
	Studi Literatur dan Evaluasi Kode Sumber Lama Penelaahan referensi teknis serta audit struktur kode eksisting (<i>legacy code</i>) untuk mendeteksi batasan performa dan merumuskan strategi pengembangan subsistem yang lebih efisien. Timestamp: 18-11-2025 Pukul 20:00 - 23:00
	Perencanaan Manajemen Proyek Berbasis <i>Agile Kanban</i> Penyusunan <i>backlog</i> pengembangan dan visualisasi alur kerja menggunakan metode <i>Kanban</i> untuk menjamin efektivitas pemantauan progres dan manajemen tugas tim. Timestamp: 20-11-2025 Pukul 7:00 – 15:00
	Validasi Kebutuhan dan Reviu Rancangan Sistem Diskusi teknis untuk memvalidasi spesifikasi kebutuhan serta mengevaluasi artefak perancangan sistem (<i>Use Case</i>, <i>Activity Diagram</i>, ERD, dan <i>Wireframe</i>) agar selaras dengan logika bisnis.

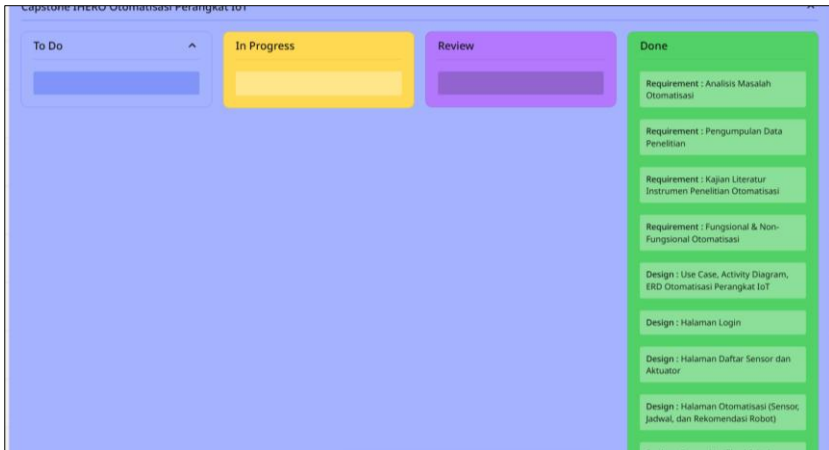
Bukti Foto	Detail Informasi
	Timestamp: 20-11-2025 Pukul 17:00 – 18:00 Observasi Lapangan dan Verifikasi Perangkat Keras Peninjauan langsung infrastruktur fisik <i>greenhouse</i> guna memastikan kompatibilitas rancangan subsistem dengan spesifikasi sensor, aktuator, dan instrumen yang tersedia di lapangan. Timestamp: 23-11-2025 Pukul 17:30 – 18:00
	Implementasi Kode dan Pengembangan Fitur Subsistem Tahapan konstruksi perangkat lunak meliputi penulisan kode <i>backend</i> untuk logika otomatisasi, pengembangan antarmuka <i>frontend</i>, serta integrasi basis data sesuai arsitektur yang dirancang. Timestamp: 27-11-2025 sd. 31-12-2025
	Pengujian Fungsional Otomatisasi Aktuator Uji coba mekanisme kontrol otomatis pada perangkat (<i>Cooling Pad</i> dan <i>Agitator</i> Nutrisi) untuk memverifikasi akurasi respons sistem terhadap kondisi lingkungan. Timestamp: 01-01-2026 Pukul 13:30
	Review Pertama Product Setelah Deploy Pelaksanaan tinjauan langsung terhadap sistem yang telah diunggah ke server produksi guna memvalidasi kesesuaian fitur dengan kebutuhan operasional serta menginventarisasi catatan perbaikan (<i>feedback</i>) dari pengguna sebelum finalisasi. Timestamp 5 Januari 2026 Pukul 13:30

Bukti Foto	Detail Informasi
	Pengujian Validasi, <i>Usability</i>, dan Pelatihan Pengguna Pelaksanaan pengujian Black Box dan evaluasi usability serta sesi pendampingan operasional (tutorial) kepada staf/pengelola untuk menjamin keberterimaan sistem. Timestamp: 6 Januari 2026 Pukul 19:30
	Review kedua dan Serah Terima <i>project Capstone</i> kepada Ibu Zuna selaku <i>product owner</i> Penyerahan resmi aset digital, kode sumber, dan hak akses sistem kepada Ibu Zunanik Mufidah selaku representasi pengelola <i>greenhouse</i>. Timestamp: 05-01-2026 Pukul 19:30
	Penyusunan Dokumentasi Akhir Proyek Kompilasi laporan teknis, buku panduan manual, dan dokumen administratif proyek sebagai kelengkapan syarat penyelesaian <i>Capstone</i>. Timestamp: 07-01-2026 Pukul 07:00

D. Rincian Proses Agile Kanban

No.	Nama Fitur dan Proses Penelitian	<i>To Do</i>	<i>In Progress</i>	<i>Review</i>	<i>Done</i>
1.	Requirement : Analisis Masalah Otomatisasi	✓	✓	✓	✓
2.	Requirement : Pengumpulan Data Penelitian	✓	✓	✓	✓
3.	Requirement : Kajian Literatur Instrumen Penelitian Otomatisasi Perangkat IoT	✓	✓	✓	✓
4.	Requirement : Fungsional & Non-Fungsional Otomatisasi Perangkat IoT	✓	✓	✓	✓
5.	Design : Use Case, Activity Diagram, ERD Otomatisasi Perangkat IoT	✓	✓	✓	✓
6.	Design : Wireframe Halaman Login	✓	✓	✓	✓
7.	Design : Wireframe Halaman Perangkat IoT (Sensor dan Aktuator)	✓	✓	✓	✓
8.	Design : Wireframe Halaman Otomatisasi Perangkat IoT (Sensor, Jadwal, Rekomendasi Robot)	✓	✓	✓	✓
9.	Design : Wireframe Tooltip dan Halaman Notifikasi	✓	✓	✓	✓
10.	Frontend : Halaman Login	✓	✓	✓	✓
11.	Frontend : Halaman Perangkat IoT (Sensor, Aktuator)	✓	✓	✓	✓
12.	Frontend : Halaman Otomatisasi Perangkat IoT (Sensor, Jadwal, Rekomendasi Robot)	✓	✓	✓	✓
13.	Frontend : Tooltip dan Halaman Notifikasi	✓	✓	✓	✓
14.	Backend : Fungsi Login	✓	✓	✓	✓
15.	Backend : Fungsi Perangkat IoT (Sensor, Aktuator)	✓	✓	✓	✓
16.	Backend : Fungsi Otomatisasi Perangkat IoT (Sensor, Jadwal, Rekomendasi Robot)	✓	✓	✓	✓
17.	Backend : Fungsi Tooltip dan Halaman Notifikasi	✓	✓	✓	✓
18.	Integrasi Frontend Backend : Fitur Login	✓	✓	✓	✓

No.	Nama Fitur dan Proses Penelitian	To Do	In Progress	Review	Done
19.	Integrasi Frontend Backend : Fitur Perangkat IoT (Sensor, Aktuator)	✓	✓	✓	✓
20.	Integrasi Frontend Backend : Fitur Otomatisasi Perangkat IoT (Sensor, Jadwal, Rekomendasi Robot)	✓	✓	✓	✓
21.	Integrasi Frontend Backend : Fitur Tooltip dan Halaman Notifikasi	✓	✓	✓	✓
22.	Pengujian Blackbox: Fitur Login	✓	✓	✓	✓
23.	Pengujian Blackbox: Fitur Perangkat IoT (Sensor, Aktuator)	✓	✓	✓	✓
24.	Pengujian Blackbox: Fitur Otomatisasi Perangkat IoT (Sensor, Jadwal, Rekomendasi Robot)	✓	✓	✓	✓
25.	Pengujian Blackbox: Fitur Tooltip dan Halaman Notifikasi	✓	✓	✓	✓
26.	Pengujian Usability SUS : Pengujian menyeluruh ke fitur Otomatisasi Perangkat IoT	✓	✓	✓	✓
27.	Analisis dan Penarikan Kesimpulan : Analisis dan Penarikan Kesimpulan Seluruh Hasil Pengujian Blackbox, Responsif, dan Usability	✓	✓	✓	✓
28.	Perbaikan Otomatisasi Jadwal	✓	✓	✓	✓
29.	Perbaikan Otomatisasi Sensor	✓	✓	✓	✓
30.	Perbaikan Otomatisasi Robot	✓	✓	✓	✓
31.	Perbaikan Notifikasi	✓	✓	✓	✓
32.	Perbaikan Log Aktivitas	✓	✓	✓	✓
Total Fitur dan Proses Penelitian (%)		100%	100%	100%	100%



Link Kanban Board : <https://s.id/XvO&t>

E. *Stakeholder yang memiliki akses*

Nama	Status/Jabatan	Program Studi	Hak Akses Sistem
Zunanik Mufidah, S.TP., M.Si.	Dosen	Teknik Biosistem	Admin
Dr. Suratun Nafisah, M.Sc.	Dosen	Teknik Elektro	Pengelola
Kisna Pertiwi, S.Si., M.Si.	Dosen	Teknik Biosistem	Pengelola
Raizummi Fil'aini, S.TP., M.Si.	Dosen	Teknik Biosistem	Pengelola
M Gilang Indra Mardika, M.T.	Dosen	Teknik Sipil	Pengelola
Alif Renaldi Pratama	Mahasiswa	Teknik Elektro	Pengelola
Alfin Mahmud	Mahasiswa	Teknik Elektro	Pengelola
Gerry Romora Tarigan	Mahasiswa	Teknik Biosistem	Pengelola
Amas Muda Luckyto Tamba	Mahasiswa	Teknik Biosistem	Pengelola
Iqbal Darwis	Mahasiswa	Teknik Biosistem	Pengelola
Istifa Zuriyanti	Mahasiswa	Teknik Biosistem	Pengelola
Cornelius Linux	Mahasiswa	Teknik Informatika	Pengelola
Muslimin	Mahasiswa	Rekayasa Instrumentasi Automasi	Pengelola
Muhammad Farhan Yusuf Almajid	Mahasiswa	Teknik Elektro	Staff
Rolas Agusta Nugrahadi	Mahasiswa	Teknik Elektro	Staff
Akbar Tri Agung	Mahasiswa	Teknik Elektro	Staff
Nadya Shafwah	Mahasiswa	Teknik Informatika	Staff
Louis Hutabarat	Mahasiswa	Teknik Informatika	Staff
Martino Kelvin	Mahasiswa	Teknik Informatika	Staff
Samuel Cornelius Evaldo	Mahasiswa	Rekayasa Instrumentasi Automasi	Staff
Dwifachruzi Al Jariah	Mahasiswa	Rekayasa Instrumentasi Automasi	Staff
Muhammad Zaki Irfandi	Mahasiswa	Rekayasa Instrumentasi Automasi	Staff
Sherly Anggraeni	Mahasiswa	Teknik Biosistem	Staff
Isnaton Kurniawati	Mahasiswa	Teknik Biosistem	Staff
Syahra Nurhafizah	Mahasiswa	Teknik Biosistem	Staff
Raihan Haikal Fikri	Mahasiswa	Teknik Sipil	Staff
Elsa Davina	Mahasiswa	Teknik Sipil	Staff
Ridha Nurfaizi	Mahasiswa	Teknik Sipil	Staff
Cris Natalia Damanik	Mahasiswa	Teknik Biosistem	Staff
Asisi Br Bangun	Mahasiswa	Teknik Biosistem	Staff
Alzahra Mulyana	Mahasiswa	Teknik Biosistem	Staff
Nurlis Nurlita	Mahasiswa	Teknik Biosistem	Staff
M Arif Nadhir	Mahasiswa	Teknik Biosistem	Staff

Nama	Status/Jabatan	Program Studi	Hak Akses Sistem
Fakih Qathi Alfarizi	Mahasiswa	Teknik Biosistem	Staff
Eci Erin Br Purba	Mahasiswa	Teknik Biosistem	Staff
Nadila Destarina	Mahasiswa	Teknik Biosistem	Staff
Martabe Hasudungan Sagala	Mahasiswa	Teknik Biosistem	Staff
Yeni Anastasia Gultom	Mahasiswa	Teknik Biosistem	Staff
M. Naufal Azmi Ulwan	Mahasiswa	Teknik Biosistem	Staff
Novenri Tumiur Pasaribu	Mahasiswa	Teknik Biosistem	Staff
Rifki Mualidi	Mahasiswa	Teknik Biosistem	Staff
Rodo Winata Sihaloho	Mahasiswa	Teknik Biosistem	Staff
Nia Carolina Lubis	Mahasiswa	Teknik Biosistem	Staff
Nayla Azzahra A	Mahasiswa	Teknik Biosistem	Staff
Total Staff			30
Total Pengelola			12
Total Admin			1
Total Pengguna Memiliki Akses			43

F. Rincian Pengujian *Blackbox Testing*

No	Kode	Fitur	Skenario	Ekspektasi	Hasil
1	LOGIN-001	Login	Pengguna memasukkan email dan password Admin yang benar (admin@iterahero.com, Admin@123)	Sistem menerima kredensial dan pengguna berhasil login	Berhasil
2	LOGIN-002	Login	Pengguna memasukkan email Admin yang benar dengan password yang salah	Sistem menolak kredensial dan menampilkan pesan kesalahan	Berhasil
3	LOGIN-003	Login	Pengguna memasukkan email yang tidak terdaftar dalam sistem	Sistem menolak kredensial dan menampilkan pesan kesalahan	Berhasil
4	LOGIN-004	Login	Pengguna memasukkan email dan password Pengelola yang benar (pengelola@iterahero.com, Pengelola@123)	Sistem menerima kredensial dan pengguna berhasil login	Berhasil
5	LOGIN-005	Login	Pengguna memasukkan email Pengelola yang benar dengan password yang salah	Sistem menolak kredensial dan menampilkan pesan kesalahan	Berhasil
6	LOGIN-006	Login	Pengguna memasukkan email dan password Staff yang benar (staff@iterahero.com, Staff@123)	Sistem menerima kredensial dan pengguna berhasil login	Berhasil
7	LOGIN-007	Login	Pengguna memasukkan email Staff yang benar dengan password yang salah	Sistem menolak kredensial dan menampilkan pesan kesalahan	Berhasil
8	LOGIN-008	Login	Pengguna mencoba login dengan field email dikosongkan	Sistem menolak permintaan dan tidak memproses login	Berhasil
9	LOGIN-009	Login	Pengguna mencoba login dengan field password dikosongkan	Sistem menolak permintaan dan tidak memproses login	Berhasil
10	SENSOR-001	Tambah Sensor	Pengguna menambahkan sensor baru dengan data lengkap (nama, tipe, satuan, rumus kalibrasi, lokasi)	Sistem menyimpan data sensor baru dan membuat kode perangkat serta topik MQTT secara otomatis	Berhasil
11	SENSOR-002	Baca Sensor	Pengguna mengakses daftar sensor yang tersedia dalam sistem	Sistem menampilkan seluruh data sensor yang tersimpan	Berhasil
12	SENSOR-003	Edit Sensor	Pengguna mengubah nama sensor yang sudah ada	Sistem menyimpan perubahan nama sensor dengan benar	Berhasil
13	SENSOR-004	Tambah Sensor	Sistem membuat kode perangkat secara otomatis saat sensor ditambahkan	Kode perangkat ter-generate secara otomatis dengan format yang sesuai	Berhasil
14	SENSOR-005	Tambah Sensor	Sistem membuat topik MQTT secara otomatis saat sensor ditambahkan	Topik MQTT ter-generate sesuai dengan lokasi dan kode perangkat	Berhasil

No	Kode	Fitur	Skenario	Ekspektasi	Hasil
15	SENSOR-006	Tambah Sensor	Pengguna mencoba menambahkan sensor dengan nama kosong	Sistem menolak permintaan dan menampilkan pesan validasi	Berhasil
16	SENSOR-007	Hapus Sensor	Pengguna menghapus sensor yang sudah ada	Sensor berhasil dihapus dari sistem	Berhasil
17	SENSOR-008	Tambah Sensor	Sistem membuat kode perangkat yang unik untuk setiap sensor baru	Setiap sensor memiliki kode perangkat yang berbeda	Berhasil
18	SENSOR-009	Nilai Sensor via MQTT	Sistem menerima data sensor melalui MQTT dengan nilai 25.5	Nilai terakhir sensor terupdate menjadi 25.5	Berhasil
19	SENSOR-010	Nilai Sensor via MQTT	Sistem menerima data sensor kedua melalui MQTT dengan nilai 30.2	Nilai terakhir sensor terupdate menjadi 30.2	Berhasil
20	SENSOR-011	Riwayat Sensor	Sistem menyimpan setiap data yang diterima sebagai riwayat	Riwayat sensor bertambah sesuai jumlah data yang diterima	Berhasil
21	SENSOR-012	Filter Sensor	Pengguna mem-filter sensor berdasarkan tipe sensor (Temperature)	Sistem menampilkan sensor yang sesuai dengan tipe yang dipilih	Berhasil
22	SENSOR-013	Filter Sensor	Pengguna mem-filter sensor berdasarkan lokasi greenhouse	Sistem menampilkan sensor yang berada di greenhouse yang dipilih	Berhasil
23	SENSOR-014	Filter Sensor	Pengguna mem-filter sensor berdasarkan status online	Sistem menampilkan sensor dengan status online	Berhasil
24	SENSOR-015	Filter Sensor	Pengguna mem-filter sensor berdasarkan status offline	Sistem menampilkan sensor dengan status offline	Berhasil
25	SENSOR-016	Filter Tanggal	Pengguna mem-filter riwayat sensor berdasarkan tanggal tertentu	Sistem menampilkan riwayat sensor pada tanggal yang dipilih	Berhasil
26	SENSOR-017	Filter Tanggal	Pengguna mem-filter riwayat sensor berdasarkan rentang tanggal (7 hari terakhir)	Sistem menampilkan riwayat sensor dalam rentang tanggal yang dipilih	Berhasil
27	SENSOR-018	Halaman Detail	Pengguna mengakses halaman detail sensor tertentu	Sistem menampilkan informasi lengkap sensor termasuk tipe, satuan, dan lokasi	Berhasil
28	ACTUATOR-001	Tambah Aktuator	Pengguna menambahkan aktuator baru dengan data lengkap (nama, tipe, lokasi)	Sistem menyimpan data aktuator baru dengan kode perangkat dan topik MQTT otomatis	Berhasil
29	ACTUATOR-002	Baca Aktuator	Pengguna mengakses daftar aktuator yang tersedia dalam sistem	Sistem menampilkan seluruh data aktuator yang tersimpan	Berhasil
30	ACTUATOR-003	Edit Aktuator	Pengguna mengubah nama aktuator yang sudah ada	Sistem menyimpan perubahan nama aktuator dengan benar	Berhasil
31	ACTUATOR-004	Tambah Aktuator	Sistem membuat kode perangkat secara otomatis saat aktuator ditambahkan	Kode perangkat ter-generate secara otomatis	Berhasil

No	Kode	Fitur	Skenario	Ekspektasi	Hasil
32	ACTUATOR-005	Tambah Aktuator	Sistem membuat topik MQTT secara otomatis saat aktuator ditambahkan	Topik MQTT ter-generate sesuai dengan lokasi dan kode perangkat	Berhasil
33	ACTUATOR-006	Hapus Aktuator	Pengguna menghapus aktuator yang sudah ada	Aktuator berhasil dihapus dari sistem	Berhasil
34	ACTUATOR-007	Tambah Aktuator	Sistem membuat kode perangkat yang unik untuk setiap aktuator baru	Setiap aktuator memiliki kode perangkat yang berbeda	Berhasil
35	ACTUATOR-008	Toggle Manual ON	Pengguna mengaktifkan aktuator secara manual melalui toggle	Status aktuator berubah menjadi aktif (ON)	Berhasil
36	ACTUATOR-009	Toggle Manual OFF	Pengguna menonaktifkan aktuator secara manual melalui toggle	Status aktuator berubah menjadi nonaktif (OFF)	Berhasil
37	ACTUATOR-010	Log Aktuator	Sistem mencatat setiap perubahan status aktuator	Log aktivitas aktuator tersimpan dengan benar	Berhasil
38	ACTUATOR-011	Filter Log Tanggal	Pengguna mem-filter log aktuator berdasarkan tanggal	Sistem menampilkan log aktuator pada tanggal yang dipilih	Berhasil
39	ACTUATOR-012	Filter Aktuator	Pengguna mem-filter aktuator berdasarkan tipe (Cooling Pad)	Sistem menampilkan aktuator yang sesuai dengan tipe yang dipilih	Berhasil
40	ACTUATOR-013	Filter Aktuator	Pengguna mem-filter aktuator berdasarkan status aktif	Sistem menampilkan aktuator dengan status aktif	Berhasil
41	ACTUATOR-014	Filter Aktuator	Pengguna mem-filter aktuator berdasarkan status nonaktif	Sistem menampilkan aktuator dengan status nonaktif	Berhasil
42	ACTUATOR-015	Halaman Detail	Pengguna mengakses halaman detail aktuator tertentu	Sistem menampilkan informasi lengkap aktuator termasuk tipe	Berhasil
43	AUTOPAGE-001	Insight Otomatisasi	Sistem menampilkan jumlah otomatisasi jadwal	Jumlah otomatisasi jadwal ditampilkan dengan benar	Berhasil
44	AUTOPAGE-002	Insight Otomatisasi	Sistem menampilkan jumlah otomatisasi sensor	Jumlah otomatisasi sensor ditampilkan dengan benar	Berhasil
45	AUTOPAGE-003	Insight Otomatisasi	Sistem menampilkan jumlah otomatisasi robot	Jumlah otomatisasi robot ditampilkan dengan benar	Berhasil
46	AUTOPAGE-004	Daftar Otomatisasi	Pengguna mengakses daftar seluruh otomatisasi	Sistem menampilkan daftar otomatisasi dari semua kategori	Berhasil
47	AUTOPAGE-005	Insight Otomatisasi	Sistem menampilkan jumlah otomatisasi yang aktif	Jumlah otomatisasi aktif ditampilkan dengan benar	Berhasil
48	AUTOPAGE-006	Riwayat Eksekusi	Sistem menyimpan riwayat eksekusi otomatisasi	Riwayat eksekusi dapat diakses dan ditampilkan	Berhasil
49	AUTOPAGE-007	Filter Tanggal	Pengguna mem-filter riwayat eksekusi berdasarkan tanggal	Sistem menampilkan riwayat eksekusi pada tanggal yang dipilih	Berhasil
50	AUTOPAGE-008	Filter Tipe	Pengguna mem-filter riwayat eksekusi berdasarkan tipe otomatisasi	Sistem menampilkan riwayat eksekusi sesuai tipe yang dipilih	Berhasil

No	Kode	Fitur	Skenario	Ekspektasi	Hasil
51	SCHEDULE-001	Tambah Jadwal	Pengguna menambahkan jadwal otomatisasi baru dengan data lengkap (aktuator, durasi, waktu mulai, hari)	Sistem menyimpan jadwal otomatisasi baru	Berhasil
52	SCHEDULE-002	Baca Jadwal	Pengguna mengakses detail jadwal otomatisasi	Sistem menampilkan data jadwal dengan benar	Berhasil
53	SCHEDULE-003	Edit Jadwal	Pengguna mengubah durasi dan waktu mulai jadwal	Sistem menyimpan perubahan jadwal dengan benar	Berhasil
54	SCHEDULE-004	Tambah Jadwal	Sistem menyimpan array hari dengan benar	Array hari tersimpan sesuai yang di-input	Berhasil
55	SCHEDULE-005	Edit Jadwal	Pengguna mengatur iterasi dan interval jadwal	Sistem menyimpan pengaturan iterasi dengan benar	Berhasil
56	SCHEDULE-006	Nyalakan Otomatisasi	Pengguna mengaktifkan jadwal otomatisasi	Status jadwal berubah menjadi aktif	Berhasil
57	SCHEDULE-007	Matikan Otomatisasi	Pengguna menonaktifkan jadwal otomatisasi	Status jadwal berubah menjadi nonaktif	Berhasil
58	SCHEDULE-008	Hapus Jadwal	Pengguna menghapus jadwal otomatisasi	Jadwal berhasil dihapus dari sistem	Berhasil
59	SCHEDULE-009	Validasi Aktuator	Pengguna mencoba membuat jadwal tanpa memilih aktuator	Sistem menolak permintaan dan menampilkan pesan validasi	Berhasil
60	SCHEDULE-010	Validasi Durasi	Pengguna mencoba membuat jadwal tanpa mengisi durasi	Sistem menolak permintaan dan menampilkan pesan validasi	Berhasil
61	AUTOSENSOR-001	Tambah Otomasi Sensor	Pengguna menambahkan aturan otomatisasi sensor dengan data lengkap (sensor, aktuator, kondisi, nilai, aksi)	Sistem menyimpan aturan otomatisasi baru	Berhasil
62	AUTOSENSOR-002	Baca Otomasi Sensor	Pengguna mengakses detail aturan otomatisasi sensor	Sistem menampilkan data aturan dengan benar	Berhasil
63	AUTOSENSOR-003	Edit Otomasi Sensor	Pengguna mengubah kondisi, nilai ambang, dan aksi aturan	Sistem menyimpan perubahan dengan benar	Berhasil
64	AUTOSENSOR-004	Kondisi Operator	Sistem mendukung berbagai operator kondisi ($>$, $>=$, $<$, $<=$, $=$, $!=$)	Semua operator dapat disimpan dan digunakan	Berhasil
65	AUTOSENSOR-005	Nyalakan Otomasi Sensor	Pengguna mengaktifkan aturan otomatisasi sensor	Status aturan berubah menjadi aktif	Berhasil
66	AUTOSENSOR-006	Matikan Otomasi Sensor	Pengguna menonaktifkan aturan otomatisasi sensor	Status aturan berubah menjadi nonaktif	Berhasil
67	AUTOSENSOR-007	Hapus Otomasi Sensor	Pengguna menghapus aturan otomatisasi sensor	Aturan berhasil dihapus dari sistem	Berhasil

No	Kode	Fitur	Skenario	Ekspektasi	Hasil
68	AUTOSENSOR-008	Validasi Sensor	Pengguna mencoba membuat aturan tanpa memilih sensor	Sistem menolak permintaan dan menampilkan pesan validasi	Berhasil
69	AUTOSENSOR-009	Validasi Aktuator	Pengguna mencoba membuat aturan tanpa memilih aktuator	Sistem menolak permintaan dan menampilkan pesan validasi	Berhasil
70	AUTOROBOT-001	Tambah Otomasi Robot	Pengguna menambahkan aturan otomatisasi robot dengan data lengkap (robot, definisi data, aktuator, kondisi, nilai, aksi)	Sistem menyimpan aturan otomatisasi baru	Berhasil
71	AUTOROBOT-002	Baca Otomasi Robot	Pengguna mengakses detail aturan otomatisasi robot	Sistem menampilkan data aturan dengan benar	Berhasil
72	AUTOROBOT-003	Edit Otomasi Robot	Pengguna mengubah kondisi, nilai, dan aksi aturan	Sistem menyimpan perubahan dengan benar	Berhasil
73	AUTOROBOT-004	Kondisi Operator	Sistem mendukung operator untuk tipe string (==, !=, contains, startsWith, endsWith)	Semua operator dapat disimpan dan digunakan	Berhasil
74	AUTOROBOT-005	Nyalakan Otomasi Robot	Pengguna mengaktifkan aturan otomatisasi robot	Status aturan berubah menjadi aktif	Berhasil
75	AUTOROBOT-006	Matikan Otomasi Robot	Pengguna menonaktifkan aturan otomatisasi robot	Status aturan berubah menjadi nonaktif	Berhasil
76	AUTOROBOT-007	Durasi dan Interval	Pengguna mengatur durasi, interval, dan iterasi untuk aturan	Sistem menyimpan pengaturan dengan benar	Berhasil
77	AUTOROBOT-008	Hapus Otomasi Robot	Pengguna menghapus aturan otomatisasi robot	Aturan berhasil dihapus dari sistem	Berhasil
78	AUTOROBOT-009	Validasi Robot	Pengguna mencoba membuat aturan tanpa memilih robot	Sistem menolak permintaan dan menampilkan pesan validasi	Berhasil
79	AUTOROBOT-010	Validasi Data Def	Pengguna mencoba membuat aturan tanpa memilih definisi data	Sistem menolak permintaan dan menampilkan pesan validasi	Berhasil
80	NOTIF-001	Buat Notifikasi	Sistem membuat notifikasi baru untuk pengguna	Notifikasi berhasil dibuat dan tersimpan	Berhasil
81	NOTIF-002	Baca Notifikasi	Pengguna mengakses daftar notifikasi	Sistem menampilkan daftar notifikasi terbaru	Berhasil
82	NOTIF-003	Struktur Data	Notifikasi memiliki struktur data yang lengkap (judul, deskripsi)	Struktur data notifikasi sesuai standar	Berhasil
83	NOTIF-004	Tandai Dibaca	Pengguna menandai notifikasi sebagai telah dibaca	Status notifikasi berubah menjadi terbaca	Berhasil
84	NOTIF-005	Filter Tipe	Sistem mengkategorikan notifikasi berdasarkan tipe	Notifikasi terkategorisasi dengan benar	Berhasil

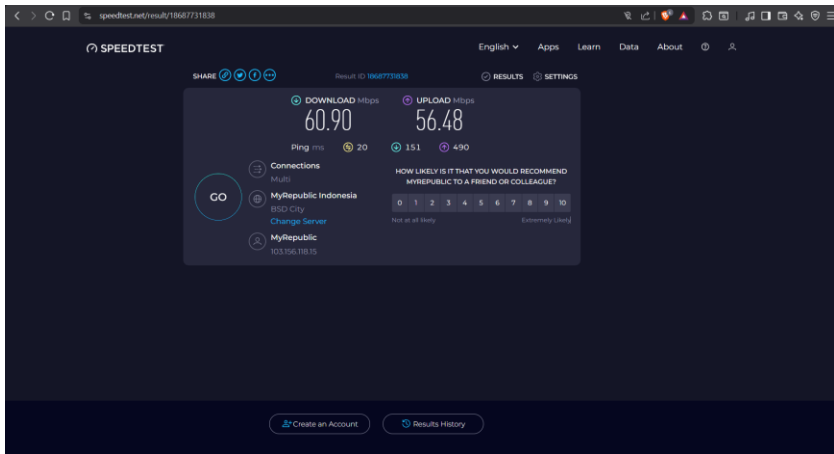
No	Kode	Fitur	Skenario	Ekspektasi	Hasil
			(info, danger, warning, success)		
85	NOTIF-006	Notif Alert Sensor	Sistem mengirimkan notifikasi saat sensor melebihi ambang batas	Notifikasi tipe danger berhasil dibuat	Berhasil
86	NOTIF-007	Notif Otomasi	Sistem mengirimkan notifikasi saat otomatisasi berhasil dijalankan	Notifikasi tipe success berhasil dibuat	Berhasil
87	NOTIF-008	Hapus Notifikasi	Pengguna menghapus notifikasi	Notifikasi berhasil dihapus dari sistem	Berhasil
88	LOG-001	Buat Log	Sistem mencatat aktivitas pengguna ke dalam log	Log aktivitas berhasil dibuat	Berhasil
89	LOG-002	Baca Log	Admin mengakses detail log aktivitas	Sistem menampilkan data log dengan benar	Berhasil
90	LOG-003	Total Log	Sistem menyimpan seluruh log aktivitas	Total log dapat dihitung dan ditampilkan	Berhasil
91	LOG-004	Filter Aksi	Admin mem-filter log berdasarkan jenis aksi (create, update, delete, login, automation, alert)	Sistem menampilkan log sesuai aksi yang dipilih	Berhasil
92	LOG-005	Filter Entitas	Admin mem-filter log berdasarkan tipe entitas (Sensor, Actuator, AutomationSchedule)	Sistem menampilkan log sesuai entitas yang dipilih	Berhasil
93	LOG-006	Filter Kategori	Admin mem-filter log berdasarkan kategori (automation, alert, auth, control, crud)	Sistem menampilkan log sesuai kategori yang dipilih	Berhasil
94	LOG-007	Filter Severity	Admin mem-filter log berdasarkan tingkat severity (info, warning, critical)	Sistem menampilkan log sesuai severity yang dipilih	Berhasil
95	LOG-008	Filter Tanggal	Admin mem-filter log berdasarkan tanggal tertentu	Sistem menampilkan log pada tanggal yang dipilih	Berhasil
96	LOG-009	Filter User	Admin mem-filter log berdasarkan pengguna yang melakukan aktivitas	Sistem menampilkan log sesuai pengguna yang dipilih	Berhasil
97	LOG-010	Log Perubahan	Sistem mencatat detail perubahan pada aksi update	Perubahan field tercatat dalam log	Berhasil
98	LOG-011	Pencarian Log	Admin mencari log berdasarkan kata kunci dalam deskripsi	Sistem menampilkan log yang mengandung kata kunci	Berhasil
99	AKSES-001	Admin Sensor	Admin mengakses daftar sensor	Admin dapat melihat daftar sensor	Berhasil
100	AKSES-002	Admin Sensor	Admin menambahkan sensor baru	Admin dapat menambahkan sensor	Berhasil
101	AKSES-003	Admin Sensor	Admin mengedit data sensor	Admin dapat mengedit sensor	Berhasil
102	AKSES-004	Admin Sensor	Admin menghapus sensor	Admin dapat menghapus sensor	Berhasil

No	Kode	Fitur	Skenario	Ekspektasi	Hasil
103	AKSES-005	Admin Aktuator	Admin mengakses daftar aktuator	Admin dapat melihat daftar aktuator	Berhasil
104	AKSES-006	Admin Aktuator	Admin melakukan toggle aktuator	Admin dapat mengaktifkan/menonaktifkan aktuator	Berhasil
105	AKSES-007	Admin Otomasi Jadwal	Admin mengakses daftar otomatisasi jadwal	Admin dapat melihat daftar otomatisasi jadwal	Berhasil
106	AKSES-008	Admin Otomasi Jadwal	Admin menambahkan otomatisasi jadwal	Admin dapat menambahkan otomatisasi jadwal	Berhasil
107	AKSES-009	Admin Otomasi Jadwal	Admin menyalakan/mematikan otomatisasi jadwal	Admin dapat mengubah status aktif otomatisasi jadwal	Berhasil
108	AKSES-010	Admin Otomasi Sensor	Admin mengakses daftar otomatisasi sensor	Admin dapat melihat daftar otomatisasi sensor	Berhasil
109	AKSES-011	Admin Otomasi Sensor	Admin menyalakan/mematikan otomatisasi sensor	Admin dapat mengubah status aktif otomatisasi sensor	Berhasil
110	AKSES-012	Admin Otomasi Robot	Admin mengakses daftar otomatisasi robot	Admin dapat melihat daftar otomatisasi robot	Berhasil
111	AKSES-013	Admin Otomasi Robot	Admin menambahkan otomatisasi robot	Admin dapat menambahkan otomatisasi robot	Berhasil
112	AKSES-014	Admin Otomasi Robot	Admin menyalakan/mematikan otomatisasi robot	Admin dapat mengubah status aktif otomatisasi robot	Berhasil
113	AKSES-015	Admin User	Admin mengakses daftar pengguna	Admin dapat melihat daftar pengguna	Berhasil
114	AKSES-016	Admin Download	Admin mengunduh data	Admin dapat mengunduh data	Berhasil
115	AKSES-017	Admin Log Aktivitas	Admin mengakses halaman log aktivitas	Admin dapat melihat seluruh log aktivitas sistem	Berhasil
116	AKSES-018	Pengelola Sensor	Pengelola mengakses daftar sensor	Pengelola dapat melihat daftar sensor	Berhasil
117	AKSES-019	Pengelola Sensor	Pengelola menambahkan sensor baru	Pengelola dapat menambahkan sensor	Berhasil
118	AKSES-020	Pengelola Sensor	Pengelola mengedit data sensor	Pengelola dapat mengedit sensor	Berhasil
119	AKSES-021	Pengelola Sensor	Pengelola menghapus sensor	Pengelola dapat menghapus sensor	Berhasil
120	AKSES-022	Pengelola Aktuator	Pengelola mengakses daftar aktuator	Pengelola dapat melihat daftar aktuator	Berhasil
121	AKSES-023	Pengelola Aktuator	Pengelola melakukan toggle aktuator	Pengelola dapat mengaktifkan/menonaktifkan aktuator	Berhasil

No	Kode	Fitur	Skenario	Ekspektasi	Hasil
122	AKSES-024	Pengelola Otomasi Jadwal	Pengelola mengakses daftar otomatisasi jadwal	Pengelola dapat melihat daftar otomatisasi jadwal	Berhasil
123	AKSES-025	Pengelola Otomasi Jadwal	Pengelola menambahkan otomatisasi jadwal	Pengelola dapat menambahkan otomatisasi jadwal	Berhasil
124	AKSES-026	Pengelola Otomasi Jadwal	Pengelola menyalakan/mematikan otomatisasi jadwal	Pengelola dapat mengubah status aktif otomatisasi jadwal	Berhasil
125	AKSES-027	Pengelola Otomasi Sensor	Pengelola mengakses daftar otomatisasi sensor	Pengelola dapat melihat daftar otomatisasi sensor	Berhasil
126	AKSES-028	Pengelola Otomasi Sensor	Pengelola menyalakan/mematikan otomatisasi sensor	Pengelola dapat mengubah status aktif otomatisasi sensor	Berhasil
127	AKSES-029	Pengelola Otomasi Robot	Pengelola mengakses daftar otomatisasi robot	Pengelola dapat melihat daftar otomatisasi robot	Berhasil
128	AKSES-030	Pengelola Otomasi Robot	Pengelola menambahkan otomatisasi robot	Pengelola dapat menambahkan otomatisasi robot	Berhasil
129	AKSES-031	Pengelola Otomasi Robot	Pengelola mengedit otomatisasi robot	Pengelola dapat mengedit otomatisasi robot	Berhasil
130	AKSES-032	Pengelola Otomasi Robot	Pengelola menghapus otomatisasi robot	Pengelola dapat menghapus otomatisasi robot	Berhasil
131	AKSES-033	Pengelola Otomasi Robot	Pengelola menyalakan/mematikan otomatisasi robot	Pengelola dapat mengubah status aktif otomatisasi robot	Berhasil
132	AKSES-034	Pengelola User	Pengelola mencoba mengakses daftar pengguna	Pengelola tidak dapat mengakses (dibatasi)	Berhasil
133	AKSES-035	Pengelola Download	Pengelola mengunduh data	Pengelola dapat mengunduh data	Berhasil
134	AKSES-036	Pengelola Log Aktivitas	Pengelola mencoba mengakses halaman log aktivitas	Pengelola tidak dapat mengakses (dibatasi, hanya admin)	Berhasil
135	AKSES-037	Staff Sensor	Staff mengakses daftar sensor	Staff dapat melihat daftar sensor	Berhasil
136	AKSES-038	Staff Sensor	Staff mencoba menambahkan sensor	Staff tidak dapat menambahkan (dibatasi)	Berhasil
137	AKSES-039	Staff Sensor	Staff mencoba mengedit sensor	Staff tidak dapat mengedit (dibatasi)	Berhasil
138	AKSES-040	Staff Sensor	Staff mencoba menghapus sensor	Staff tidak dapat menghapus (dibatasi)	Berhasil
139	AKSES-041	Staff Aktuator	Staff mengakses daftar aktuator	Staff dapat melihat daftar aktuator	Berhasil
140	AKSES-042	Staff Aktuator	Staff melakukan toggle aktuator	Staff dapat mengaktifkan/menonaktifkan aktuator	Berhasil

No	Kode	Fitur	Skenario	Ekspektasi	Hasil
141	AKSES-043	Staff Otomasi Jadwal	Staff mengakses daftar otomatisasi jadwal	Staff dapat melihat daftar otomatisasi jadwal	Berhasil
142	AKSES-044	Staff Otomasi Jadwal	Staff mencoba menambahkan otomatisasi jadwal	Staff tidak dapat menambahkan (dibatasi)	Berhasil
143	AKSES-045	Staff Otomasi Jadwal	Staff mencoba menyalakan/mematikan otomatisasi jadwal	Staff tidak dapat mengubah status aktif (dibatasi)	Berhasil
144	AKSES-046	Staff Otomasi Sensor	Staff mengakses daftar otomatisasi sensor	Staff dapat melihat daftar otomatisasi sensor	Berhasil
145	AKSES-047	Staff Otomasi Sensor	Staff mencoba menyalakan/mematikan otomatisasi sensor	Staff tidak dapat mengubah status aktif (dibatasi)	Berhasil
146	AKSES-048	Staff Otomasi Robot	Staff mencoba mengakses daftar otomatisasi robot	Staff tidak dapat mengakses (dibatasi)	Berhasil
147	AKSES-049	Staff Otomasi Robot	Staff mencoba menyalakan/mematikan otomatisasi robot	Staff tidak dapat mengubah status aktif (dibatasi)	Berhasil
148	AKSES-050	Staff User	Staff mencoba mengakses daftar pengguna	Staff tidak dapat mengakses (dibatasi)	Berhasil
149	AKSES-051	Staff Download	Staff mencoba mengunduh data	Staff tidak dapat mengunduh (dibatasi)	Berhasil
150	AKSES-052	Staff Log Aktivitas	Staff mencoba mengakses halaman log aktivitas	Staff tidak dapat mengakses (dibatasi, hanya admin)	Berhasil
151	LOGOUT-001	Logout	Pengguna yang sudah login menekan tombol logout	Sesi pengguna berakhir dan diarahkan ke halaman login	Berhasil

G. Rincian Pengujian Komunikasi Data dan Otomatisasi



Pengujian Komunikasi Data

Pengujian (ms)	P1	P2	P3	P4	P5	P6	P7	P8	P9	P10	Rata-rata	Max	Min	Standar Deviasi
Pengujian Manual Perangkat Sensor (QoS 0, Retain= False)														
MQTT-01	106	107	110	110	102	112	106	108	103	109	107.3	112	102	3.16
MQTT-02	103	102	107	108	107	102	106	108	108	108	105.9	108	102	2.56
MQTT-03	109	108	103	105	103	101	109	105	100	108	105.1	109	100	3.31
MQTT-04	106	105	110	105	102	109	107	104	108	103	105.9	110	102	2.6
MQTT-05	106	110	109	110	101	100	102	106	102	107	105.3	110	100	3.8
Pengujian Manual Perangkat Aktuator (QoS 1, Retain= True)														
MQTT-06	110	103	102	100	104	106	106	102	106	105	104.4	110	100	2.84
MQTT-07	109	107	116	110	101	107	108	106	108	104	107.6	116	101	3.92
MQTT-08	110	107	105	102	106	109	109	102	102	105	105.7	110	102	3.06
MQTT-09	105	106	103	109	107	109	102	103	105	108	105.7	109	102	2.54
MQTT-10	101	107	102	107	117	108	103	109	103	108	106.5	117	101	4.67
MQTT-11	100	107	108	110	101	102	100	114	104	107	105.3	114	100	4.69
MQTT-12	109	101	105	108	108	108	110	104	108	103	106.4	110	101	2.95
Pengujian Manual Data Robot (QoS 1, Retain= False)														
MQTT-13	104	110	101	109	101	108	101	104	102	112	105.2	112	101	4.18

Pengujian Otomatisasi

Perangkat Aktuator ON															
Kode	Pengujian (ms)	P1	P2	P3	P4	P5	P6	P7	P8	P9	P10	Rata-rata	Max	Min	Std Dev
OS-01	Exhaust Fan	4950	3000	2050	2080	2040	2060	2090	2070	2060	2100	2450	4950	2040	856.4
OS-02	Cooling Pad	4800	4500	3250	3220	3280	3240	3260	3210	3240	3500	3550	4800	3210	536.2
OS-03	Katup Air	4200	2500	740	730	760	750	745	735	670	670	1250	4200	670	1084.5
OS-04	Pompa Nutrisi	3800	2200	460	470	455	465	480	450	510	510	980	3800	450	1025.6
OP-01	Jadwal Fan	4100	2800	1150	1160	1140	1170	1130	1160	1150	1260	1622	4100	1130	908.4
OP-02	Jadwal CoolPad	4150	2850	1180	1190	1170	1200	1160	1190	1180	1130	1640	4150	1130	915.2
OP-03	Jadwal Agitator	4120	2820	1185	1195	1175	1205	1165	1195	1185	1205	1645	4120	1165	905.8
OP-04	Jadwal Bahan	4200	2900	1200	1210	1190	1220	1180	1210	1200	1310	1682	4200	1180	920.5
OP-05	Jadwal Siram	4050	2750	1140	1150	1130	1160	1120	1150	1140	1190	1598	4050	1120	895.6
OR-01	Siram Robot*	3500	2200	1120	1130	1110	1140	1105	1125	1115	1105	1465	3500	1105	732.6

OFF															
Kode	Pengujian (ms)	P1	P2	P3	P4	P5	P6	P7	P8	P9	P10	Rata-rata	Max	Min	Std Dev
OS-01	Exhaust Fan	1250	800	140	145	135	150	130	140	145	165	320	1250	130	358.4
OS-02	Cooling Pad	1600	900	200	205	195	210	190	200	205	195	410	1600	190	435.2
OS-03	Katup Air	1100	700	120	125	115	130	110	120	125	155	280	1100	110	315.6
OS-04	Pompa Nutrisi	650	400	60	65	55	70	50	60	65	75	155	650	50	185.3
OP-01	Jadwal Fan	4400	2900	1280	1290	1270	1300	1260	1290	1280	1230	1750	4400	1230	960.5
OP-02	Jadwal CoolPad	4300	2850	1250	1260	1240	1270	1230	1260	1250	1240	1715	4300	1230	935.2
OP-03	Jadwal Agitator	4100	2700	1160	1170	1150	1180	1140	1170	1160	1170	1610	4100	1140	890.6
OP-04	Jadwal Bahan	4250	2800	1200	1210	1190	1220	1180	1210	1200	1190	1665	4250	1180	925.8
OP-05	Jadwal Siram	4350	2850	1230	1240	1220	1250	1210	1240	1230	1200	1702	4350	1200	950.4
OR-01	Siram Robot*	3800	2250	1160	1170	1150	1180	1140	1170	1160	1280	1546	3800	1140	830.5

Log Lengkap

No	Waktu (WIB)	Kode	Skenario Pengujian	Trigger / Payload Masukan	Aksi Aktuator (Output)	Latensi (ms)	Status
I	SESI PAGI (06:00 - 11:00)						
1	6:55:00	[OP-03]	Jadwal Agitator Mix (Start)	Cron Job: 06:55	[A-03] Agitator Mix ON	1645	Sukses
2	6:55:00	[OP-04]	Jadwal Agitator Bahan (Start)	Cron Job: 06:55	[A-04] Agitator Bahan ON	1682	Sukses
3	6:55:00	[OP-05]	Jadwal Siram Interval (Start)	Cron Job: 06:55	[A-07] Pompa Siram ON	1598	Sukses
4	7:00:00	[OP-03]	Jadwal Agitator Mix (Stop)	Timer Finish (300s)	[A-03] Agitator Mix OFF	1610	Sukses
5	7:00:00	[OP-04]	Jadwal Agitator Bahan (Stop)	Timer Finish (300s)	[A-04] Agitator Bahan OFF	1665	Sukses
6	7:00:00	[OP-05]	Jadwal Siram Interval (Stop)	Timer Finish (300s)	[A-07] Pompa Siram OFF	1702	Sukses
7	7:00:00	[OP-01]	Jadwal Exhaust Fan (Start)	Cron Job: 07:00	[A-01] Exhaust Fan ON	1622	Sukses
8	7:00:00	[OP-02]	Jadwal Cooling Pad (Start)	Cron Job: 07:00	[A-02] Cooling Pad ON	1640	Sukses
9	8:15:20	[OS-01]	Sensor Suhu Tinggi	Payload: 35.5 (°C)	[A-01] Exhaust Fan ON	2450	Sukses
10	8:45:10	[OS-01]	Sensor Suhu Normal	Payload: 25.0 (°C)	[A-01] Exhaust Fan OFF	320	Sukses
11	9:30:00	[OR-01]	Robot Siram (Trigger Awal)	Payload: 380 (ml)	[A-07] Pompa Siram ON	845	Sukses
12	9:35:00	[OR-01]	Robot Siram (Stop)	Timer Finish (300s)	[A-07] Pompa Siram OFF	1120	Sukses
13	10:20:15	[OS-02]	Sensor Kelembapan Rendah	Payload: 60 (%)	[A-02] Cooling Pad ON	3550	Sukses
II SESI SIANG (11:00 - 15:00)							
14	11:05:40	[OS-02]	Sensor Kelembapan Tinggi	Payload: 80 (%)	[A-02] Cooling Pad OFF	410	Sukses
15	11:30:00	[OR-01]	Repetisi Robot #1 (Start)	Auto-Schedule (Interval 2h)	[A-07] Pompa Siram ON	1625	Sukses
16	11:35:00	[OR-01]	Repetisi Robot #1 (Stop)	Timer Finish (300s)	[A-07] Pompa Siram OFF	1650	Sukses
17	13:10:05	[OS-03]	Sensor Tandon Kosong	Payload: 5 (%)	[A-06] Katup Air ON	1250	Sukses
18	13:30:00	[OR-01]	Repetisi Robot #2 (Start)	Auto-Schedule (Interval 2h)	[A-07] Pompa Siram ON	1610	Sukses
19	13:35:00	[OR-01]	Repetisi Robot #2 (Stop)	Timer Finish (300s)	[A-07] Pompa Siram OFF	1645	Sukses
20	14:45:30	[OS-03]	Sensor Tandon Penuh	Payload: 90 (%)	[A-06] Katup Air OFF	280	Sukses
III	SESI SORE (15:00 - 18:00)						
21	15:30:00	[OR-01]	Repetisi Robot #3 (Start)	Auto-Schedule (Interval 2h)	[A-07] Pompa Siram ON	1630	Sukses

22	15:30:22	[OS-04]	Sensor Nutrisi Rendah	Payload: 100 (PPM)	[A-05] Pompa Nutrisi ON	980	Sukses
23	15:35:00	[OR-01]	Repetisi Robot #3 (Stop)	Timer Finish (300s)	[A-07] Pompa Siram OFF	1660	Sukses
24	15:55:10	[OS-04]	Sensor Nutrisi Pekat	Payload: 1200 (PPM)	[A-05] Pompa Nutrisi OFF	155	Sukses
25	17:00:00	[OP-01]	Jadwal Exhaust Fan (Stop)	Time Limit (10 Jam)	[A-01] Exhaust Fan OFF	1750	Sukses
26	17:00:00	[OP-02]	Jadwal Cooling Pad (Stop)	Time Limit (10 Jam)	[A-02] Cooling Pad OFF	1715	Sukses
27	17:30:00	[OR-01]	Repetisi Robot #4 (Start)	Auto-Schedule (Interval 2h)	[A-07] Pompa Siram ON	1615	Sukses
28	17:35:00	[OR-01]	Repetisi Robot #4 (Stop)	Timer Finish (300s)	[A-07] Pompa Siram OFF	1655	Sukses

312

Plaintext

AllReceivedPublished

Topic: iterahero/kebun-raya-itera/actuator/actuator-ct6x/cmd

QoS: 1

Retained

ON

2026-01-08 07:14:11:879

Waktu08 Jan 2026, 07:00:07

StatusOn

Dipicu OlehPengisian

Pengujian Otomatisasi Penjadwalan

Topic: iterahero/kebun-raya-itera/sensor/sensor-nywh/data QoS: 0
21.5

2026-01-07 19:51:08:051

Topic: iterahero/kebun-raya-itera/sensor/sensor-nywh/data QoS: 0
21.5

2026-01-07 19:51:08:217

Topic: iterahero/kebun-raya-itera/actuator/actuator-p9zb/cmd QoS: 1 Retained
OFF
2026-01-07 19:51:08:716

2026-01-07 19:51:08:963

Plaintext ▼ QoS 0 ▼ ☐ Retain Meta ▲

< 10/10 >

iterahero/kebun-raya-itera/sensor/sensor-nywh/data ▼

21.5

Pengujian Otomatisasi Sensor

Topic: iterahero/kebun-raya/robot/robot-tes-1/data/mltanaman QoS: 0
380

2026-01-07 20:26:08:467

Topic: iterahero/kebun-raya/robot/robot-tes-1/data/mltanaman QoS: 0
380

2026-01-07 20:26:09:413

Topic: iterahero/kebun-raya-itera/actuator/actuator-p9zb/cmd QoS: 1 Retained
OFF
2026-01-07 20:26:22:163

2026-01-07 20:26:22:516

Topic: iterahero/kebun-raya-itera/actuator/actuator-p9zb/cmd QoS: 1 Retained
ON

Pengujian Otomatisasi Data Definisi Robot